



综合论文训练

Rust 异步时间驱动模块设计与实现

系 别 : 计算机系
专 业 : 计算机科学与技术
姓 名 : 黄 昊 颖
指导教师 : 向 勇 副研究员

二〇二六年五月

关于论文使用授权的说明

本人完全了解清华大学有关保留、使用综合论文训练论文的规定，即：学校有权保留论文的复印件，允许论文被查阅和借阅；学校可以公布论文的全部或部分内容，可以采用影印、缩印或其他复制手段保存论文。

作者签名：

黄昊颖

导师签名：

何勇

日 期： 2026-05-28

日 期： 2026-05-28

摘 要

随着操作系统内核逐渐面向多核并发、低延迟 I/O 和复杂外设管理场景发展，传统同步阻塞式驱动模型在资源利用率和并发扩展性方面逐渐暴露出局限^[1-3]。Rust 语言提供的 `async/await`、`Future` 和 `Waker` 机制，为在系统软件中构建轻量级异步任务模型提供了新的可能^[4-6]。相比传统线程模型，`Rust Future` 属于无栈协程，其挂起状态由编译器生成的状态机保存，不需要为每个异步任务维护独立运行栈，因此适合嵌入式系统和操作系统内核等资源敏感场景。

本文围绕 Rust 异步驱动模块的设计与实现展开，以 `rCore-N` 为实验平台，在详细分析 `Rust Future`、`Waker`、`Embassy` 异步运行时以及 `rCore-N` 相关机制的基础上，设计并实现了一个异步定时器驱动原型。该驱动将 Rust 异步编程模型引入 `rCore-N` 内核 `timer` 路径，使定时事件能够以异步任务唤醒的形式被处理，从而验证 Rust 异步机制在操作系统内核定时器场景中的适用性。

本文进一步通过实验对异步定时器驱动进行验证。主要测试多核环境下的 `sleep` 响应。实验结果表明，本文实现的异步定时器驱动能够完成定时等待、到期唤醒和任务恢复执行等基本功能，说明该异步定时器在 `rCore-N` 内核中是可用的。

关键词：Rust；Future；异步驱动；`rCore-N`；Timer

Abstract

As operating system kernels increasingly evolve toward multi-core concurrency, low-latency I/O, and complex peripheral management, traditional synchronous blocking driver models have gradually exposed limitations in resource utilization and concurrency scalability ^[1-3]. The `async/await`, `Future`, and `Waker` mechanisms provided by Rust offer new possibilities for building lightweight asynchronous task models in system software ^[4-6]. Compared with traditional thread models, Rust `Future` is a stackless coroutine: its suspended state is stored in a compiler-generated state machine, and no independent execution stack needs to be maintained for each asynchronous task. Therefore, it is well suited to resource-sensitive scenarios such as embedded systems and operating system kernels.

Focusing on the design and implementation of Rust asynchronous driver modules, this thesis takes `rCore-N` as the experimental platform. On the basis of a detailed analysis of Rust `Future`, `Waker`, the Embassy asynchronous runtime, and related mechanisms in `rCore-N`, this thesis designs and implements an asynchronous timer driver prototype. The driver introduces Rust's asynchronous programming model into the kernel timer path of `rCore-N`, enabling timer events to be handled in the form of asynchronous task wake-up, thereby verifying the applicability of Rust asynchronous mechanisms in operating system kernel timer scenarios.

This thesis further validates the asynchronous timer driver through experiments, focusing mainly on sleep response in a multi-core environment. The experimental results show that the implemented asynchronous timer driver can complete basic functions such as timed waiting, expiration wake-up, and task resumption, indicating that the asynchronous timer is usable in the `rCore-N` kernel.

Keywords: Rust; `Future`; Asynchronous Driver; `rCore-N`; Timer

目 录

第 1 章 绪论	1
1.1 研究背景	1
1.2 国内外研究现状	2
1.3 本文主要内容	3
1.4 本文结构	4
第 2 章 相关技术介绍	5
2.1 Rust Future 无栈协程	5
2.1.1 无栈状态机模型	5
2.1.2 核心抽象：基于轮询的 Future Trait	5
2.1.3 唤醒机制与 Reactor-Executor 模型	6
2.1.4 异步运行时的解耦特性	6
2.2 Embassy 介绍	7
2.2.1 Embassy 的任务模型	7
2.2.2 执行器与任务调度	8
2.2.3 Waker、TaskRef 与任务唤醒	8
2.2.4 Embassy 的定时器等待机制概述	9
2.2.5 中断执行器与 pender() 机制	10
2.2.6 对本文异步驱动设计的启示	10
2.3 rCore 介绍	11
2.3.1 基于 Rust 的内核实现	11
2.3.2 rCore 的核心内核结构	12
2.3.3 rCore 的 timer 与阻塞式 sleep 机制	12
2.3.4 rCore 与本文工作的关系	13
2.3.5 关于 rCore-N	13
2.4 本章小结	15
第 3 章 timer 基本机制	16
3.1 timer 简介	16
3.2 rCore timer 实现	17
3.2.1 rCore-2025S 的固定 tick 与阻塞式 sleep	17
3.2.2 rCore-N 的 virtual timer 机制	19

3.2.3 rCore-N 中用户态 timer 事件投递	20
3.2.4 rCore-N 中 sleep 机制的局限性	21
3.2.5 小结	22
3.3 Embassy timer 实现	22
3.3.1 Timer Future 的 poll 过程	23
3.3.2 时间驱动接口与平台实现	23
3.3.3 timer queue 与 Waker 到 TaskRef 的转换	24
3.3.4 硬件 alarm 设置与中断处理	26
3.3.5 到期任务的唤醒与重新入队	26
3.3.6 从中断唤醒到 executor 再次 poll	27
3.4 本章小结	30
第 4 章 异步驱动模块	31
4.1 异步 timer 设计与实现	31
4.1.1 设计目标与 rCore-N 约束	32
4.1.2 基于 blocked 的 sys_sleep 初始设想及放弃原因	32
4.1.3 当前实现：基于 suspend-loop 的 sys_sleep	33
4.1.4 基于 Future 的异步 timer 抽象	34
4.1.5 TaskRef、TaskHeader 与 Waker 的转换关系	34
4.1.6 定时队列与 Time Driver 设计	36
4.1.7 S Timer Trap 中统一执行 wake 与 poll	37
4.1.8 小结	38
4.2 异步 timer 正确性分析与响应测试	39
4.2.1 Rust Future 无栈协程分析	40
4.2.2 多核下 sleep 响应误差测试	41
4.3 本章小结	47
第 5 章 总结和展望	48
5.1 论文总结	48
5.2 目前实现的问题与展望	49
5.2.1 异步 timer 的问题	49
5.2.2 后续推进设想	50
参考文献	53
附录 A 外文资料书面翻译 1	55
附录 B 外文资料书面翻译 2	76

致 谢	96
声 明	97

插图清单

图 3.1 Embassy 中 timer queue、TaskRef 与 run queue 的关系示意图	25
图 3.2 以 embassy-rp 为例的中断模式 Timer Future 挂起与恢复执行关键链路图 ..	29
图 4.1 rCore-2025S、rCore-N 和 async_timer 三种 timer 实现的对比图	39
图 4.2 Future 返回 Poll::Pending 后调用栈逐层回退示意图	41

附表清单

表 4.1 不同目标等待时间下 sleep_blocking 的响应结果	43
表 4.2 不同 CPU load 压力下 sleep_blocking 的响应结果	44
表 4.3 不同目标时间并发 sleep 的返回顺序测试结果	46
表 4.4 不同目标时间并发 sleep 的返回顺序统计结果	46

符号和缩略语说明

CPU	中央处理器 (Central Processing Unit)
I/O	输入/输出 (Input/Output)
OS	操作系统 (Operating System)
SBI	监督二进制接口 (Supervisor Binary Interface)
RISC-V	开源精简指令集架构
RP2040	Embassy 示例中使用的树莓派微控制器平台

第 1 章 绪论

1.1 研究背景

操作系统内核需要同时处理进程调度、外设中断、系统调用、定时器事件以及 I/O 请求等多类并发事件。传统内核通常采用同步阻塞式接口组织这些逻辑：当任务等待某个事件尚未完成时，可能通过忙等、阻塞挂起或轮询方式等待事件发生。对于教学操作系统而言，这种设计结构清晰、实现直接，便于理解操作系统的基本原理；但在多核并发和复杂 I/O 场景下，传统同步模型容易带来额外的 CPU 空转、上下文切换开销以及共享数据结构竞争问题^[1-3]。

近年来，异步编程模型在用户态高性能网络服务、嵌入式系统和系统软件中得到广泛关注^[2-3,7-8]。Rust 语言原生支持 `async/await` 语法，并通过 `Future trait`、`Poll` 状态和 `Waker` 机制构建了基于轮询的异步执行模型^[4-6]。与传统线程相比，Rust `Future` 是一种无栈协程：异步任务在挂起时不会保存独立调用栈，而是将跨越 `await` 点的局部状态保存到编译器生成的状态机对象中。这种模型具有较低的内存占用和较强的可组合性，为在操作系统内核中实现轻量级异步驱动提供了可能。

在嵌入式 Rust 生态中，Embassy 已经较完整地展示了 `Future`、`Waker`、执行器、中断和硬件 `timer` 结合的设计方式^[9-11]。Embassy 的 `timer` 机制能够在任务等待定时事件时注册 `Waker`，并在硬件 `timer` 中断到来后唤醒对应任务，使其重新进入执行器运行队列。该机制说明，异步 `timer` 并不是简单的延时函数，而是需要建立从 `Future`、`Waker`、`timer queue`、硬件中断到 `executor poll` 的完整唤醒链路。

rCore 是使用 Rust 语言实现的教学操作系统内核，而 rCore-N 是在 rCore 基础上进一步扩展的研究型分支^[12-14]。与标准 rCore 相比，rCore-N 引入了多核执行、用户态中断、`virtual timer` 和异步串口等机制，同时也删去了标准 rCore 中基于 `Blocked` 状态和定时器等待队列的 `sys_sleep` 实现。rCore-N 用户库中的 `sleep` 更接近用户态忙等式延时，而不是内核调度意义上的阻塞睡眠。因此，在 rCore-N 中重新讨论基于 Rust `Future` 的异步 `timer` 和 `sleep` 机制，既有助于探索 Rust 异步模型在内核中的适用性，也能暴露多核调度、`timer` 中断和任务唤醒之间的工程约束。

基于上述背景，本文以 rCore-N 为实验平台，尝试设计并实现一个内核级异步 `timer` 原型模块。该模块借鉴 Embassy `timer` 的核心思想，但不直接移植 Embassy 执行器，而是结合 rCore-N 的 `S Timer Trap`、调度器和多核共享队列特点

进行适配。本文希望通过这一实验回答以下问题：Rust Future/Waker 机制能否接入 rCore-N 的 timer 路径？异步 timer 如何在内核中完成任务挂起与唤醒？在多核共享调度队列和中断上下文限制下，传统 blocked sleep 与异步 sleep 的实现会遇到哪些工程取舍？

1.2 国内外研究现状

异步 I/O 和事件驱动机制是操作系统和高性能服务端系统中的重要研究方向 [1-3,7,15-17]。传统操作系统中，异步 I/O 通常通过中断、等待队列、事件通知和回调机制实现。例如，当设备尚未就绪时，内核可以将任务挂入等待队列；当设备中断到来后，再由中断处理逻辑唤醒等待任务。这类机制在 Linux 等成熟内核中已经长期存在，但其接口和实现通常与 C 语言、线程调度、等待队列和内核同步机制紧密绑定。

在用户态编程领域，异步编程模型已经形成了较成熟的生态 [2-3,7-8]。例如 JavaScript 使用事件循环和回调机制组织异步任务，Go 使用轻量级 goroutine 和运行时调度器管理并发，Rust 则采用基于 Future 的零成本异步抽象。Rust 的异步模型并不规定具体运行时，而是将任务状态机、轮询接口和唤醒机制分离出来，由 Tokio、async-std、Embassy 等运行时根据不同场景实现调度器和 reactor [4,9,18]。这种解耦设计使 Rust 异步机制既可以用于用户态网络服务，也可以被移植到 no_std 嵌入式和内核环境中。

在嵌入式领域，Embassy 是较有代表性的 Rust 异步运行时 [9-10]。它面向 no_std 环境提供异步任务、执行器、定时器和外设驱动支持。Embassy 的 timer 机制通过 Timer Future 表达定时等待，通过 time driver 管理硬件 timer，通过 Waker 或 TaskRef 将到期事件映射回等待任务。这种设计避免了传统阻塞等待和忙轮询，使嵌入式程序能够以较低资源开销表达并发逻辑。Embassy 的实现对本文具有直接参考意义，尤其是在 Timer Future、Waker、timer queue、硬件中断和 executor 之间的协作关系方面。

在操作系统教学和研究领域，rCore 展示了 Rust 语言在内核实现中的可行性 [12-13]。标准 rCore-Tutorial 主要面向教学，采用较直接的同步阻塞式机制组织任务调度、timer 和 sys_sleep。rCore-N 在 rCore 基础上进行了多核、用户态中断和异步 I/O 方向的扩展，为探索 Rust 异步模型在内核中的应用提供了更合适的平台。不过，rCore-N 的现有 sleep 机制仍然存在忙等或工程简化问题，并没有形成完整的 Future/Waker 异步 timer 模型。因此，如何在 rCore-N 中引入类似 Embassy 的异步 timer 思路，并与内核 S Timer Trap、调度器和多核同步机制结合，是本文关

注的核心问题。

总体来看，现有研究和工程实践已经分别在用户态异步运行时、嵌入式异步框架和 Rust 教学内核方面积累了较多经验，但将 Rust Future/Waker 机制直接接入多核操作系统内核 timer 路径仍然具有探索价值。本文的工作并不试图构建完整通用的异步内核运行时，而是以 timer 和 sleep 为切入点，研究 Future/Waker 异步模型在 rCore-N 内核中的具体适配方式、可行性与局限性。

1.3 本文主要内容

本文围绕 rCore-N 中异步 timer 模块的设计与实现展开，主要内容包括以下三个方面。

第一，分析已有异步机制及相关 timer 实现。本文首先介绍 Rust Future 无栈协程模型、Poll 语义和 Waker 唤醒机制，说明 Rust 异步任务如何通过状态机保存挂起状态，并由执行器重新推进执行。随后，本文分析 Embassy 异步运行时中的任务模型、执行器、timer queue 和中断唤醒链路，提炼出异步 timer 所需的 Future、Waker、定时队列、硬件中断和 executor poll 之间的协作关系。在此基础上，本文进一步比较 rCore-2025S 与 rCore-N 的 timer 机制和 sleep 语义差异，为后续在 rCore-N 中引入异步 timer 提供背景。

第二，在 rCore-N 中设计并实现异步 timer 原型。本文结合 rCore-N 的多核调度结构、S Timer Trap 语义和共享 ready_queue 约束，实现了一个基于 Rust Future/Waker 的异步 timer 模块。该模块包括 Timer Future、每 hart 独立的 TimerDriver 与 Executor、基于 Vec 的 timer queue、TaskHeader/TaskRef/RawWaker 唤醒链路等部分。针对 rCore-N 中直接恢复 blocked sleep 和直接照搬 Embassy 中断执行器会遇到的上下文问题，本文采用 suspend-loop sys_sleep 与 S Timer Trap 内统一 wake/poll 的工程折中方案，完成了从 timer 到期事件到异步任务恢复执行的完整链路。

第三，对异步 timer 模块进行理论分析和实验验证。本文通过对 Rust Future 无栈协程调用栈变化进行代码层面的理论分析，说明 Poll::Pending 返回后调用栈会回退到执行器附近，说明 Future 挂起时保存状态而非独立栈的特征；通过基础响应误差测试、并发 sleep 与 CPU load 压力测试、不同到期顺序测试，验证 sleep_blocking 在 rCore-N 多核环境下的基本可用性。实验主要关注 timer 是否能够正确注册、到期、唤醒和恢复执行，以及多核调度压力下 sleep 响应误差是否处于可解释范围内。

1.4 本文结构

本文共分为五章，各章内容安排如下。

第一章从操作系统内核中同步等待机制的局限出发，引出在 rCore-N 中引入 Rust Future/Waker 异步 timer 的研究目标，并概述本文的研究思路与章节安排。

第二章为相关技术介绍，首先介绍 Rust Future 无栈协程模型，包括 Future trait、Poll 状态、Waker 唤醒机制和异步运行时解耦特性；随后介绍 Embassy 异步运行时，重点说明其任务模型、执行器、Waker、TaskRef 以及中断执行器机制；最后介绍 rCore 和 rCore-N 的基本结构，说明 rCore-N 的多核、用户态中断、virtual timer 和 busy-wait sleep 等机制为本文工作提供的实验背景。

第三章为 timer 基本机制，首先介绍 RISC-V timer 的基本硬件机制和操作系统软件 timer 的组织方式；然后分析 rCore-2025S 的固定 tick 与阻塞式 sys_sleep，以及 rCore-N 的 virtual timer、用户态 timer 事件投递和 sleep 机制局限；最后结合 Embassy timer 的实现，进一步说明异步 timer 的 Future/Waker 唤醒链路，为第四章的实现设计提供参考。

第四章为异步时间驱动模块设计与实现，详细说明本文在 rCore-N 中实现的异步 timer 原型。该章包括设计目标与约束、blocked sleep 初始设想及放弃原因、suspend-loop sys_sleep 折中实现、Timer Future、TaskRef/TaskHeader/Waker 转换、timer queue 与 time driver、S Timer Trap 中统一 wake 与 poll 等内容，并通过实验验证异步 timer 的无栈协程特性与多核下 sleep 响应行为。

第五章为总结和展望，首先总结本文完成的主要工作和实验结论；随后分析当前实现中仍存在的问题，包括 suspend-loop sleep 的效率局限、timer queue 线性扫描、任务生命周期管理不完善以及中断上下文执行 poll 的开销等；最后提出后续进一步完善 rCore-N 异步 timer 和异步驱动框架的可能方向。

第 2 章 相关技术介绍

2.1 Rust Future 无栈协程

在现代操作系统与高性能网络编程中，协程（Coroutine）作为一种用户态的轻量级并发单元，因其上下文切换开销远低于操作系统线程而被广泛应用 [3,8,19]。根据实现方式的不同，协程主要分为有栈协程（Stackful Coroutine）与无栈协程（Stackless Coroutine）。Rust 语言在构建其异步编程模型时，基于零成本抽象（Zero-cost Abstraction）的设计哲学，最终确立了以 Future Trait 为核心的无栈协程机制 [4-5]。

2.1.1 无栈状态机模型

与 Go 语言中的 Goroutine 等有栈协程不同，Rust 的无栈协程不会为每个并发任务分配独立的运行栈空间。在 Rust 中，使用 `async` 关键字修饰的函数或代码块，在编译期会被 Rust 编译器（`rustc`）自动转换为一个实现了 Future 特型的匿名状态机（State Machine）[4-5]。

该状态机本质上是一个枚举（Enum）类型，其变体（Variant）对应了协程在各个 `await` 挂起点的状态，并负责保存跨越 `await` 点的所有局部变量。由于不依赖系统级的栈空间分配与切换，Rust 的无栈协程在创建和切换时的内存占用极小（通常仅需数十字节到数百字节），且生命周期完全由编译器静态推导和优化。这种极高的内存利用率和执行效率，使其非常适合应用于资源受限的嵌入式环境或对性能要求极高的操作系统内核级并发中。

2.1.2 核心抽象：基于轮询的 Future Trait

Rust 异步生态的基础是标准库提供的 `core::future::Future Trait` [5]。与基于事件回调的模型不同，Rust 采用了基于轮询（Polling）的驱动模型。Future 的核心定义如下：

```
pub trait Future {
    type Output;
    fn poll(self: Pin<&mut Self>, cx: &mut Context<'_>) ->
    Poll<Self::Output>;
}
```

任何异步任务都必须实现该 Trait。执行器通过调用 `poll` 方法推动异步任务向前执行。`poll` 的返回值主要有两种：

- `Poll::Ready(Output)`: 当异步任务已经计算完成时, `poll` 返回该状态, 并携带最终执行结果, 此时该 `Future` 对应的状态机终止。

- `Poll::Pending`: 当异步任务由于等待 I/O 就绪、定时器未超时或其他异步事件尚未发生而无法继续推进时, `poll` 返回该状态。此时 `Future` 本身不会阻塞当前执行流, 而是将控制权返回给上层 `executor`; `executor` 可以选择继续轮询其他已经就绪的任务, 也可以在操作系统内核调度逻辑配合下让当前任务让出 CPU。

需要注意的是, `Poll::Pending` 只是 `Future` 层面的“暂时无法继续执行”语义, 并不等价于操作系统线程或进程意义上的阻塞。`Future` 返回 `Pending` 后, 当前这次 `poll` 调用会结束, 调用栈会逐层返回给执行器; 至于后续是否发生任务切换、是否让出 CPU, 则取决于具体执行器和所在系统的调度策略。对于本文实现的 `rCore-N` 异步 timer 而言, `Timer Future` 返回 `Pending` 后, 异步执行器会停止继续推进该任务, 等待后续 `S Timer` 中断通过 `Waker` 将其重新标记为可轮询状态。

此外, `poll` 方法的接收者类型为 `Pin<&mut Self>` [20]。这是 `Rust` 为了保证异步状态机自引用安全而引入的内存固定机制。由于 `async` 函数在编译后会被转换为状态机, 某些状态中可能保存跨越 `await` 点的局部变量引用, 因此 `Rust` 需要保证 `Future` 在被 `poll` 期间不会被随意移动, 从而避免内部引用失效或悬垂指针问题。

2.1.3 唤醒机制与 Reactor-Executor 模型

由于 `Future` 的轮询是被动的, 这就引发了一个核心问题: 当 `Future` 返回 `Poll::Pending` 后, 调度器应当在何时再次轮询该任务? 为了避免无效的忙轮询 (`Busy Wait`), `Rust` 引入了 `Waker` 唤醒机制 [6]。

在 `poll` 方法的参数中, `Context` 上下文对象封装了一个 `Waker` 实例。当底层的异步事件 (例如本文后续章节将探讨的硬件定时器 `Timer` 触发或外设中断到来) 就绪时, 底层事件分发器 (`Reactor`) 会调用注册在其内部的 `Waker::wake()` 方法。该方法会将对应的任务标记为 `Ready` 就绪态, 并通知顶层的执行器 (`Executor`) 将该任务重新推入可执行队列中等待下一次 `poll`。

2.1.4 异步运行时的解耦特性

值得注意的是, `Rust` 标准库仅提供了 `Future` 接口、编译器状态机生成以及基础的唤醒抽象, 并未提供内置的异步运行时 (`Runtime`) [4-5]。这种极度的解耦设计虽然增加了初期的开发门槛, 但赋予了开发者极大的灵活性。开发者可以根据具体的应用场景选择或定制 `Executor` 执行器。

例如，在传统的用户态应用程序中，通常会使用 Tokio 或 `async-std`；而在本文重点探讨的裸机（`no_std`）与微控制器环境中，则需要引入为嵌入式定制的宏观异步执行器（如 Embassy）^[9,18]，或在内核层面自主研发底层的协程调度框架（如在 `rCore` 中实现特定的异步驱动模块）。这种语言层面的机制支持，正是本文后续实现操作系统级异步驱动模块的理论基石。

2.2 Embassy 介绍

正如上一节所述，Rust 语言虽然在语法层面原生支持了无栈协程与 `Future Trait`，但将其应用于实际的工程开发时，仍需要一个具体的异步运行时（`Runtime`）来负责任务的调度与事件的轮询。在传统的基于底层操作系统的用户态应用程序中，通常依赖 Tokio 等基于标准库（`std`）和堆内存分配的庞大运行时；然而，在操作系统内核（如 `rCore`）或微控制器（MCU）等缺乏底层 OS 支撑的裸机（`Bare-metal/no_std`）环境中，此类运行时无法运行。为此，Embassy 框架应运而生。

Embassy 是 Rust 嵌入式生态中较有代表性的异步运行时框架^[9-10]，其核心目标是在 `no_std` 的嵌入式环境中提供基于 `async/await` 的并发编程模型。与传统嵌入式程序中常见的阻塞式等待、轮询式状态机或中断回调式编程方式相比，Embassy 将异步任务、执行器、`Waker`、硬件中断和底层驱动进行统一组织，使应用层可以用接近同步代码的形式表达异步逻辑，同时由运行时负责在合适的时机恢复任务执行。

Embassy 的设计对本文具有重要参考价值。本文研究的目标是在 `rCore-N` 中设计并实现异步驱动模块，而 Embassy 已经在嵌入式场景下较完整地实现了基于 Rust `Future` 的任务调度、定时器等待和中断唤醒机制。因此，分析 Embassy 的任务模型和 `timer` 机制，有助于明确异步驱动中任务如何挂起、中断如何唤醒任务、唤醒后任务如何重新进入执行器等关键问题。

2.2.1 Embassy 的任务模型

在 Embassy 中，用户通常使用 `#[embassy_executor::task]` 标记一个异步函数^[21]，使其成为可以被执行器调度的任务。该宏并不是直接让异步函数立即运行，而是将用户编写的 `async fn` 转换为符合 Embassy 任务模型的代码结构。而用户实现的 `main` 函数程序入口则需要使用 `#[embassy_executor::main]` 宏进行标记。分析使用 `cargo-expand` 工具宏展开的代码可以发现，`#[embassy_executor::task]` 宏大致会生成两层结构：一层是真正包含异步逻辑的隐藏函数，另一层是与原函数同名、

返回 `SpawnToken` 的外层函数。也就是说，调用任务函数本身并不会执行任务，而是创建一个待调度的任务 `token`，后续需要通过 `Spawner::spawn()` 将任务交给执行器管理。

这种设计将任务创建和任务运行分离开来。任务创建阶段会从静态任务池 `TaskPool` 中获取一个任务槽位 `TaskStorage`，将异步函数生成的 `Future` 写入任务存储结构中；任务运行阶段则由执行器根据任务状态和运行队列决定何时调用 `Future` 的 `poll` 方法。由于嵌入式系统通常需要避免动态内存分配，`Embassy` 使用静态任务池保存任务相关数据，这一点对操作系统内核中的异步驱动设计也具有借鉴意义。

2.2.2 执行器与任务调度

`Embassy` 提供了不同形式的执行器^[21]。以 `RP2040` 平台上的 `multiprio.rs` 示例为例^[22]，该程序创建了高优先级、中优先级和低优先级三个执行环境：高优先级和中优先级任务分别运行在 `SWI_IRQ_1` 和 `SWI_IRQ_0` 中断对应的 `InterruptExecutor` 中，低优先级任务则运行在线程模式下的普通 `Executor` 中。该示例展示了高优先级中断执行器可以打断中、低优先级执行器，中优先级执行器也可以打断低优先级执行器。

从调度流程上看，任务被 `Spawner::spawn()` 加入执行器后，并不会一直占用 CPU。执行器在合适的时机取出任务，对任务对应的 `Future` 调用 `poll`。如果 `Future` 返回 `Poll::Pending`，说明该任务当前无法继续执行，执行器会让出 CPU，等待后续事件将其唤醒；如果 `Future` 返回 `Poll::Ready`，说明任务执行完成，执行器会进入任务清理流程。

`Embassy` 任务内部使用状态位描述任务是否已经生成以及是否位于运行队列 `run queue` 中。任务状态表示为 `SPAWNED` 和 `RUN_QUEUED` 两个对应状态位的按位或，前者表示任务是否已经被创建，而后者表示任务此时是否被放进了 `embassy` 的运行队列里等待被 `poll` 并执行。在一个典型的定时器等待过程中，任务首次生成时进入 `SPAWNED | RUN_QUEUED` 状态；执行器取出任务并准备 `poll` 时，会清除 `RUN_QUEUED`；当任务执行到 `Timer::after_ticks(...).await` 并返回 `Pending` 后，任务保持 `SPAWNED` 状态但不在运行队列中；当定时器到期后，任务被唤醒并重新加入运行队列，状态再次变为 `SPAWNED | RUN_QUEUED`。这一状态变化过程可以概括为：`00 -> 11 -> 01 -> 11 -> 01 -> ...`。

2.2.3 Waker、TaskRef 与任务唤醒

Rust Future 机制中，Waker 是异步任务挂起后重新被调度的关键。Future 在 poll 过程中如果无法继续执行，会返回 Poll::Pending；与此同时，它通常需要将当前任务对应的 Waker 注册到底层事件源中。后续当事件发生时，底层驱动或时间驱动通过调用 Waker::wake() 通知执行器：该任务已经具备继续执行的条件。

Embassy 对标准库中的 Waker 机制进行了面向自身执行器的特殊实现^[6,21]。根据 Embassy 的实现思路，Waker 底层通过 RawWaker 构造，其中 RawWaker 的 data 字段保存了指向任务头部结构的指针，vtable 字段则保存对应的唤醒函数表。当调用 Waker::wake() 时，最终会进入 Embassy 定义的唤醒逻辑，并将对应任务重新标记为可运行状态。

Embassy 中还存在 TaskRef 这一任务引用结构^[21]。TaskRef 可以理解为对任务头部结构 TaskHeader 指针的轻量封装，用于在执行器、Waker 和 timer queue 等模块之间传递任务身份。Embassy 可以通过 from_task() 从 TaskRef 构造 Waker，也可以在特定条件下从 Embassy 自己生成的 Waker 中反推出 TaskRef。这种设计使得任务唤醒路径不需要额外查找复杂对象，而是可以直接根据任务头指针定位对应任务。

TaskHeader 中包含任务调度所需的元信息，例如任务状态、运行队列节点、所属执行器指针以及任务 poll_fn 等。Embassy 在运行队列和定时器队列设计中大量采用 intrusive data structure 思想，即将队列节点直接嵌入任务控制结构内部，而不是为每个任务额外动态分配链表节点。这种方式可以减少运行时内存分配，适合嵌入式和内核这类资源敏感环境。

对于本文而言，Embassy 的 Waker 与 TaskRef 设计提供了重要启示：异步驱动模块不能只关注硬件事件本身，还需要建立从“硬件事件发生”到“具体 Future 任务被重新 poll”的映射关系。本文后续在 rCore-N 中实现异步 timer 时，也采用了类似的思路：通过 TaskHeader 保存异步任务状态，通过 TaskRef 在 Waker 唤醒路径中定位任务，并通过 RawWaker 将 Rust 标准 Waker 机制接入内核中的异步执行器。

2.2.4 Embassy 的定时器等待机制概述

Embassy 的 timer 机制建立在 Rust Future 的 poll 语义之上^[4,11]。以 Timer::after_secs(1).await 为例，当执行器第一次 poll 该 Timer Future 时，如果当前时间尚未到达目标时间，Timer 会将当前任务的唤醒信息注册到底层时间驱动中，并返回 Poll::Pending。此后，该任务不再继续占用 CPU，而是等待硬件 timer 到期后被重新唤醒。

在具体平台上，Embassy 的时间驱动负责读取当前硬件时间、维护 timer queue、设置下一次硬件 alarm，并在 timer 中断到来后唤醒到期任务。随后，执行器会在合适的上下文中再次 poll 该任务，使其从 .await 之后继续执行。

因此，Embassy timer 的核心思想可以概括为：Future 负责表达等待语义，time driver 负责管理硬件时间和定时队列，Waker 或 TaskRef 负责将到期事件映射回等待任务，executor 负责恢复任务执行。关于 Timer::poll、schedule_wake、timer queue 和中断唤醒之间的具体源码链路，将在第 3.3 节中进一步展开。

2.2.5 中断执行器与 pender() 机制

Embassy 的一个重要特点是可以将执行器与中断机制结合^[21-22]。Embassy 针对不同的平台分别提供了两种模式的执行器：线程模式执行器和中断模式执行器。对于普通线程模式执行器，任务被唤醒后可以通过类似事件通知的方式唤醒主循环；对于 InterruptExecutor，任务被唤醒后则可以通过挂起指定软件中断的方式，使 CPU 进入对应中断处理函数，并在中断上下文中执行任务 poll。但 Embassy 并非为所有平台都提供了这两种模式的执行器可选。比如针对 Cortex-M 平台就提供了两种模式下的执行器实现，而 Riscv32 平台就只提供了线程模式执行器实现，不支持中断模式执行器。

在 Embassy 的执行器中，pender() 用于任务唤醒^[21]，不同平台提供了不同的实现。当任务从未排队状态变为重新进入运行队列时，执行器会调用 pender.pend()，进而触发 pender(context)。在 Cortex-M 平台上，如果目标是线程模式执行器，pender() 会通过 SEV 唤醒处于 WFE 等待状态的执行器；如果目标是中断执行器，则通过 NVIC::pend(irq) 挂起对应的软件中断，使 CPU 尽快进入该中断处理函数。

以高优先级任务为例，其完整执行链路可以概括为：任务通过 Spawner::spawn() 加入 InterruptExecutor，执行器触发 pender() 挂起对应软件中断；CPU 进入中断处理函数后调用 EXECUTOR_HIGH.on_interrupt()，执行器开始 poll 任务；任务执行到 Timer await 后注册唤醒并返回 Pending；硬件定时器中断到来后时间驱动调用 wake_task()；任务重新入队后再次触发 pender()，最终再次进入软件中断执行器并继续 poll 该任务。

2.2.6 对本文异步驱动设计的启示

Embassy 的实现表明，异步驱动模块的核心并不只是将驱动接口改写为 async fn，而是需要同时解决任务保存、事件注册、任务唤醒和执行器调度等问题。异

步任务在返回 `Poll::Pending` 后必须保存自身状态，底层驱动需要记录该任务的唤醒信息，硬件事件发生后需要通过 `Waker` 或任务引用唤醒对应任务，最后由执行器统一恢复任务执行。

这一思路对本文具有直接启发。本文后续在 `rCore-N` 中实现异步 `timer` 时，也需要建立从 `Timer Future`、`Waker`、`timer queue`、`S Timer` 中断到 `executor poll` 的完整链路。不过，`rCore-N` 是多核操作系统内核，其 `S Timer Trap` 同时承担 OS 时间片调度职责，且多个 `hart` 共享 `ready_queue`。因此，本文不会直接照搬 `Embassy` 的执行器模型，而是在借鉴其 `Future/Waker/timer queue` 思路的基础上，根据 `rCore-N` 的中断上下文和调度约束重新设计。

2.3 rCore 介绍

`rCore` 是由清华大学计算机系操作系统实验室及 `rcore-os` 社区主导开发的一款基于 `Rust` 语言的开源操作系统内核^[12-13]。与传统基于 `C` 语言实现的教学内核相比，`rCore` 的重要特点在于将 `Rust` 的所有权、生命周期和类型系统引入操作系统内核开发过程，使内核在保持底层控制能力的同时，尽可能利用编译期检查减少内存安全和并发安全问题。

本文关注的是在 `rCore-N` 中引入 `Rust` 异步机制并改造 `timer` 相关路径。由于 `rCore-N` 是在 `rCore` 实验基础上进一步演化而来，因此有必要先介绍标准 `rCore` 的基本结构，尤其是其任务管理、陷阱处理、时钟中断与睡眠机制。这些内容构成后续分析 `rCore-N` 差异以及设计异步 `timer` 模块的基础。

2.3.1 基于 `Rust` 的内核实现

传统操作系统内核通常采用 `C/C++` 编写，开发者需要手动管理指针、对象生命周期和并发访问关系，容易出现空指针解引用、释放后使用、缓冲区越界以及数据竞争等问题。`rCore` 选择 `Rust` 作为主要实现语言，试图在教学内核中展示一种更安全的系统编程方式。

`Rust` 的所有权和生命周期机制使许多内存错误能够在编译期被发现。例如，内核中的某些资源对象只能有唯一所有者，引用必须满足生命周期约束，已经释放的对象不能继续被访问。对于共享数据结构，`Rust` 还通过 `Send` 和 `Sync` 等标记 `trait` 约束跨线程或跨执行上下文的数据访问方式。虽然操作系统内核仍然不可避免地需要使用 `unsafe` 代码访问硬件寄存器、操作页表或完成上下文切换，但 `Rust` 的类型系统能够将不安全代码限制在相对集中的底层模块中，从而提升整体代码的可维护性和安全性。

对于本文而言，rCore 的意义不仅在于其使用 Rust 编写，更在于它提供了一个适合观察 Rust 与操作系统内核机制结合方式的实验平台。本文后续引入的 Rust Future、Waker 和异步 timer 机制，也是在这一 Rust 内核基础上进一步展开的。

2.3.2 rCore 的核心内核结构

标准 rCore 主要面向 RISC-V 架构，采用较为典型的宏内核结构 [12-13,23]。其核心功能包括地址空间管理、任务管理、陷阱处理、系统调用和设备抽象等。

在内存管理方面，rCore 基于 RISC-V Sv39 分页机制实现虚拟地址空间管理。内核通过物理页帧分配器管理可用物理内存，并为每个应用构造独立的地址空间。应用程序运行在用户态，内核运行在 S 态，二者通过页表和特权级机制实现隔离。

在陷阱处理方面，rCore 利用 RISC-V 的特权级切换机制处理系统调用、异常和中断。当用户程序执行系统调用，或发生缺页异常、时钟中断等事件时，处理器会从用户态切换到内核态，并进入统一的 trap 处理流程。内核在 trap 入口保存用户上下文，分发不同类型的 trap，并在处理完成后恢复上下文返回用户态。

在任务管理方面，rCore 使用任务控制块保存任务状态、上下文、地址空间等信息，并通过就绪队列维护可运行任务。配合时钟中断，rCore 可以实现基于时间片的抢占式调度。当 timer interrupt 到来时，内核有机会检查当前任务是否需要让出 CPU，并切换到下一个就绪任务。

这些机制共同构成了 rCore 的基本执行框架：用户程序通过系统调用进入内核，内核通过 trap 机制接管控制流，通过任务调度器决定下一个运行任务，并通过地址空间机制保证不同任务之间的隔离。

2.3.3 rCore 的 timer 与阻塞式 sleep 机制

在标准 rCore 中，timer 机制不仅用于周期性触发任务调度，也用于实现用户程序的睡眠等待。内核通过底层 SBI 接口设置下一次时钟中断 [24-25]。当 timer interrupt 到来时，内核会重新设置下一次触发时间，并执行与调度相关的处理逻辑。

除了时间片调度外，rCore 还提供了基于阻塞状态的 sys_sleep 实现。当用户程序调用 sys_sleep 时，内核不会让该任务继续占用 CPU 轮询时间，而是将当前任务设置为 Blocked 状态，并将其注册到定时器等待队列中。之后，该任务会从正常的就绪调度路径中暂时移除。等到对应的定时器事件到期后，内核再将该任务重新设置为就绪状态，并放回 ready queue，等待调度器后续重新选择它运行。

这种设计体现了传统操作系统中常见的阻塞式等待思想：等待事件尚未发生

时，任务主动放弃 CPU；事件发生后，由内核负责唤醒任务。相比用户态忙等循环，阻塞式 `sleep` 可以避免任务在等待期间反复占用处理器时间，因此效率更高，语义也更符合操作系统调度模型。

不过，从异步机制角度看，标准 `rCore` 的 `sys_sleep` 仍然是同步阻塞式接口。任务等待期间由内核以 `Blocked` 状态管理，唤醒逻辑由 `timer` 中断和内核等待队列完成。该机制并没有引入 `Rust Future` 中的 `Poll::Pending`、`Waker` 或执行器模型，也没有将等待过程表达为可组合的异步任务。因此，标准 `rCore` 的 `timer sleep` 机制可以作为本文后续异步 `timer` 设计的参照：它提供了清晰的阻塞唤醒语义，但尚未形成基于 `Future/Waker` 的异步事件驱动模型。

2.3.4 `rCore` 与本文工作的关系

本文并不是直接在标准 `rCore` 上实现异步 `timer`，而是在 `rCore-N` 这一分支上开展实验。因此，标准 `rCore` 在本文中的作用主要是提供对照基线。通过分析标准 `rCore` 的 `timer` 和 `sys_sleep` 机制，可以明确传统阻塞式 `sleep` 的基本语义：任务等待时进入 `Blocked` 状态，`timer` 到期后由内核将其重新加入 `ready queue`。

后续章节将说明，`rCore-N` 在多核调度、用户态中断和 `timer` 管理方面对标准 `rCore` 进行了修改，其中一个重要变化是删去了标准 `rCore` 中基于 `Blocked` 状态和定时器等待队列的 `sys_sleep` 实现。`rCore-N` 用户库中提供的 `sleep` 更接近用户态忙等延时，而不是内核调度意义上的阻塞睡眠。这一差异使得本文需要重新讨论：在 `rCore-N` 的多核共享 `ready queue` 和中断上下文约束下，是否能够引入一种基于 `Rust Future` 的异步 `timer` 机制，以替代简单忙等或传统阻塞式 `sleep`。

因此，`rCore` 为本文提供了基础内核模型和阻塞式 `timer` 参照；`rCore-N` 则提供了更接近本文研究目标的实验环境。二者之间的差异构成了后文分析异步 `timer` 设计动机和工程取舍的重要背景。

2.3.5 关于 `rCore-N`

`rCore-N` 是在 `rCore` 第 5 章实验成果基础上进一步演化得到的一个内核分支^[14]。与标准 `rCore-Tutorial` 相比，`rCore-N` 在继承其基本地址空间、陷阱处理与任务调度框架的同时，对内核结构进行了面向研究需求的裁剪与扩展。一方面，`rCore-N` 不再保留完整的线程模型，而是主要围绕进程模型组织任务管理；另一方面，`rCore-N` 也删减了标准 `rCore` 中基于 `Blocked` 状态和定时器等待队列的 `sys_sleep` 实现。`rCore-N` 的用户库虽然向用户态提供了 `sleep` 接口，但该接口只是不断调用 `sys_get_time()` 获取当前时间，并与目标到期时间戳进行比较的空转

循环，本质上属于忙等式延时，而不是内核调度意义上的阻塞睡眠。因此，相比标准 rCore 中较完整的阻塞式 sleep 路径，rCore-N 为本文重新讨论内核态 sleep 与异步 timer 的实现方式提供了实验空间。

rCore-N 的一个重要扩展是用户态中断机制。为了支持该机制，rCore-N 在内核中引入了用户态 Trap 管理、用户态中断记录队列以及用户态定时器事件投递等逻辑。内核侧使用 UserTrapRecord 描述一次用户态中断事件，其中 cause 字段用于区分用户态时钟中断、用户态外部中断和用户态软中断，message 字段则根据中断类型保存触发时间、外部中断编号或用户自定义消息。用户程序可以通过 sys_init_user_trap 初始化用户态中断队列，通过 sys_set_timer 设置未来某个时刻触发的用户态定时器事件，也可以通过 sys_send_msg 向其他任务发送用户态软中断消息。

在用户程序侧，rCore-N 的用户库会在程序入口 _start 中设置用户态 trap 入口，将 utvec 指向 alltraps_u。当用户态中断触发时，处理流程会先进入用户库中的汇编入口保存上下文，再跳转到 user_trap_handler。该处理函数会根据中断类型进一步调用 timer_intr_handler、soft_intr_handler 或 ext_intr_handler。这些 handler 在用户库中提供了弱符号形式的默认实现，用户程序可以通过定义同名强符号对其进行覆盖，从而实现自定义的用户态中断处理逻辑。

需要说明的是，本文的异步 timer 改造主要针对 S 态 timer 路径，即操作系统内核用于时间片调度和 sys_sleep 的定时器机制。用户态中断中的 sys_set_timer 和 timer_intr_handler 属于 U 态事件回调机制，其目标是让用户程序在用户态响应定时事件，并不直接参与本文实现的内核异步 timer 调度。因此，本文仅将用户态中断作为 rCore-N 的相关背景机制进行介绍，而不将其作为异步 timer 模块的主要实现对象。通过这一说明，可以区分 rCore-N 中两类不同的 timer 使用路径：一类是服务于内核调度和系统调用的 S 态 timer，另一类是服务于用户程序事件处理的 U 态 timer。

在体系结构上，rCore-N 还从单核执行环境扩展为多核并行运行模型。系统启动后，多个 hart 分别完成初始化，并运行各自的 idle 进程；但在调度层面，它们共享同一个全局 ready_queue。这一设计一方面提高了系统的并发执行能力，另一方面也显著增加了共享数据结构访问时的同步复杂度。任一 hart 在访问 ready_queue 或其他共享内核对象时，都需要通过锁机制保证互斥；而如果这些锁的获取出现在中断处理路径中，就可能引入竞争、嵌套甚至死锁风险。因此，rCore-N 在定时器、睡眠和任务唤醒机制上的设计，与单核版本 rCore 相比呈现出更加复杂的工程约束。

这一多核共享队列结构直接影响了本文后续对异步 `sys_sleep` 的实现选择。理想情况下，阻塞式 `sleep` 可以在系统调用中将当前任务从 `ready_queue` 中移除，并在 `timer` 到期后重新放回调度队列。但在 `rCore-N` 中，`timer` 到期通常发生在中断上下文；如果在该路径中直接访问多个 `hart` 共享的 `ready_queue` 并获取其锁，可能破坏内核稳定性，本文实验过程中发现，直接在 `timer` 中断路径中访问多个 `hart` 共享的 `ready_queue` 并进行任务重新入队，会触发内核 `panic`，说明该路径存在中断上下文持锁与多核调度一致性方面的风险。因此，本文后续实现异步 `sys_sleep` 时，并未简单恢复标准 `rCore` 中的 `Blocked` 状态和阻塞唤醒队列，而是在 `rCore-N` 现有调度模型约束下采用了更保守的实现方式。

对于本文而言，`rCore-N` 构成了一个比标准 `rCore` 更贴近研究目标的实验平台。它既保留了操作系统内核的基本骨架，又具备多核、用户态中断等扩展特性，能够更真实地暴露异步 `timer` 在内核环境中面临的并发、调度与中断上下文问题。

2.4 本章小结

本章围绕本文后续实现异步 `timer` 所需的相关技术进行了介绍。首先，本章分析了 `Rust Future` 无栈协程模型，说明 `async` 函数在编译后会被转换为状态机，`Future` 在返回 `Poll::Pending` 时并不会保留独立运行栈，而是将控制权返回给执行器，并依赖 `Waker` 在事件到来后重新进入可轮询状态。该机制为本文在内核中实现轻量级异步任务提供了语言基础。

其次，本章介绍了 `Embassy` 异步运行时。`Embassy` 面向 `no_std` 嵌入式环境，将异步任务、执行器、`Waker`、`timer queue` 和硬件中断组织为完整的事件驱动链路。其 `timer` 机制说明，异步定时等待并不是简单延时，而是需要完成从 `Timer Future` 注册唤醒信息、硬件 `timer` 到期、中断处理唤醒任务到 `executor` 重新 `poll` 的完整过程。该设计为本文实现 `rCore-N` 异步 `timer` 提供了直接参考。

最后，本章介绍了 `rCore` 和 `rCore-N` 的基本结构。标准 `rCore` 提供了较清晰的阻塞式 `sys_sleep` 参照，而 `rCore-N` 在多核、用户态中断、`virtual timer` 和异步串口等方面进行了扩展，同时也删去了标准 `rCore` 中基于 `Blocked` 状态和定时器等待队列的 `sys_sleep` 实现。由此可见，`rCore-N` 既具备引入异步 `timer` 的实验基础，也暴露出多核调度、`timer` 中断和任务唤醒之间的工程约束。这些内容构成了后续章节分析 `timer` 机制并设计异步 `timer` 模块的基础。

第 3 章 timer 基本机制

时钟 (Timer) 机制是操作系统内核中的基础机制之一。它不仅为系统提供时间基准, 也直接参与多任务抢占式调度、睡眠等待、超时控制以及异步事件处理等功能。对于本文而言, timer 机制尤其重要: 一方面, rCore-N 的 S 态 timer 是时间片调度和 `sys_sleep` 的基础; 另一方面, Rust Future 异步模型中的定时等待也需要依赖 timer 中断完成任务唤醒。因此, 本章首先介绍 timer 的基本硬件原理和常见软件组织方式, 随后分析 rCore 与 rCore-N 中 timer 机制的差异, 并进一步说明 rCore-N 当前 sleep 机制的局限性, 为后文设计内核级异步 timer 模块提供背景。

3.1 timer 简介

在硬件层面, 处理器通常会提供持续递增的时间计数器以及用于触发中断的比较寄存器^[23]。以 RISC-V 架构为例, 机器模式下定义了 `mtime` 和 `mtimecmp` 相关机制。其中, `mtime` 是持续递增的时间源, 用于表示当前硬件时间; `mtimecmp` 则保存下一次 timer interrupt 的比较阈值。当 `mtime` 的值增长到大于或等于 `mtimecmp` 时, 硬件会产生一次时钟中断。

对于运行在 S 态的操作系统内核而言, 内核通常并不直接操作机器模式下的 `mtimecmp`, 而是通过 SBI 接口请求底层运行环境设置下一次时钟中断^[25]。例如在 rCore 和 rCore-N 中, 内核最终都会通过 `set_timer` 或其封装函数设置下一次 timer interrupt 的触发时间; 当底层硬件时钟到期后, 控制流进入 S 态 trap 处理路径, 并由内核处理 Supervisor Timer Interrupt。也就是说, 硬件 timer 提供的是最基础的“到某个时间点触发一次中断”的能力, 而操作系统需要在此基础上构造更高层的调度、睡眠和超时机制。

从软件组织方式看, 操作系统中的 timer 机制通常包含两类需求^[17,26]。第一类是周期性调度 tick。内核按照固定频率设置下一次 timer interrupt, 例如每隔 10ms 触发一次时钟中断。每次中断到来时, 内核更新系统时间, 并检查当前任务的时间片是否耗尽。该模型实现简单, 适合与时间片轮转调度结合, 因此在教学操作系统中较为常见。

第二类是软件定时事件管理。除了固定周期的调度 tick 之外, 内核还可能需要处理大量非周期性的定时事件, 例如任务睡眠、I/O 超时和异步定时等待等。由于底层硬件 timer 对每个 hart 同一时刻通常只能设置一个下一次触发时间, 硬

件本身不会自动维护多个定时事件队列。因此，当系统中存在多个定时请求时，操作系统需要在软件层面维护额外的 timer 集合，例如使用堆、平衡树或链表等数据结构记录多个到期时间。

周期性 tick 和软件 timer 集合可以采用不同的组合方式。一种简单方式是：硬件 timer 始终按固定周期触发，内核在每次 tick 到来时检查软件 timer 集合中是否存在已经到期的事件。rCore-2025S 的 sys_sleep 定时唤醒机制就属于这种模式。另一种方式是：内核从软件 timer 集合中选择最近即将到期的事件，并将其对应时间设置到底层硬件 timer；事件到期后再重新选择下一项。这种方式具有 one-shot 或 tickless 的特点，能够减少不必要的周期性中断。rCore-N 的 virtual timer 机制更接近这种按最近事件设置硬件 timer 的设计。

因此，在实际内核中，周期性 tick 与软件 timer 事件并不是完全互斥的两种机制。一个系统可以一方面保留固定周期 tick 用于时间片调度，另一方面维护软件 timer 集合用于 sleep 或超时处理。不同内核之间的差异，往往不在于是否存在软件 timer 集合，而在于该集合如何驱动硬件 timer、服务哪些事件，以及到期后如何完成任务唤醒或事件投递。

3.2 rCore timer 实现

rCore-2025S 的 timer 主要承担两个职责^[12-13]：一是周期性触发内核调度，支持 OS 时间片轮转；二是配合内核中的定时等待队列，实现阻塞式 sys_sleep。从具体实现看，rCore-2025S 采用的是固定周期 tick 与软件 timer heap 相结合的方式：底层硬件 timer 基本按照固定周期触发，而 sys_sleep 等定时等待事件则保存在内核的软件数据结构中，并在每次 tick 到来时被检查。

相比之下，rCore-N 在底层 timer 设置路径上引入了 virtual timer 机制^[14]。rCore-N 的 set_next_trigger() 不再直接调用 set_timer(time::read() + CLOCK_FREQ / TICKS_PER_SEC)，而是调用 set_virtual_timer(time::read() + CLOCK_FREQ / TICKS_PER_SEC, 0)。其中，pid = 0 用于表示该 timer 属于内核自身的调度时钟事件。与此同时，用户态通过 sys_set_timer 设置的定时事件也会进入同一套 virtual timer 管理框架。这样，rCore-N 中下一次硬件 timer 的触发时间不再严格固定为 10ms 后，而是由当前 hart 上最早到期的 virtual timer 决定。

3.2.1 rCore-2025S 的固定 tick 与阻塞式 sleep

在 rCore-2025S 中，底层时间读取与 timer interrupt 设置主要依赖 RISC-V 的时间寄存器和 RustSBI 提供的接口^[24]。内核通过 time::read() 读取当前硬件时间，

并根据 `CLOCK_FREQ` 将其转换为毫秒或微秒。例如，`get_time_ms()` 通过 `time::read() * MSEC_PER_SEC / CLOCK_FREQ` 计算当前毫秒时间，`get_time_us()` 则以类似方式返回微秒时间。

rCore-2025S 通过 `set_next_trigger()` 设置下一次时钟中断。其核心逻辑如下：

```
pub fn set_next_trigger() {  
    set_timer(get_time() + CLOCK_FREQ / TICKS_PER_SEC);  
}
```

其中，`TICKS_PER_SEC` 为 100，因此该函数每次都会在当前时间基础上增加一个固定 tick 间隔，设置下一次 timer interrupt。换言之，rCore-2025S 的底层调度时钟是典型的固定周期 tick 模型，约每 10ms 触发一次时钟中断。

当 Supervisor Timer Interrupt 触发后，内核会进入 trap 处理流程。在时钟中断处理路径中，rCore 通常会重新调用 `set_next_trigger()` 设置下一次 tick，并执行调度相关逻辑。如果当前任务的时间片已经耗尽，内核会挂起当前任务，将其重新放回就绪队列，并切换到下一个可运行任务。因此，timer interrupt 是 rCore 实现时间片轮转调度的重要基础。

除了时间片轮转调度外，rCore-2025S 的 timer interrupt 还参与阻塞式 sleep 的唤醒。内核中维护了一个全局 `TIMERS`，其类型为 `UPSafeCell<BinaryHeap<TimerCondVar>>`。其中，`TimerCondVar` 保存一个到期时间 `expire_ms` 和一个需要被唤醒的任务 `task: Arc<TaskControlBlock>`。当用户程序调用 `sys_sleep` 时，内核会将当前任务设置为阻塞态，并通过 `add_timer(expire_ms, task)` 将该任务对应的定时唤醒项插入 `BinaryHeap`。

由于 Rust 标准库中的 `BinaryHeap` 默认是最大堆，rCore-2025S 在 `TimerCondVar` 的比较逻辑中对 `expire_ms` 取负后再比较，从而使堆顶对应最早到期的 timer 项。每次 timer interrupt 到来后，内核会调用 `check_timer()` 检查软件 timer heap。该函数首先读取当前毫秒时间 `current_ms`，然后反复查看堆顶 timer。如果堆顶 timer 的 `expire_ms` 小于或等于当前时间，说明该 timer 已经到期，内核就调用 `wakeup_task(Arc::clone(&timer.task))` 唤醒对应任务，并将该 timer 从堆中弹出；如果堆顶 timer 尚未到期，则说明后续 timer 也不会到期，检查过程结束。

需要注意的是，在 rCore-2025S 中，`BinaryHeap` 中保存的 sleep timer 并不会直接改变下一次硬件 timer 的触发时间。硬件 timer 仍然由 `set_next_trigger()` 按照固定周期设置。也就是说，`sys_sleep` 和 OS 时间片在逻辑上是相对独立的：OS 时间片 tick 决定硬件 timer 何时触发，而 `BinaryHeap` 只是在每次固定 tick 到来后被检查。因此，rCore-2025S 的 timer 模式可以概括为“固定周期 tick + 软件 timer heap 检查”。

这种设计实现简单，适合教学内核，也能够完成阻塞式 sleep 的基本语义：任务在等待期间进入 Blocked 状态，不再参与正常调度，待 timer 到期后由内核唤醒。不过，由于 sleep timer 的检查依赖固定 tick 到来，因此其唤醒精度受到 tick 周期限制。例如，如果固定 tick 为 10ms，那么一个 sleep timer 即使在两次 tick 之间已经到期，也通常需要等到下一次 timer interrupt 到来后才会被检查和唤醒。

3.2.2 rCore-N 的 virtual timer 机制

rCore-N 在 timer 机制上进一步引入了 virtual timer 抽象。与 rCore-2025S 中 set_next_trigger() 直接设置下一次固定 tick 不同，rCore-N 的 set_next_trigger() 被改写为：

```
pub fn set_next_trigger() {  
    set_virtual_timer(time::read() + CLOCK_FREQ / TICKS_PER_SEC, 0);  
}
```

这里的 pid = 0 表示该 timer 属于内核自身的周期性调度事件，而不是某个具体用户任务。也就是说，rCore-N 将 OS 时间片 tick 也纳入 virtual timer 框架中统一管理。除此之外，用户程序通过 sys_set_timer 设置的用户态定时事件，同样会进入该 virtual timer 管理框架。这样，内核调度事件和用户态定时事件不再是两条完全独立的 timer 路径，而是共享同一套按到期时间管理的 virtual timer 机制。

rCore-N 中的 virtual timer 由每个 hart 各自维护^[14]。其核心数据结构如下：

```
lazy_static! {  
    pub static ref TIMER_MAP: [Arc<Mutex<BTreeMap<usize, usize>>>;  
CPU_NUM] = Default::default();  
}
```

其中，数组下标对应不同 hart，BTreeMap 的键表示 timer 到期时间，值表示该 timer 对应的 pid。set_virtual_timer(time, pid) 会根据当前 hart 编号访问对应的 timer map，并将新的 (time, pid) 插入其中。如果插入的时间与已有 key 冲突，则通过不断将 time 加一的方式避免相同 key 覆盖旧事件。

插入完成后，rCore-N 会检查当前 timer map 中最早到期的事件：

```
if let Some((timer_min, _)) = timer_map.first_key_value() {  
    if time == *timer_min {  
        set_timer(time);  
    }  
}
```

这段逻辑说明，只有当新插入的 timer 成为当前 hart 上最早到期的事件时，系统才会真正调用 set_timer(time) 更新底层硬件 timer。反过来说，如果新插入的 timer 晚于当前最早到期事件，则不需要改写硬件 timer，因为当前硬件 timer 已经

指向一个更早的触发时间。待最早事件到期并被处理后，内核再从 BTreeMap 中选择下一项重新设置底层 timer。

因此，rCore-N 的 virtual timer 更接近“按最近事件设置硬件 timer”的 one-shot 管理方式。它与 rCore-2025S 的关键区别在于：rCore-2025S 的 BinaryHeap 主要是在固定 tick 到来后辅助检查 sleep 任务是否到期，并不决定下一次硬件 timer 触发时间；而 rCore-N 的 BTreeMap 则直接参与下一次底层硬件 timer 触发时间的选择。换言之，rCore-2025S 中的软件 timer heap 是固定周期 tick 之后的检查结构，而 rCore-N 的 virtual timer map 更像是硬件 timer 之上的软件多路复用层。

由此也可以看出，rCore-N 的 timer interrupt 不再严格固定为每 10ms 一次。由于 OS 时间片事件和用户态 sys_set_timer 事件都进入 set_virtual_timer 管理，下一次硬件 timer 的触发时间实际上等于当前 hart 上最早到期的 virtual timer。当某个用户态定时事件早于下一个 OS 时间片 tick 到期时，硬件 timer 会被设置为更早触发；而在没有更早用户事件时，系统仍然可以表现为接近固定周期的普通调度 tick。可以将其概括为：

```
下一次硬件 timer 触发时间
    = min(下一个 OS 时间片 tick, 当前 hart 上最早到期的用户态 virtual
timer)
```

这一区别非常重要。rCore-2025S 中，sys_sleep 的到期检查依附于固定 tick；而 rCore-N 中，用户态定时事件可以直接影响下一次硬件 timer 的触发时间。因此，rCore-N 的 timer 机制已经不再只是用于内核时间片调度或阻塞式任务唤醒，而是进一步承担“定时事件管理”和“面向任务的事件投递”职责。

3.2.3 rCore-N 中用户态 timer 事件投递

在 virtual timer 机制基础上，rCore-N 提供了 sys_set_timer 系统调用^[14]，使用户程序可以主动申请在未来某一时刻触发面向自身的定时事件。该机制与 rCore-2025S 中的 sys_sleep 有明显不同：sys_sleep 的核心语义是让当前任务进入内核阻塞态，等待 timer 到期后由内核唤醒；而 sys_set_timer 更像是向内核注册一个未来到期的用户态事件，事件到期后由内核投递给目标任务。

当用户态 timer 到期时，如果当前正在运行的正是设置该 timer 的任务，内核可以直接设置用户态时钟中断标志，使控制流进入用户态 trap 处理逻辑；如果当前运行的是其他任务，内核则会先将该定时事件以 Trap Record 的形式写入目标任务的用户态中断队列，待该任务后续再次获得调度时再进行处理。也就是说，rCore-N 中的 timer 到期结果不一定是“唤醒一个阻塞任务”，也可能是“向某个用户任务投递一条用户态中断事件”。

这一点进一步体现了 rCore-N 与 rCore-2025S 的抽象差异。rCore-2025S 中的 timer heap 主要服务于内核阻塞式 sleep；而 rCore-N 的 virtual timer 则将 timer 抽象为一种可复用的事件源，使其既可以表示内核调度 tick，也可以表示用户态定时事件。虽然 rCore-N 的现有实现尚未形成完整的 Future/Waker 异步模型，但其将 timer 事件从单纯的阻塞式任务唤醒扩展为面向任务的事件投递机制，已经与后续异步 timer 所需的事件驱动思想存在一定相似性。本文后续工作正是在这一背景下，尝试进一步引入 Rust Future 的挂起与唤醒模型，使 timer 事件能够以异步任务的方式被注册、挂起和恢复。

3.2.4 rCore-N 中 sleep 机制的局限性

需要指出的是，采用忙等式 sleep 机制的并不是 rCore-Tutorial-2025S，而是原始 rCore-N 用户库中提供的 sleep^[14]。在 rCore-N 中，user_lib 提供给用户态程序的 sleep 并不是一个真正的内核阻塞式睡眠接口，而是通过不断调用 sys_get_time() 获取当前时间，并与目标到期时间戳进行比较来实现。换言之，该 sleep 本质上是一个空转等待循环：只要当前时间尚未到达目标时间，用户程序就会继续执行时间检查逻辑。

这种实现方式与 rCore-Tutorial-2025S 的 sys_sleep 有明显区别。在 rCore-Tutorial-2025S 中，调用 sys_sleep 的任务会进入 Blocked 状态，并从就绪调度路径中移除，直到 timer 到期后再由内核唤醒。而在 rCore-N 的用户态 busy-wait sleep 中，任务并不会真正进入内核阻塞状态，也不会从调度系统中被移除。它只是持续消耗 CPU 时间进行时间判断，因此更接近用户态延时函数，而不是操作系统调度意义上的阻塞睡眠。

与 rCore-Tutorial-2025S 实现相比，原始 rCore-N 之所以删除了 sys_sleep 系统调用，简化 sleep 实现为 busy-wait，主要是因为内核架构的修改导致继续沿用 rCore-Tutorial-2025S 实现会引发冲突。rCore-N 运行在多 hart 环境下，多个 hart 共享同一个全局 ready_queue。如果采用传统 blocked sleep 方案，那么任务调用 sleep 时需要被移出调度队列，而 timer 到期后又需要在中断处理路径中将该任务重新加入 ready_queue。由于 ready_queue 是多个 hart 共享的数据结构，在中断上下文中直接获取其锁并修改队列，可能与其他 hart 上正在进行的调度操作发生竞争。本文在实验过程中也观察到，直接在 timer 中断路径中访问共享 ready_queue 并进行任务重新入队，会引发内核 panic，说明该路径存在中断上下文持锁和多核调度一致性方面的风险。

因此，rCore-N 中 sleep 机制的局限性并不只是“缺少一个标准 rCore 风格的

`sys_sleep` 实现”，而是暴露出多核调度、`timer` 中断和任务唤醒之间的同步问题。如何在避免中断上下文复杂加锁的同时，实现比 `busy-wait` 更高效的等待机制，是后续引入异步 `timer` 机制需要面对的核心问题之一。本文后续设计的异步 `timer` 模块，正是在这一背景下尝试将 `Rust Future` 的挂起/唤醒模型引入 `rCore-N` 的 `timer` 路径，以探索更适合多核内核环境的定时等待实现方式。

3.2.5 小结

通过上述分析可以看出，`rCore-2025S` 与 `rCore-N` 的 `timer` 机制差异主要体现在两个方面。第一，`rCore-2025S` 采用“固定周期 `tick` + 软件 `timer heap` 检查”的方式，硬件 `timer` 基本每 10ms 触发一次，`sys_sleep` 等待项只在 `tick` 到来后被检查。第二，`rCore-N` 引入 `virtual timer` 机制，将 OS 时间片事件和用户态定时事件统一纳入按到期时间管理的 `timer map` 中，使下一次硬件 `timer` 触发时间由当前 `hart` 上最早到期事件决定。

与此同时，`rCore-N` 删去了标准 `rCore` 中基于 `Blocked` 状态和 `timer heap` 的 `sys_sleep` 实现，使用户库中的 `sleep` 退化为用户态忙等式延时。这一现状为本文引入异步 `timer` 提供了直接动因：`rCore-N` 已经具备较灵活的 `timer` 事件管理基础，但尚未形成基于 `Rust Future/Waker` 的内核等待与唤醒模型。因此，下一节将进一步分析 `Embassy timer` 的实现思路，为第 4 章在 `rCore-N` 中设计异步 `timer` 原型提供参考。

3.3 Embassy timer 实现

`Embassy` 的 `timer` 机制是其异步运行时与底层硬件交互的重要组成部分^[11,27]。对于用户而言，定时等待通常表现为如下形式：

```
#[embassy_executor::task]
async fn run() {
    loop {
        info!("tick");
        Timer::after_secs(1).await;
    }
}
```

从应用层看，`Timer::after_secs(1).await` 类似于一个普通的延时操作；但从运行时实现看，它并不会阻塞当前 CPU，而是通过 `Rust Future` 的 `poll` 机制将当前任务挂起，并将任务的唤醒信息注册到底层时间驱动中。当硬件定时器中断到来后，时间驱动再根据 `timer queue` 中记录的信息唤醒对应任务，使其重新进入执行器的运行队列。

3.3.1 Timer Future 的 poll 过程

Timer::after_secs(1) 会返回一个实现了 Future trait 的 Timer 对象^[11]。该 Future 的核心逻辑位于 poll 方法中。根据已有源码分析，其主要结构如下：

```
impl Future for Timer {
    type Output = ();

    fn poll(mut self: Pin<&mut Self>, cx: &mut Context<'_>) ->
    Poll<Self::Output> {
        if self.yielded_once && self.expires_at <= Instant::now() {
            Poll::Ready(())
        } else {
            embassy_time_driver::schedule_wake(
                self.expires_at.as_ticks(),
                cx.waker()
            );
            self.yielded_once = true;
            Poll::Pending
        }
    }
}
```

该逻辑可以分为两种情况。第一种情况是当前 Timer 已经至少让出过一次执行机会，并且当前时间已经达到或超过目标时间 expires_at，此时 poll 返回 Poll::Ready(())，表示定时等待结束。第二种情况是当前时间尚未到期，或者 Timer 第一次被 poll，此时 Timer 会调用 embassy_time_driver::schedule_wake()，将目标唤醒时间和当前任务的 Waker 注册到底层时间驱动中，随后返回 Poll::Pending。

这里的关键点是：Timer Future 本身并不直接操作硬件定时器，而是通过 embassy_time_driver 提供的统一接口与具体平台的时间驱动交互。schedule_wake() 接收的参数包括一个以 ticks 为目标时间戳，以及当前任务对应的 Waker。其中，cx.waker() 由执行器在 poll 当前任务时传入，代表“当前正在被 poll 的这个任务应如何被唤醒”。

因此，Timer::poll() 的作用并不是“等待时间流逝”，而是“登记当前任务未来应该在某个时间点被唤醒”。当它返回 Poll::Pending 后，当前任务结束本轮 poll，在后续 timer 事件到来前不会重新进入 executor 的运行队列，直到后续由 Waker 或 TaskRef 将其唤醒。

3.3.2 时间驱动接口与平台实现

Embassy 将通用的 Timer Future 和具体硬件平台的定时器实现解耦^[11,27]。embassy_time::Timer 只负责表达“在某个时间点之后继续执行”这一异步语义，而

具体如何读取当前时间、如何设置硬件 alarm、如何处理中断，则由平台相关的时间驱动完成。

以 RP2040 平台为例，Embassy 在 `embassy-rp/src/time_driver.rs` 中提供了 `TimerDriver` ^[27]。该驱动按照 Embassy 的时间驱动规范实现 `Driver trait`，其中最重要的两个接口是 `now()` 和 `schedule_wake()`。`now()` 用于返回当前硬件时间，`schedule_wake()` 用于登记某个 `Waker` 应在指定时间点被唤醒。

RP2040 的硬件时间值由高、低两个 32 位寄存器组成，因此 `now()` 读取当前时间时需要注意一致性问题。其实现会先读取高位寄存器，再读取低位寄存器，随后再次读取高位寄存器；只有当前后两次高位值相同时，才说明读取低位期间高位没有发生进位，此时可以组合出一个一致的 64 位时间值。

`schedule_wake()` 则负责将 `Timer Future` 的唤醒请求登记到 `timer queue` 中。其逻辑大致如下：

```
fn schedule_wake(&self, at: u64, waker: &core::task::Waker) {
    critical_section::with(|cs| {
        let mut queue = self.queue.borrow(cs).borrow_mut();

        if queue.schedule_wake(at, waker) {
            let mut next = queue.next_expiration(self.now());
            while !self.set_alarm(cs, next) {
                next = queue.next_expiration(self.now());
            }
        }
    })
}
```

该函数首先进入临界区，避免 `timer queue` 在并发场景下被中断处理函数或其他上下文同时修改。随后，它调用 `queue.schedule_wake(at, waker)` 将当前任务登记到 `timer queue` 中。如果这次登记改变了 `timer queue` 中最早的到期时间，则需要重新设置硬件 alarm。设置硬件 alarm 时还需要考虑一种边界情况：如果准备设置的时间点已经过期，`set_alarm()` 会返回失败，此时需要再次调用 `next_expiration()` 处理已经到期的任务，并重新计算下一次需要设置的 alarm 时间。

3.3.3 timer queue 与 Waker 到 TaskRef 的转换

在 RP2040 的 Embassy 时间驱动中，`timer queue` 使用的是 `embassy-time-queue-utils` 中的 `integrated queue` 实现。该实现与普通保存完整 `Waker` 的设计不同，它并不长期保存完整的 `Waker`，而是会将传入的 Embassy 专用 `Waker` 反解为 `TaskRef`。

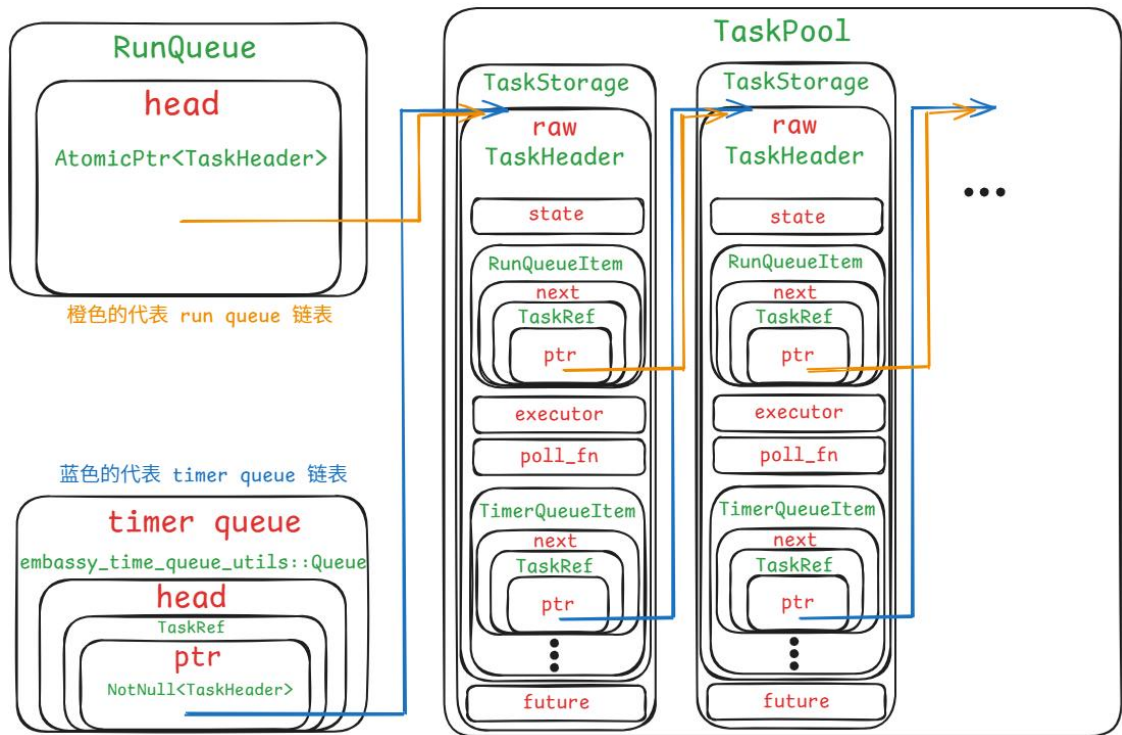


图 3.1 Embassy 中 timer queue、TaskRef 与 run queue 的关系示意图

Embassy 的 Waker 由 RawWaker 构造。RawWaker 中的 `data` 字段会被设置为指向 TaskHeader 的指针，`vtable` 字段则保存 Embassy 自己定义的 Waker 虚函数表。当调用 `Waker::wake()` 时，会通过该虚函数表进入 Embassy 的唤醒逻辑，最终调用 `wake_task()`。

Embassy 提供了两个方向的转换函数：一方面可以通过 `from_task()` 从 TaskRef 构造 Waker；另一方面也可以通过 `task_from_waker()` 从 Waker 反推出 TaskRef。后者会检查 Waker 的 `vtable` 是否为 Embassy 执行器创建的专用 `vtable`，如果不是，则说明该 Waker 并非 Embassy executor 生成，不能被 integrated timer queue 使用。

因此，在 timer queue 的 `schedule_wake()` 中，Embassy 可以执行如下转换：

```
let task = embassy_executor::raw::task_from_waker(waker);
```

转换后，timer queue 不再需要保存完整的 Waker，而是保存一个更轻量的 TaskRef。这也是 integrated timer queue 能够复用 TaskHeader::timer_queue_item 的基础。TaskHeader 中除了任务状态、运行队列节点、执行器指针、`poll_fn` 之外，还包含 `timer_queue_item` 字段。该字段中保存 timer queue 链表的 `next` 指针以及当前任务对应的到期时间 `expires_at`。

从数据结构上看，integrated timer queue 本质上是一个链表，其队列头部保存一个 `Option<TaskRef>`。访问下一个 timer queue 节点时，并不是访问额外分配的

链表节点，而是通过 TaskRef 找到对应任务的 TaskHeader，再访问其中的 timer_queue_item.next 字段。

这种设计使得 Timer Future 的等待信息直接嵌入任务结构中，减少了运行时动态内存分配，也减少了保存完整 Waker 带来的额外空间开销。对于嵌入式系统来说，这种 intrusive queue 设计符合其对内存使用可控性的要求。

3.3.4 硬件 alarm 设置与中断处理

在 Timer Future 首次被 poll 并调用 schedule_wake() 后，时间驱动会根据 timer queue 中最早到期的任务设置硬件 alarm。RP2040 平台的 TimerDriver 中，set_alarm() 负责设置硬件下一次触发的时间戳，同时也会在软件结构中记录该时间戳。如果发现要设置的时间戳已经过期，则说明对应任务应当立即被处理，此时 set_alarm() 返回失败，外层逻辑会调用 next_expiration() 处理过期项，并重新选择下一次到期时间。

当硬件定时器到期后，CPU 会进入对应的定时器中断处理函数。在 RP2040 的时间驱动中，该中断处理函数形式如下：

```
#[interrupt]
fn TIMER_IRQ_0() {
    DRIVER.check_alarm()
}
```

#[interrupt] 属性宏用于将普通 Rust 函数声明为中断服务例程，使其能够被放入对应平台的中断向量表中。对于 RP2040，当硬件定时器中断触发时，CPU 会进入 TIMER_IRQ_0，随后调用 DRIVER.check_alarm() 检查当前 alarm 是否确实到期。

check_alarm() 的主要作用是判断当前记录的 alarm 时间戳是否已经小于或等于当前时间。如果已经到期，则调用 trigger_alarm()；如果硬件中断提前触发，而实际时间尚未到期，则重新设置硬件 alarm。trigger_alarm() 会进一步调用 timer queue 的 next_expiration()，处理所有已经到期的 Timer，并为下一次最近的到期时间重新设置硬件中断。

3.3.5 到期任务的唤醒与重新入队

timer queue 的 next_expiration() 是 Embassy timer 唤醒链路中的关键函数。它会遍历 timer queue 中登记的任务，检查每个任务对应的 expires_at 是否已经小于或等于当前时间。如果某个任务已经到期，队列会调用：

```
embassy_executor::raw::wake_task(p);
```

其中，p 是 timer queue 中保存的 TaskRef。也就是说，在 integrated timer

queue 中，到期时并不是再通过完整 Waker 调用 wake()，而是直接根据 TaskRef 调用 wake_task()。

wake_task() 的作用是将任务重新放回执行器的运行队列。结合 Embassy 任务状态机来看，一个等待 Timer 的任务在执行到 .await 并返回 Poll::Pending 后，通常处于 SPAWNED 状态，但不在运行队列中。定时器到期后，wake_task() 会调用状态转移逻辑，将任务重新标记为 RUN_QUEUED，使任务状态从 SPAWNED 回到 SPAWNED | RUN_QUEUED。

Embassy 的状态分析中，任务常见的 timer 等待状态变化可以概括为：

```
初次 spawn:           00 -> 11
executor 取出任务 poll: 11 -> 01
Timer 返回 Pending:    保持 01
Timer 到期 wake:       01 -> 11
```

其中，SPAWNED 表示任务 Future 仍然存在，RUN_QUEUED 表示任务已经进入执行器运行队列，等待下一次被 poll。

任务被 wake_task() 重新加入运行队列后，执行器并不会在 wake_task() 中直接同步执行该任务的 poll。相反，wake_task() 只负责修改任务状态并将任务入队。后续真正执行 poll 的过程仍然由 executor 完成。这一点非常重要，因为它将“中断中发现事件到期”和“执行异步任务代码”解耦开来，避免驱动层直接执行任意任务逻辑。

3.3.6 从中断唤醒到 executor 再次 poll

当 timer queue 中的定时任务到期后，Embassy 需要完成两步工作：首先由 time driver 找到到期任务并将其重新标记为可运行，其次通知对应 executor，使该任务能够在后续被再次 poll。这一过程连接了底层硬件 timer 中断和上层 Rust Future 执行，是 Embassy timer 机制中从“等待”回到“继续执行”的关键环节。

在任务唤醒方面，Embassy 并不是直接在 timer 中断中继续执行用户 Future，而是通过任务引用完成状态更新。对于 integrated timer queue 这类实现，timer queue 中保存的是由 Waker 转换而来的 TaskRef。当硬件 timer 到期后，平台相关的 timer 中断处理函数会进入 time driver，例如在 embassy-rp 平台上会进入对应的 timer 中断路径，并调用 DRIVER.check_alarm() 检查当前 alarm 是否已经到期。若 alarm 到期，则进一步调用 trigger_alarm()，由 timer queue 找出已经到期的定时项，并对相应任务调用 wake_task(task_ref)。wake_task 的作用是将该任务重新标记为可运行，并将其加入 executor 的 run queue，而不是直接执行该任务的 Future。

当任务被重新加入 run queue 后，Embassy 还需要通知对应 executor：当前已

经有任务可以继续运行。这一步由 `pender` 机制完成。根据 `SyncExecutor::enqueue()` 的实现，当运行队列从空变为非空时，会调用 `self.pender.pend()`，进而触发架构相关的 `pender()`。对于本文图示所采用的中断模式 `executor`，`pender()` 会通过平台中断控制器挂起对应的软件中断，例如在 `Cortex-M/RP` 平台上可通过 `NVIC::pend(irq)` 挂起 `SWI_IRQ_0`。随后 CPU 进入对应的软件中断处理函数，并在该中断上下文中调用 `InterruptExecutor::on_interrupt()`，由 `executor` 对 `run queue` 中的任务执行下一轮 `poll`。

以中断模式执行器为例，任务恢复执行的路径可以概括为：`timer` 中断首先负责检查硬件 `alarm` 与 `timer queue`，并通过 `wake_task(task_ref)` 将到期任务重新放入 `executor run queue`；随后 `pender.pend()` 触发平台相关的 `pender()`，挂起对应的软件中断；CPU 进入软件中断处理函数后调用 `InterruptExecutor::on_interrupt()`；`on_interrupt()` 内部进一步驱动 `executor poll run queue` 中的就绪任务。此时，先前因 `Timer::after_secs(1).await` 返回 `Poll::Pending` 而挂起的任务会再次被 `poll`。由于当前时间已经达到或超过 `expires_at`，`Timer::poll(cx)` 会返回 `Poll::Ready()`，于是异步任务可以从 `.await` 之后继续执行。

由于 `Embassy timer` 的执行链路同时涉及通用的 `embassy_time`、`embassy_time_driver` 抽象层，以及具体平台上的硬件 `timer` 中断和软件中断执行器逻辑，如果完整展开所有中间调用过程，图示会较为繁杂，不利于突出 `Timer Future` 挂起与恢复的主线。因此，本文以 `embassy-rp` 平台的中断模式 `executor` 为例，将 `Timer Future` 从首次 `poll` 挂起到后续被 `timer` 中断唤醒并由软件中断 `executor` 恢复执行的两个过程合并绘制在同一张图中，并省略部分与主线关系较弱的中间调用，仅保留关键模块、关键函数及主要控制流，如下图所示。

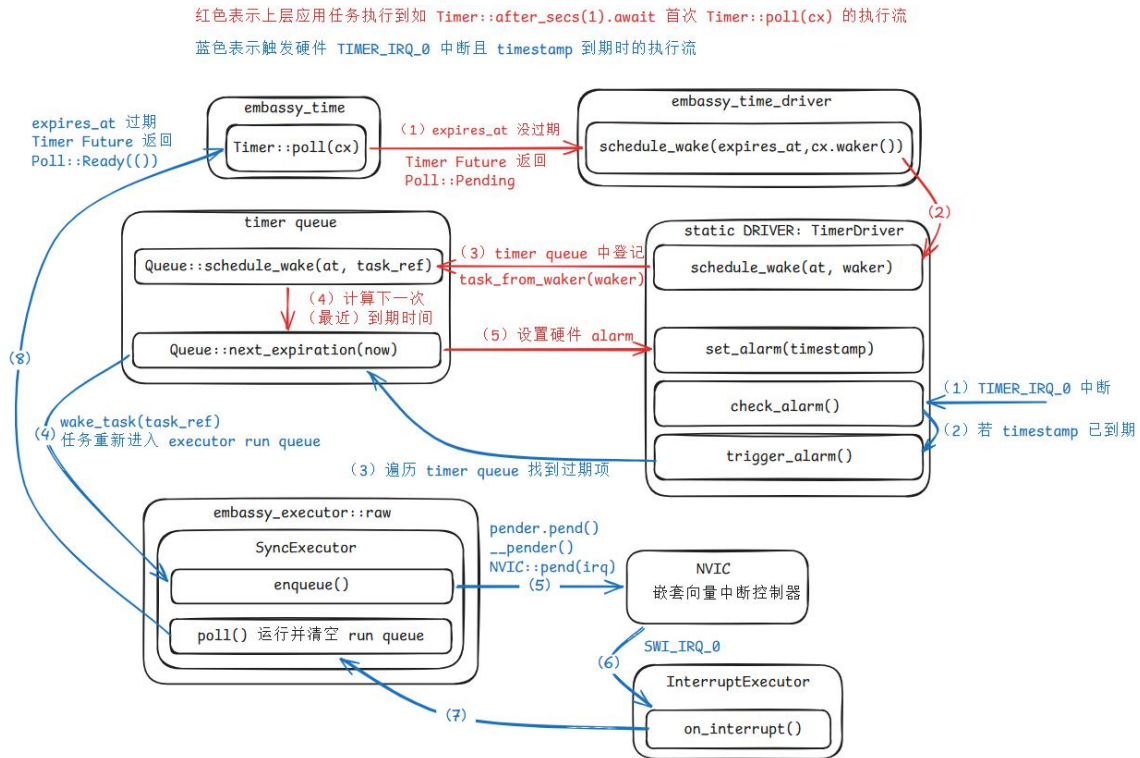


图 3.2 以 embassy-rp 为例的中断模式 Timer Future 挂起与恢复执行关键链路图

图中，红色箭头表示挂起与定时注册路径，蓝色箭头表示中断唤醒与恢复执行路径。红色路径中，当异步任务执行到 `Timer::after_secs(1).await` 时，中断模式 executor 调用 `embassy_time::Timer::poll(cx)`。若当前时间尚未到达 `expires_at`，则 `Timer::poll` 通过 `embassy_time_driver::schedule_wake(expires_at, cx.waker())` 将目标到期时间和当前任务的唤醒信息注册到底层 time driver 中。随后，底层 `TimerDriver` 将对应任务登记到 timer queue，并根据最近到期时间设置硬件 alarm。完成上述注册后，`Timer::poll` 返回 `Poll::Pending`，该任务结束本轮 poll，并在后续 timer 事件到来前不再进入 executor run queue。

蓝色路径中，当底层硬件 timer 到期后，系统进入 embassy-rp 平台对应的 timer 中断处理函数。中断处理函数调用 `TimerDriver` 的 `check_alarm()` 检查当前 alarm 是否到期；若到期，则进一步通过 `trigger_alarm()` 处理 timer queue 中已经到期的定时项。对于到期任务，time driver 通过 `wake_task(task_ref)` 将其重新加入 executor run queue。由于该 executor 为中断模式 executor，任务入队后会借助 pender 机制触发 `pender()`，并进一步挂起对应的软件中断。CPU 随后进入该软件中断处理函数，在其中调用 `InterruptExecutor::on_interrupt()`，由中断执行器对 run queue 中的就绪任务进行下一轮 poll。此时，`Timer::poll(cx)` 发现当前时间已经到达 `expires_at`，于是返回 `Poll::Ready()`，异步任务即可从 `.await` 之后继续执行。

这条链路说明，Embassy timer 的本质并不是阻塞式睡眠，而是基于 Future、Waker/TaskRef、timer queue、硬件 timer 中断、软件中断 executor 和 pender 机制的协作式异步等待机制。需要说明的是，上图是面向机制分析的关键路径图，而非源码级完整调用图；图中省略的平台初始化、辅助封装函数以及部分底层寄存器访问细节，不影响对 Embassy timer 核心工作机制的理解。

3.4 本章小结

本章围绕 timer 基本机制展开，重点分析了 rCore-2025S、rCore-N 和 Embassy timer 在定时事件管理方式上的差异。首先，本章从 RISC-V timer 的硬件机制出发，说明底层硬件 timer 提供的是“到某个时间点触发一次中断”的能力，而操作系统需要在此基础上构造调度 tick、sleep 等待、超时控制和异步定时事件等更高层机制。

随后，本章分析了 rCore-2025S 的 timer 实现。rCore-2025S 采用“固定周期 tick + 软件 timer heap 检查”的方式：硬件 timer 基本按照固定周期触发，内核在每次 timer interrupt 到来后处理时间片调度，并检查 BinaryHeap 中是否存在已经到期的 sys_sleep 项。这种方式实现简单，能够支持阻塞式 sleep，但 sleep 到期检查依赖固定 tick 到来，因此其唤醒精度受到 tick 周期限制。

接着，本章分析了 rCore-N 的 virtual timer 机制。rCore-N 将 OS 时间片事件和用户态定时事件统一纳入按到期时间管理的 timer map 中，使下一次硬件 timer 的触发时间由当前 hart 上最早到期事件决定。相比 rCore-2025S，rCore-N 的 timer 管理方式更接近 one-shot 模型。不过，rCore-N 当前用户库中的 sleep 仍然是基于 sys_get_time() 的忙等式延时，并没有形成内核阻塞式 sleep 或 Future/Waker 式异步等待模型。

最后，本章分析了 Embassy timer 的实现链路。Embassy 通过 Timer Future 表达定时等待，通过 time driver 管理硬件 timer 和 timer queue，通过 Waker 或 TaskRef 将到期事件映射回等待任务，并最终由 executor 重新 poll 被唤醒任务。该机制为本文提供了重要启示：异步 timer 的核心并不是单纯设置一个未来时间点，而是要建立从 Future 挂起、Waker 注册、硬件 timer 到期、中断唤醒到 executor 恢复执行的完整链路。下一章将在此基础上，结合 rCore-N 的多核调度结构和 S Timer Trap 约束，设计并实现本文的异步 timer 原型模块。

第 4 章 异步驱动模块

4.1 异步 timer 设计与实现

本节面向 rCore-N 的多核内核环境，设计并实现一个参考 Embassy timer 思路、但又适配 rCore-N 内核结构的异步定时模块 [10-11,14]。本文的目标并不是在 rCore-N 中完整移植 Embassy 执行器，也不是一次性实现一个完全理想的阻塞式异步 sleep，而是在保留 rCore-N 现有 Trap、timer 和任务调度结构的前提下，引入基于 Future/Waker 的定时等待机制，验证 timer 事件能否以异步任务的方式被注册、挂起、唤醒和继续执行。

从模块划分上看，本文实现的异步 timer 主要位于 os/src/async_timer 目录下，由 task、waker、queue、timer 与 time_driver 等部分组成。其中，timer 模块提供 Timer Future 抽象，time_driver 模块负责读取当前时间、设置硬件 timer 和管理到期事件，queue 模块维护待唤醒 timer 项，task 模块提供轻量级异步任务与执行器，waker 模块则负责将异步任务与 Rust 标准 Waker 机制连接起来。

整体调用链可以概括为：上层通过 after_ticks 或 after_ms 创建一个异步定时任务；该任务内部 await 一个 Timer Future；当 Timer::poll 发现当前时间尚未到期时，将当前任务对应的 Waker 注册到定时队列中并返回 Poll::Pending；当 S 态时钟中断到来时，time_driver 检查已到期定时事件并调用对应 Waker 完成唤醒；随后执行器在当前实现中于同一个 S Timer Trap 上下文中继续轮询已经变为就绪态的异步任务。

需要说明的是，本文实现过程中进行了两项重要工程取舍。第一，本文最初尝试为 rCore-N 补充类似 rCore-Tutorial 的 blocked sys_sleep，即在系统调用中将当前任务设置为阻塞态并移出调度路径，timer 到期后再重新加入 ready_queue。但 rCore-N 的多核调度结构与 rCore-Tutorial 不同，多个 hart 共享同一个全局 ready_queue，在 timer 中断上下文中直接访问该队列并加锁会引入稳定性风险。第二，本文最初参考 Embassy 的中断执行器设计，尝试在 S Timer 中断中唤醒异步任务，再通过 S Software Interrupt 执行 executor poll；但 OS 时间片 timer 与 suspend_current_and_run_next() 的上下文强相关，若将 poll 移到 S Software Interrupt 中执行，会导致时间片切换逻辑的上下文不一致。因此，本文最终采用了更保守但可运行的设计：sys_sleep 层面采用 suspend-loop 折中实现，而异步 timer 的 wake 与 poll 则统一放在 S Timer Trap 中完成。

4.1.1 设计目标与 rCore-N 约束

本文异步 timer 模块的设计目标主要包括三个方面。

第一，在 rCore-N 内核中引入一个能够表达定时等待的 Timer Future。该 Future 使用 Rust 的 Poll::Pending 与 Poll::Ready 表达定时等待的挂起与完成，使 timer 等待过程不再只是同步函数中的忙等判断，而可以被表示为一个可由 executor 推进的异步状态机。

第二，建立 timer 中断与异步任务之间的唤醒关系。当某个 Timer Future 尚未到期时，它会将当前任务对应的 Waker 注册到底层 timer queue 中；当 S Timer 中断到来并发现该 timer 已经到期时，驱动通过 Waker 将对应异步任务重新标记为可轮询状态。这样，硬件 timer 中断、软件定时队列和 Rust Future 的唤醒机制之间就形成了完整链路。

第三，在不大幅重构 rCore-N 现有调度器和 Trap 结构的前提下，将异步 timer 原型接入 S 态 timer 路径。rCore-N 已经具有多核调度、全局 ready_queue、S Timer Trap 和用户态 timer 事件投递等机制，本文的实现必须在这些已有机制约束下完成，而不是重新设计一个脱离 rCore-N 的独立异步运行时。

这一目标同时受到两个重要约束。

首先，rCore-N 运行在多 hart 环境下，多个 hart 共享同一个全局 ready_queue。如果采用传统 blocked sleep 方案，timer 到期时需要在中断处理路径中重新访问 ready_queue 并将任务入队。由于该队列是多个 hart 共享的数据结构，直接在中断上下文中获取其锁并修改队列，容易与其他 hart 上正在发生的调度操作产生竞争。本文实验过程中也观察到，直接在 timer 中断路径中操作共享 ready_queue 会引发内核 panic，因此该路径不能简单照搬 rCore-Tutorial 的阻塞式 sleep 实现。

其次，本文改造的是 rCore-N 的 S 态 timer 路径。该 timer 不仅是普通定时事件源，还承担 OS 时间片调度职责。当时间片到期后，内核需要调用 suspend_current_and_run_next() 执行任务切换，而该函数的语义依赖于当前 S Timer Trap 所建立的上下文。如果将 executor poll 移动到 S Software Interrupt 或其他独立上下文中执行，那么由 OS tick 对应 Future 间接触发的任务切换就可能发生在错误的 Trap 上下文中，从而破坏原有调度流程。因此，本文不能直接照搬 Embassy 的 interrupt executor 模型。

4.1.2 基于 blocked 的 sys_sleep 初始设想及放弃原因

在实现 sys_sleep 时，本文最初考虑采用传统操作系统中的阻塞式设计。该方案中，用户任务调用 sys_sleep 后，内核将当前任务状态设置为 Blocked，并将其

从正常就绪调度路径中移除；随后通过异步 timer 模块注册一个到期事件。当 timer 到期后，超时回调负责将该任务状态重新设置为 Ready，并将其放回 ready_queue，等待调度器后续重新选择执行。

该方案语义最接近 rCore-Tutorial 中的 sys_sleep。它的优点是效率较高：处于 sleep 状态的任务不会继续参与调度，也不会反复消耗 CPU 时间检查到期条件。从操作系统调度语义看，这也是更标准的阻塞式 sleep。

但是，在 rCore-N 中直接采用该方案会遇到两个问题。

第一个问题来自 rCore-N 的任务状态和调度结构。rCore-N 与 rCore-Tutorial 的任务模型并不完全一致，其任务状态中原本并没有直接对应 rCore Blocked 的状态；若要实现 blocked sleep，就需要补充 Blocked 状态，并实现新的 block_current_and_run_next()。然而，rCore-N 的 suspend_current_and_run_next() 与 rCore-Tutorial 中的实现不同。rCore-N 中的 suspend_current_and_run_next() 主要负责调用 schedule() 切换回当前 hart 的 idle 进程上下文，而将原任务重新设置为 Ready 并放回 ready_queue 的逻辑发生在 idle 进程后续的 suspend_current() 中。也就是说，rCore-N 的调度路径是更明显的“两段式”结构，不能直接照搬 rCore-Tutorial 中将取出当前任务、修改状态和切换任务集中在一个函数中的实现。

第二个问题来自多核共享 ready_queue。即使补充了 Blocked 状态和 block_current_and_run_next()，timer 到期后的唤醒逻辑仍然需要在 S Timer 中断路径中将任务重新加入全局 ready_queue。由于 rCore-N 的多个 hart 共享该队列，这意味着中断上下文需要直接获取共享队列的锁并修改调度结构。本文实验表明，这一路径会带来内核稳定性问题。因此，本文最终没有继续采用“timer 到期后直接将阻塞任务重新放入 ready_queue”的 blocked sleep 方案。

4.1.3 当前实现：基于 suspend-loop 的 sys_sleep

基于上述限制，本文最终在 sys_sleep 中采用了更保守的 suspend-loop 实现。具体而言，sys_sleep 不将当前任务从 ready_queue 中移除，而是创建一个与目标到期时间对应的异步 timer，并在等待条件尚未满足时循环调用 suspend_current_and_run_next()。该函数使当前任务主动让出 CPU，但不会像 blocked sleep 那样将任务彻底移出调度体系。

在当前实现中，timer 到期后的动作也不再是直接将任务重新插入 ready_queue，而是修改 sys_sleep 循环所依赖的到期条件。当 sleep 任务后续再次被调度时，会重新检查该条件；如果 timer 已经到期，则跳出循环并从 sys_sleep 返回。换言之，timer 中断路径只需要完成到期标志或等待条件的更新，而不需要

直接操作共享 `ready_queue`。

这种方案牺牲了一部分调度效率。由于 `sleep` 任务没有真正从 `ready_queue` 中移除，它在等待期间仍然可能被调度器选中，并在检查条件后再次调用 `suspend_current_and_run_next()` 让出 CPU。因此，该实现并不是严格意义上的高效 `blocked sleep`，而是一个在 `rCore-N` 当前调度结构和中断上下文约束下能够稳定运行的折中方案。

不过，该方案仍然具有实验意义。它验证了 `Rust Future` 与 `Waker` 机制可以接入 `rCore-N` 的 `S Timer` 路径，也验证了 `timer` 到期事件可以通过异步任务影响 `sys_sleep` 的控制流程。更重要的是，该实现暴露出后续进一步优化必须解决的问题：如何在不直接于中断上下文中持有共享 `ready_queue` 锁的前提下，实现真正的阻塞式 `sleep` 唤醒。

4.1.4 基于 Future 的异步 timer 抽象

本文将定时器抽象实现为一个 `Timer Future` [5,11]，其核心状态包含两个字段：到期时间 `expires_at` 与标记首次挂起行为的 `yielded_once`。其中，`expires_at` 表示该定时事件应当在何时被触发，`yielded_once` 则用于区分“首次注册等待”和“被唤醒后的再次轮询”两个阶段。

当执行器首次轮询 `Timer` 时，若当前时间尚未到达 `expires_at`，则 `Timer::poll` 会调用 `time_driver::schedule_wake(self.expires_at, cx.waker())`，将当前任务的唤醒器注册到定时队列中，并返回 `Poll::Pending`。当底层时钟推进至目标时间后，执行器再次轮询该 `Future`；此时若 `yielded_once` 已为真且当前时间满足超时条件，则返回 `Poll::Ready()`，表示定时等待结束。

对单个 `Timer Future` 而言，定时等待被表达为 `Future` 状态机上的一次挂起与一次恢复。不过，在本文当前的 `sys_sleep` 原型中，受 `rCore-N` 多核共享 `ready_queue` 和中断上下文限制，系统调用层面仍采用 `suspend-loop` 方式等待 `timer` 到期。因此，本文实现的重点并不是一次性完成理想 `blocked sleep`，而是验证 `timer` 等待过程能否通过 `Future/Waker` 机制接入内核时钟路径。

在接口层面，模块进一步向上封装了 `after_ticks`、`after_ms` 与 `add_timer` 等函数。其中，`after_ticks` 和 `after_ms` 负责生成一个异步任务，在 `Timer::after(...).await` 结束后执行给定回调；`add_timer` 则根据绝对时间戳与当前时间的差值，将其转换为相对延时后复用同一套异步机制。由此，内核中“未来某时刻执行某动作”的逻辑，可以统一映射为“创建一个等待 `Timer` 的异步任务”。

4.1.5 TaskRef、TaskHeader 与 Waker 的转换关系

为了让 `Waker` 能够在不依赖复杂对象查找的前提下直接唤醒异步任务，本文在执行器内部设计了 `TaskHeader` 与 `TaskRef` 两级任务引用结构。需要特别说明的是，`rCore-N` 原有的任务控制块和本文异步 `timer` 模块中的 `TaskRef` 不是同一个概念：前者是操作系统调度器管理的进程或任务实体，后者是异步执行器内部用于定位某个 `Future` 任务的轻量引用。

`TaskHeader` 是异步任务的实际控制头，内部保存任务状态 `state` 以及被执行器轮询的 `Future` 对象。`TaskRef` 则是对 `TaskHeader` 原始指针的轻量封装，用作唤醒路径中的任务句柄。任务创建时，本文首先创建 `Arc<TaskHeader>` 保存任务头，再在构造 `TaskRef` 时将其转换为原始指针。后续执行器或 `Waker` 需要定位任务时，可以通过该原始指针重新访问对应的 `TaskHeader`。

这种设计降低了 `Waker` 路径中的对象查找成本，使 `RawWaker` 只需携带一个任务头指针即可完成唤醒。但同时需要指出，本文当前实现仍属于原型实现，任务生命周期管理较为依赖执行器内部约束。后续若进一步完善异步运行时，需要更系统地处理任务释放、重复唤醒和定时器取消等场景下的资源管理问题。

在 `Waker` 的实现中，本文没有为每次唤醒额外构造独立的闭包对象，而是直接以 `TaskHeader` 指针作为 `RawWaker` 的数据字段^[6,21]。`from_task(task_ref)` 会基于该指针构造出一个 `Waker`；当底层调用 `wake()` 时，`RawWaker` 的 `wake` 回调会将该指针重新解释为 `TaskRef`，并进一步调用 `wake_task(task_ref)`。后者执行的核心操作是将任务状态原子地改写为 `Ready`，从而使该任务在下一轮执行器扫描时重新变为可运行状态。

也就是说，`Waker` 并不直接执行任务，而只是完成“将 `Future` 对应任务重新标记为可轮询”的状态跃迁。这种设计与 `Embassy` 中“任务头指针 + 原始唤醒器”的基本思路一致，但实现上更贴合本文的内核实验环境。`TaskRef` 充当了 `Waker` 和执行器之间的最小连接件，`TaskHeader` 则承担真正的任务状态与 `Future` 存储职责，二者之间的相互转换构成了整个异步 `timer` 子模块的任务唤醒基础。

在任务状态转换方面，本文使用 `Ready`、`Running` 和 `Pending` 三种状态描述异步任务的执行情况。执行器从任务队列中取出 `Ready` 任务后，将其状态设置为 `Running` 并调用对应 `Future` 的 `poll`。如果 `Future` 返回 `Poll::Ready()`，则说明该异步任务已经完成；如果返回 `Poll::Pending`，则说明该任务当前需要等待后续事件唤醒。

需要特别说明的是，本文在 `Future` 返回 `Poll::Pending` 时，并没有无条件将任务状态写为 `Pending`，而是使用类似 `compare_exchange(Running, Pending)` 的方式，仅当任务状态仍然保持为 `Running` 时才改写为 `Pending`。这样做是为了避免丢失

唤醒：如果某个 Future 在本次 poll 过程中已经通过 Waker 被重新标记为 Ready，那么无条件写入 Pending 会覆盖该 Ready 状态，导致任务错过下一轮轮询。通过这种条件更新方式，可以避免“先被唤醒、后被覆盖为 Pending”的状态竞争问题。

4.1.6 定时队列与 Time Driver 设计

在定时事件管理方面，本文借鉴了 Embassy timer queue 的核心思想，为每个 hart 维护独立的 TimerDriver 和执行器。具体而言，异步 timer 模块中同时维护了按 CPU_NUM 构造的 TimerDriver 数组和 Executor 数组，并通过当前 hart_id() 选择本 hart 对应的实例。这样可以避免不同 hart 上的异步 timer 任务共享同一个执行器队列，降低跨 hart 同步复杂度。

每个 TimerDriver 内部包含一个 Queue 以及一个记录当前硬件告警时间的 AlarmState。其中，Queue 负责保存尚未到期的定时等待项，AlarmState 则记录当前已经写入底层 SBI set_timer 的最近到期时间。TimerDriver 对外提供 now() 和 schedule_wake(at, waker) 两个核心接口：前者读取当前硬件时间，后者将某个 Waker 注册到指定到期时间。

本文虽然引入了 Embassy 中“定时队列统一管理待唤醒任务”的设计思想^[1]，但没有直接采用 Embassy 将 timer queue 实现为侵入式链表、并把链表节点直接嵌入 TaskHeader 的做法。Embassy 之所以采用这种结构，是为了在嵌入式 no_std 场景中尽可能避免额外动态分配，并将任务控制块与定时等待节点紧密绑定。本文的实现环境则是 rCore-N 内核，目标首先是验证异步 timer 机制在操作系统中的可行性与执行语义，因此在工程实现上选择了更直接的方案：使用一个基于 Vec 的独立队列保存 timer 项，每个 timer 项显式保存到期时间 at 和对应的 Waker。

当 Timer::poll 调用 schedule_wake(at, waker) 时，队列首先会通过 Waker::will_wake() 判断该任务是否已经存在于定时队列中。如果已经存在，则只有当新的到期时间早于原有到期时间时，才更新该 timer 项；如果不存在，则克隆当前 Waker 并插入新的 (at, waker) 项。这样可以避免同一个 Future 在多次 poll 过程中反复向队列插入重复 timer，从而减少重复唤醒和队列膨胀问题。

需要注意的是，本文当前的 Vec timer queue 并不维护严格有序的数据结构。next_expiration(now) 会线性扫描整个队列：对于已经到期的 timer 项，将其从队列中移除并执行 waker.wake()；对于尚未到期的项，则取其中最小的 at 作为下一次硬件 alarm 的时间。也就是说，本文当前实现并不是基于最小堆或平衡树的有序 timer queue，而是采用简单线性扫描方式完成到期处理和下一次 alarm 选择。

这种实现简单直观，便于调试，并且足以支撑本文当前的原型验证目标。但

它的时间复杂度与队列长度相关：插入去重需要遍历已有 timer 项，到期检查也需要扫描整个队列。因此，在大量 timer 并发注册的场景下，该实现可能带来额外开销。后续若继续向高性能方向优化，可以考虑将 Vec 替换为二叉堆、BTreeMap 或更接近 Embassy 的 intrusive timer queue。

在硬件 timer 设置方面，TimerDriver::schedule_wake() 在队列发生变化后，会调用 next_expiration(now) 获取当前最近到期时间，并尝试通过 set_alarm(timestamp) 设置底层硬件 timer。如果要设置的时间戳已经小于或等于当前时间，说明该 timer 事实上已经到期，驱动会将硬件 timer 暂时设置为 usize::MAX，并返回失败，提示外层循环立即再次处理到期事件。这样可以避免将已经过期的时间戳写入硬件 timer 后造成逻辑不一致。

当 S 态时钟中断到来时，time_driver::on_interrupt() 会检查当前硬件告警是否已经到期。若到期，则调用 trigger_alarm()，进一步通过 next_expiration(now) 唤醒所有已经到期的 timer 项，并计算下一次最近到期时间重新设置硬件 timer。至此，硬件时钟中断与软件 Future 唤醒之间的桥梁就被建立起来。

4.1.7 S Timer Trap 中统一执行 wake 与 poll

本文实现与 Embassy 的一个关键差异在于：没有直接照搬 Embassy 的中断模式执行器，也没有将 executor poll 放入单独的 S Software Interrupt 中，而是将 Waker 触发与执行器 poll 统一放在同一个 S Timer Trap 上下文中完成。

本文最初曾设计过类似 Embassy interrupt executor 的两阶段方案：在 S Timer 中断中调用 async_timer::on_timer_interrupt()，由 TimerDriver::check_alarm() 修改到期任务的状态并触发 Waker；随后通过 pend S Software Interrupt 的方式，在 S Software Interrupt 中执行 executor.poll()。这种设计在结构上接近“上半部中断只处理硬件事件，下半部中断负责推进异步任务”的模式。

但是，在 rCore-N 中，OS 时间片逻辑也需要与 async timer 统一。原因在于原有 timer interrupt 逻辑和 async timer 逻辑都会调用 sbi::set_timer() 或其封装函数，两套 timer 设置逻辑不能并存，否则会互相覆盖下一次硬件 timer。另一方面，如果强行将 OS 时间片逻辑也包装成 async timer，但仍然把 executor poll 放在 S Software Interrupt 中执行，那么时间片到期后调用 suspend_current_and_run_next() 时，其 current_task() 所处的上下文将不再是被 S Timer 中断打断的用户任务，而是被 S Software Interrupt 打断的上下文。这会导致任务切换语义与原有 S Timer Trap 不一致。

因此，本文放弃 pender() 和 S Software Interrupt executor 方案，选择在

Supervisor Timer Trap 中完成 wake 与 poll。具体地，`async_timer_interrupt_handler()` 在进入 S Timer Trap 后，依次执行 `on_timer_interrupt()`、`executor_poll()` 与 `handle_os_tick()` 三个阶段：首先处理到期定时事件并唤醒对应异步任务；随后在当前 Trap 上下文中轮询本 hart 上所有就绪异步任务；最后再处理与 OS 时间片相关的调度逻辑。

采用这种设计后，又出现了新的重入与上下文切换问题。当前 `Executor::spawn()` 在将任务加入队列后会立即调用 `Executor::poll()`。如果 OS 时间片 Future 的超时回调中直接调用 `after_ticks()` 注册下一次 OS tick，就会在 `executor_poll` 尚未退出时再次进入 `Executor::poll()`，形成执行器 poll 重入。同时，如果 `executor` 在 poll 到 OS 时间片对应 Future 的过程中直接调用 `suspend_current_and_run_next()`，那么执行器遍历任务队列的过程会被中途切断，可能导致 `executor` 状态不一致。

为了解决上述问题，本文没有在 OS tick 的超时回调中直接注册下一次 tick，也没有直接调用 `suspend_current_and_run_next()`，而是仅设置一个原子标志位 `PENDING_OS_TICK`。待 `executor_poll()` 完成后，再由 `handle_os_tick()` 检查该标志位。若标志位有效，则先通过 `after_ticks(CLOCK_FREQ / TICKS_PER_SEC, next_trigger)` 注册下一次 OS tick，再调用 `suspend_current_and_run_next()` 执行任务切换。这样既避免了 `executor_poll` 重入，也避免了在 `executor_poll` 中途直接切换上下文。

综上，S Timer Trap 中统一 wake 与 poll 的设计是本文的重要工程取舍。它牺牲了 Embassy 式 interrupt executor 中 wake 与 poll 的分层结构，但保证了 OS 时间片任务切换逻辑仍然运行在正确的 S Timer Trap 上下文中，同时通过延迟处理 OS tick 标志避免了 `executor_poll` 中途切换和 poll 重入问题。

4.1.8 小结

综上，本文实现的异步 timer 借鉴了 Embassy 在 Timer Future、定时队列和 Waker 机制上的核心思想，但并未机械照搬其执行器模型，而是根据 rCore-N 的多核调度结构和 S Timer Trap 语义进行了适配。该实现的核心特征可以概括为五点。

第一，本文以 TaskHeader、TaskRef 和原始 Waker 组成轻量任务唤醒链路，使 timer 到期事件能够将对应 Future 重新标记为可轮询状态。第二，本文在任务状态转换中使用条件更新方式避免丢失唤醒，防止 Future 在返回 Poll::Pending 时覆盖已经由 Waker 设置的 Ready 状态。第三，本文以每 hart 一个 TimerDriver 和

执行器的方式组织异步 timer 运行结构，降低跨 hart 共享执行器带来的同步复杂度。第四，本文采用基于 Vec 的独立 timer queue 保存 (at, waker) 项，以较低实现复杂度完成原型验证。第五，本文将 wake 与 poll 统一收敛到 S Timer Trap 上下文中执行，以保证 OS 时间片调度路径与 suspend_current_and_run_next() 的上下文一致性。

从 timer 抽象粒度看，本文实现相较 rCore-2025S 和 rCore-N 进一步细化了定时事件的绑定对象。rCore-2025S 虽然维护了软件 timer heap，但底层硬件 timer 仍主要按照固定 tick 周期触发，sleep timer 只能在 tick 到来后被检查；rCore-N 通过 virtual timer 将定时事件组织为 (time, pid)，使下一次硬件 timer 能够由当前 hart 上最早到期事件决定，但其到期结果主要仍是面向进程或任务的事件投递。本文实现则将定时事件组织为 (time, Waker)：当 Timer Future 返回 Poll::Pending 时，time driver 将当前 Future 对应的 Waker 与目标到期时间一起注册到 timer queue 中；当 timer 到期后，time driver 可以直接调用该 Waker，将具体异步任务重新标记为可轮询状态。因此，本文实现的优势并不在于简单宣称时间误差一定小于 rCore-N，而在于将 timer 到期事件从进程级投递进一步映射到 Future 任务级唤醒，使其更贴合 Rust Future/Waker 异步执行模型。

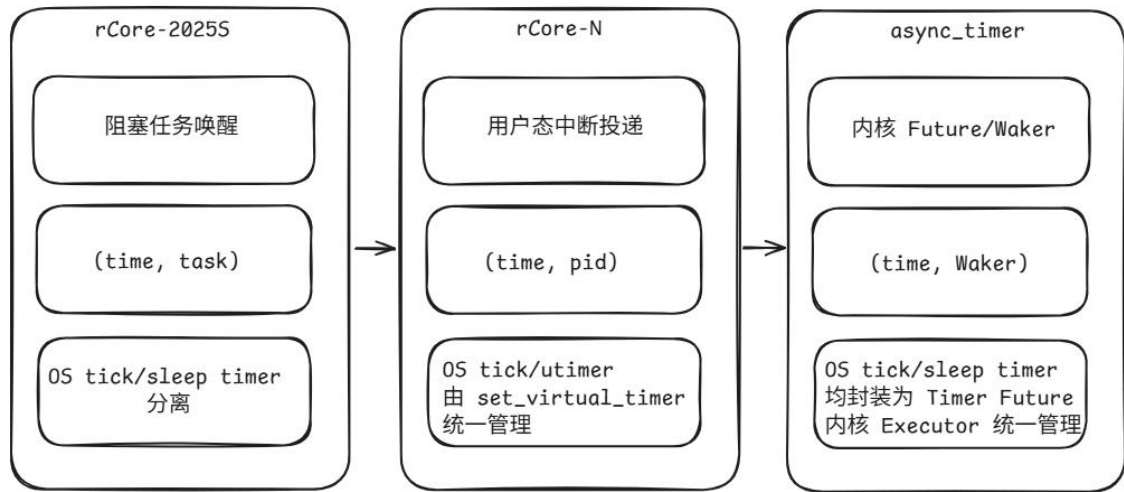


图 4.1 rCore-2025S、rCore-N 和 async_timer 三种 timer 实现的对比图

尽管当前实现仍属于原型性质，但它已经完成了从 Timer Future 注册、Waker 保存、S Timer 中断唤醒、executor 重新 poll 到 sys_sleep 控制流恢复的完整闭环。该闭环验证了 Rust Future/Waker 机制接入 rCore-N 内核 timer 路径的可行性，也为后续评估 Future 无栈特性和多核环境下 sleep 响应行为提供了基础实现。

4.2 异步 timer 正确性分析与响应测试

4.2.1 Rust Future 无栈协程分析

在理论层面，Rust Future 之所以被称为“无栈协程”，关键并不在于协程执行过程中完全不使用栈，而在于它不拥有独立于调用者之外的专用运行栈。Future 的 `poll` 本质上仍然是一个普通函数调用：当执行器调用某个顶层 Future 的 `poll` 时，若该 Future 内部继续 `poll` 其所依赖的子 Future，就会在当前执行上下文的栈上形成一条普通的函数调用链。因此，协程执行期间所使用的栈，并不是像线程那样预先分配给任务本身，而是复用了当前调用 `poll` 的执行现场。

对于本文实现的异步 timer 而言，这一点体现得尤为明显。执行器在 S 态时钟中断上下文中调用任务的顶层 `poll`；若该任务内部又 `await` 一个 Timer Future，则会进一步进入 `Timer::poll`。此时，从执行器到顶层任务，再到叶子 Timer Future，会在当前 Trap 对应的内核栈上形成一条逐层嵌套的 `poll` 调用链。换言之，异步任务运行时所使用的栈空间，始终是触发本次轮询的那一条内核控制流所持有的栈，而不是 Future 自身单独拥有的栈。

更重要的是，当最内层的叶子 Future `poll` 返回 `Poll::Pending` 时，这一“挂起”语义并不会停留在某个中间栈帧内部，而是会沿着整条调用链逐层向上返回。也就是说，叶子 Future 返回 `Poll::Pending` 后，其父 Future 的 `poll` 会结束返回，再上一层 `poll` 也会继续结束返回，直到最终返回给执行器。随着这条返回链完成，先前为各层 `poll` 调用临时建立的中间栈帧都会被依次弹出，栈指针 `sp` 最终回退到执行器发起本轮 `poll` 调用之后的位置。

这正是“无栈协程”在运行时行为上的核心特征：Future 在挂起时并不会像线程那样保留一段独立栈空间来保存调用现场，而是仅将继续执行所需的状态保存在状态机对象本身中；一旦返回 `Poll::Pending`，调用栈便会完全退回，等待下一次由执行器重新进入 `poll`。从这一角度看，Rust Future 保存的是“状态”，而不是“栈”。

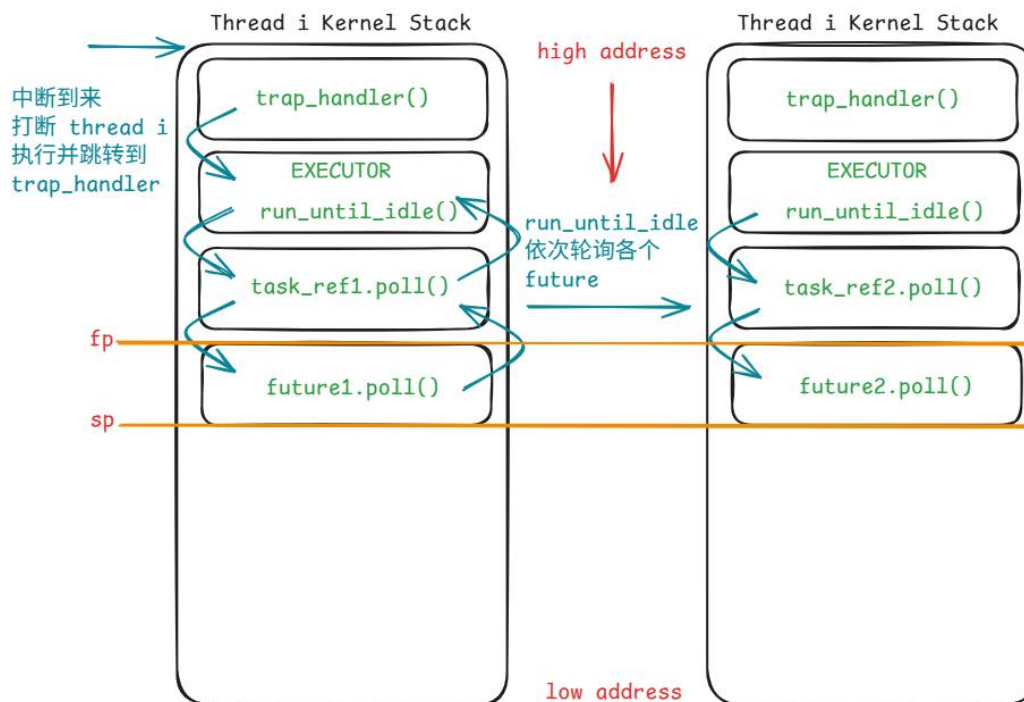


图 4.2 Future 返回 Poll::Pending 后调用栈逐层回退示意图

4.2.2 多核下 sleep 响应误差测试

为了验证本文实现的异步 timer 机制在 rCore-N 多核调度环境下是否能够按预期触发定时事件，本文设计了三组 sleep_blocking 测试实验。sleep_blocking 是用户库提供的阻塞式 sleep 接口封装，其底层会进入本文实现的 sys_sleep，因此可以用于观察异步 timer 与系统调用路径结合后的实际行为。

本节测试并不以证明严格实时性为目标，而是从基础响应误差、调度压力下的并发稳定性以及不同到期时间的处理顺序三个角度验证异步 timer 的基本可用性。由于本文当前的 sys_sleep 仍采用 suspend-loop 折中实现，测试结果中的误差不仅来自 timer 本身，也包含 S Timer Trap 处理、timer queue 检查、Waker 唤醒、executor poll、任务重新调度以及控制台输出等开销。因此，本节重点关注 timer 是否丢失、任务是否能够按预期返回、多个并发 timer 是否均能被处理，以及实际响应误差是否处于可解释范围内。

此外，本文异步执行器使用 Ready、Running 和 Pending 三种状态管理异步任务。执行器轮询 Ready 任务，任务返回 Pending 后等待 timer 到期；当 timer 到期后，Waker 将对应任务重新标记为 Ready，随后由 executor 再次轮询。因此，本节测试不仅验证 sys_sleep 的用户态行为，也间接验证了异步任务状态在 Ready/Running/Pending 之间转换的正确性。

本节统一使用如下指标描述 sleep 响应行为：

```
target_ms = 目标等待时间
start_ms  = 调用 sleep_blocking 前的时间戳
end_ms    = sleep_blocking 返回后的时间戳
elapsed_ms = end_ms - start_ms
error_ms  = elapsed_ms - target_ms
```

其中，`elapsed_ms` 表示用户进程从进入 `sleep_blocking` 到返回用户态之间实际经过的时间，`error_ms` 表示实际等待时间相对目标等待时间的偏差。理想情况下，`elapsed_ms` 应接近 `target_ms`，`error_ms` 应接近 0。但在实际系统中，`elapsed_ms` 通常会略大于 `target_ms`，因为 timer 到期后仍需经历中断处理、异步任务唤醒、执行器轮询和用户任务重新调度等步骤。

以下三组测试均在 QEMU 中执行。QEMU 启动时设置的 `-smp` 参数即 hart 数始终为 4。

(1) 基础响应误差测试

第一组实验用于验证单个进程调用 `sleep_blocking` 时能否正常返回，并观察不同目标等待时间下的响应误差。测试程序在调用 `sleep_blocking` 前后分别调用 `get_time()` 获取毫秒级时间戳，并计算实际经过时间。

单个测试进程的核心逻辑如下：

```
let start = get_time();
sleep_blocking(target_ms);
let end = get_time();

let elapsed_ms = end - start;
let error_ms = elapsed_ms as isize - target_ms as isize;

println!(
    "target = {} ms, elapsed = {} ms, error = {} ms",
    target_ms, elapsed_ms, error_ms
);
```

本实验选取多个不同的目标等待时间，例如 10ms、50ms、100ms 和 500ms。每个目标等待时间重复运行多次，并统计平均实际时间、平均误差、最小误差和最大误差。通过比较不同 `target_ms` 下的 `elapsed_ms` 和 `error_ms`，可以观察 timer 响应误差随目标等待时间变化的情况。一般而言，当目标等待时间较短时，固定的中断处理和调度恢复开销在总等待时间中占比更高，因此相对误差可能更明显；当目标等待时间增大后，固定开销的相对影响会降低。

在 Rust user shell 中手动多次运行 `sleep_blocking.rs` 进程，设置不同的目标时间各运行 10 次，实验结果整理如下：

表 4.1 不同目标等待时间下 sleep_blocking 的响应结果

目标 时间 /ms	1	2	3	4	5	6	7	8	9	10
10	12	12	12	12	12	11	11	11	12	12
50	53	52	51	52	51	52	52	52	51	53
100	102	102	101	102	101	102	102	101	102	102
500	502	501	501	501	502	501	502	501	501	502

该实验主要验证异步 timer 的基本正确性：单个 sleep_blocking 调用应当能够在目标时间之后返回，不应出现提前返回、永久等待或内核异常。

(2) 并发 sleep 与 CPU load 压力测试

第二组实验用于验证在多核调度压力下，本文实现的 sleep_blocking 是否仍能正确完成定时等待。与第一组实验不同，本实验除创建多个 sleep_blocking(100) 进程外，还额外创建若干个 CPU 密集型 cpu_load 进程。cpu_load 进程在用户态持续执行计算循环，从而使系统中始终存在可运行任务，模拟 ready queue 存在持续调度压力的场景。

该实验的父进程首先创建若干个 cpu_load 子进程，然后创建多个 sleep_blocking 子进程。每个 sleep_blocking 子进程仍然执行记录起始时间、调用 sleep_blocking(100)、记录结束时间并输出 elapsed_ms 的逻辑。父进程等待所有 sleep_blocking 子进程退出后，再通过 rCore-N 的用户态软中断消息机制向各个 cpu_load 进程发送退出消息，并通过 waitpid 回收这些进程。

父进程核心逻辑如下：

```
const CPU_LOAD_NUM: usize = 4;
const SLEEP_PROCESS_NUM: usize = 10;
const EXIT_MSG: usize = 15;

let cpu_load_pid: [usize; CPU_LOAD_NUM] =
    array_init::array_init(|_| spawn("cpu_load\0") as usize);

let sleep_blocking_pid: [usize; SLEEP_PROCESS_NUM] =
    array_init::array_init(|_| spawn("sleep_blocking\0") as usize);

let mut exit_code: i32 = 0;

// 等待所有 sleep_blocking 子进程完成测试
for i in 0..SLEEP_PROCESS_NUM {
    waitpid(sleep_blocking_pid[i], &mut exit_code);
}
```

```
// sleep 测试结束后，通知 cpu_load 进程退出
for i in 0..CPU_LOAD_NUM {
    send_msg(cpu_load_pid[i], EXIT_MSG);
    waitpid(cpu_load_pid[i], &mut exit_code);
}
```

其中，cpu_load 进程持续执行计算任务，直到收到父进程发送的软中断消息 EXIT_MSG 后设置退出标志并结束运行。这样可以保证在 sleep_blocking 测试过程中，系统中始终存在 CPU 密集型进程参与调度，从而更真实地反映多核调度压力下异步 timer 的响应情况。

该实验可以与“无 CPU load”的并发 sleep 场景进行对比。例如，先测试仅有 20 个 sleep_blocking(100) 子进程时的误差分布，再测试 4 个 cpu_load 进程与 10 个 sleep_blocking(100) 子进程同时存在时的误差分布。

实验结果整理如下：

表 4.2 不同 CPU load 压力下 sleep_blocking 的响应结果

CPU load 进程数	sleep 进程数	平均耗时/ms	最小耗时/ms	最大耗时/ms	平均误差/ms
0	10	102.2	101	104	2.2
1	10	101.5	101	103	1.5
4	10	103.2	101	107	3.2

本实验重点观察两个方面。第一，在存在 CPU 密集型任务的情况下，所有 sleep_blocking 子进程是否仍能正常返回，不出现 timer 丢失、任务永久等待或内核 panic。第二，加入 cpu_load 后，sleep_blocking 的平均误差和最大误差是否增大。

若加入 cpu_load 后误差增大，这是符合预期的。因为本文当前的 sys_sleep 并不是严格意义上的 blocked sleep，而是基于 suspend-loop 的工程折中。timer 到期后，sleep 任务还需要等待调度器再次选择它运行；当 ready queue 中持续存在 CPU 密集型任务时，这一调度恢复延迟可能会增大。因此，该实验不仅验证了异步 timer 在调度压力下的可用性，也能够体现当前 suspend-loop 方案相对于理想 blocked sleep 的局限性。

(3) 不同到期顺序测试

第三组实验用于验证异步 timer queue 是否能够按照不同定时事件的到期时间正确处理多个 sleep 请求。与第二组实验中多个进程均调用 sleep_blocking(100) 不同，本实验在同一个用户态测试程序中创建多个子进程，并让不同子进程分别调

用不同目标时间的 `sleep_blocking`。如果 `timer queue` 能够正确按照到期时间组织定时事件，则较短目标时间的子进程应当整体上更早返回。

本实验采用单文件测试程序。父进程首先定义一组故意乱序排列的目标 `sleep` 时间：

```
const CASE_NUM: usize = 4;
const TARGETS_MS: [usize; CASE_NUM] = [100, 10, 50, 20];
```

随后，父进程通过循环调用 `fork()` 创建 4 个子进程。由于 `fork()` 会复制当前进程的执行现场，因此每个子进程可以根据循环变量 `i` 读取对应的 `TARGETS_MS[i]`，从而在同一个测试程序中实现不同子进程调用不同 `sleep` 时间的效果。子进程的核心逻辑如下：

```
let target_ms = TARGETS_MS[i];

let start = get_time();
sleep_blocking(target_ms);
let end = get_time();

let elapsed = end - start;
let error = elapsed as isize - target_ms as isize;

println!(
    "[sleep order result] pid = {}, index = {}, target = {} ms, \
    start = {} ms, end = {} ms, elapsed = {} ms, error = {} ms",
    getpid(),
    i,
    target_ms,
    start,
    end,
    elapsed,
    error
);
```

父进程在创建所有子进程后，通过 `waitpid` 等待它们全部退出。由于本实验中子进程的创建顺序被故意设置为：

```
100 ms -> 10 ms -> 50 ms -> 20 ms
```

因此，如果 `timer queue` 按照到期时间正确处理定时事件，则从时间戳角度看，子进程的返回顺序应当大体接近：

```
10 ms -> 20 ms -> 50 ms -> 100 ms
```

需要注意的是，在多核环境下不能简单地以终端打印顺序作为唯一判断依据。原因在于，子进程返回用户态后还需要经过调度器重新选择、用户态打印输出以及串口输出锁等路径，不同 `hart` 上的进程输出顺序可能与实际 `timer` 到期顺序存在偏差。因此，本实验主要依据每个子进程输出的 `end` 时间戳进行判断：将所有子进程的结果按照 `end` 从小到大排序后，观察其对应的 `target_ms` 是否大体符合从

小到大的到期顺序。

运行 `sleep_blocking2` 进程，实验结果整理如下：

表 4.3 不同目标时间并发 `sleep` 的返回顺序测试结果

创建顺序	进程编号	目标时间 target/ms	开始时间 start/ms	结束时间 end/ms	实际经过 elapsed/ms	响应误差 error/ms	按 end 排序后的返回顺序
1	pid 2	100	5206	5306	100	0	4
2	pid 3	10	5216	5232	16	6	1
3	pid 4	50	5224	5274	50	0	3
4	pid 5	20	5234	5255	21	1	2

该实验可以重复运行多轮，并统计“按 end 时间戳排序后是否符合预期顺序”。例如，若每轮预期顺序均为 10ms -> 20ms -> 50ms -> 100ms，则可以统计多轮测试中的顺序正确次数、顺序异常次数和正确率：

表 4.4 不同目标时间并发 `sleep` 的返回顺序统计结果

测试轮数	预期顺序	顺序正确次数	顺序异常次数	正确率
10	10ms -> 20ms -> 50ms -> 100ms	10	0	100%

如果实验结果显示，大部分测试轮次中较短目标时间的子进程都具有更早的 end 时间戳，则说明异步 `timer queue` 能够按照到期时间较为正确地组织和处理定时事件。若出现少量顺序偏差，则需要结合具体 `elapsed` 和 `error` 进行分析：如果不同目标时间之间间隔较小，或者系统在多核调度、`executor poll`、串口输出等路径上产生额外延迟，实际返回顺序可能出现轻微扰动。但若长期出现明显反序，例如 100ms 任务稳定早于 10ms 任务返回，则说明 `timer queue` 的到期时间管理或唤醒逻辑存在问题，需要进一步排查。

小结

通过上述三组实验，可以从不同角度验证本文异步 `timer` 的基本正确性。基础响应误差测试验证单个 `sleep_blocking` 调用能否按目标时间附近返回；并发 `sleep` 与 CPU load 压力测试验证在 `ready queue` 中持续存在 CPU 密集型任务时，多个 `sleep timer` 是否仍能正常到期并返回；不同到期顺序测试则验证 `timer queue` 是否能够根据到期时间正确组织和处理定时事件。

总体而言，本文当前实验重点在于验证异步 `timer` 原型的可用性，而不是证

明其具备严格实时性。由于当前 `sys_sleep` 仍是基于 `suspend-loop` 的工程折中，实际响应时间会受到调度恢复时机影响；同时，多核调度、`executor poll`、CPU 密集型任务竞争和输出开销也会引入额外误差。因此，实验结果应结合实现约束进行解释：只要各组测试中任务均能在目标时间之后返回，多个并发 `timer` 不发生丢失或死锁，并且不同目标时间的返回顺序大体符合预期，就可以说明本文实现的 `Future/Waker` 异步 `timer` 已经能够正确接入 `rCore-N` 的 `S Timer` 路径。

4.3 本章小结

本章围绕 `rCore-N` 中异步 `timer` 模块的设计、实现与验证展开。首先，本文分析了在 `rCore-N` 中实现异步 `timer` 面临的主要约束。由于 `rCore-N` 运行在多 `hart` 环境下，多个 `hart` 共享全局 `ready_queue`，如果在 `timer` 中断上下文中直接访问该队列并重新唤醒阻塞任务，容易引入中断上下文持锁和多核调度一致性风险。因此，本文没有直接恢复 `rCore-2025S` 中的 `blocked sys_sleep`，而是在当前实现中采用了 `suspend-loop` 的折中方案。

其次，本章介绍了本文实现的异步 `timer` 原型。该模块以 `Timer Future` 表达定时等待，以 `TaskHeader`、`TaskRef` 和 `RawWaker` 建立任务唤醒链路，以每 `hart` 独立的 `TimerDriver` 和 `Executor` 组织异步任务执行，并使用基于 `Vec` 的 `timer queue` 保存 `(at, waker)` 定时项。当 `Timer Future` 尚未到期时，执行器将当前任务的 `Waker` 注册到 `time driver`；当 `S Timer` 中断到来后，`time driver` 检查到期事件并调用对应 `Waker`，使异步任务重新变为可轮询状态。

再次，本章说明了本文在执行上下文上的关键取舍。本文最初参考 `Embassy` 的中断执行器思路，考虑将 `wake` 和 `executor poll` 拆分到不同中断上下文中执行。但在 `rCore-N` 中，`OS` 时间片逻辑与 `S Timer Trap` 上下文紧密相关，如果将 `poll` 移到 `S Software Interrupt` 中执行，可能破坏 `suspend_current_and_run_next()` 的上下文语义。因此，本文最终选择在 `S Timer Trap` 中统一完成 `timer` 到期处理、`Waker` 唤醒和 `executor poll`，并通过 `PENDING_OS_TICK` 标志避免 `executor poll` 重入和中途切换上下文。

最后，本章通过理论分析和实验验证说明了异步 `timer` 原型的基本可用性。分析本文代码实现中的 `Rust Future` 无栈协程，说明 `Future` 返回 `Poll::Pending` 后调用栈回退到执行器附近；多核下 `sleep` 响应测试则通过基础响应误差测试、并发 `sleep` 与 `CPU load` 压力测试、不同到期顺序测试，验证 `sleep_blocking` 是否能够在目标时间附近返回，以及 `timer queue` 是否能够按照到期时间组织和处理定时事件。

第 5 章 总结和展望

5.1 论文总结

本文围绕“Rust 异步时间驱动模块设计与实现”这一主题，以 rCore-N 为实验平台，研究并实现了一个基于 Rust Future/Waker 机制的异步 timer 原型模块。全文工作可以概括为三个方面，与绪论中提出的主要内容相对应。

第一，本文分析了已有异步机制及相关 timer 实现。本文首先介绍 Rust Future 无栈协程模型，说明 Future 在返回 Poll::Pending 时并不会保留独立运行栈，而是将跨越 await 点的状态保存在 Future 状态机对象中，并依赖 Waker 在事件到来后重新进入可轮询状态。随后，本文分析 Embassy 异步运行时中的任务模型、执行器、Waker、timer queue 和中断唤醒链路，明确异步 timer 所需的关键环节。最后，本文比较 rCore-2025S 与 rCore-N 的 timer 机制和 sleep 语义差异，指出 rCore-2025S 采用固定 tick 与软件 timer heap 检查相结合的阻塞式 sleep 方案，而 rCore-N 虽然引入 virtual timer 机制，但当前用户态 sleep 仍然更接近忙等式延时，尚未形成 Future/Waker 式异步等待模型。

第二，本文在 rCore-N 中设计并实现了异步 timer 原型模块。该模块以 Timer Future 表达定时等待，以 TaskHeader、TaskRef 和 RawWaker 建立任务唤醒链路，以每 hart 独立的 TimerDriver 和 Executor 组织异步任务执行，并使用基于 Vec 的 timer queue 保存 (at, waker) 定时项。与 rCore-2025S 的固定 tick 检查和 rCore-N 的 (time, pid) virtual timer 相比，本文实现进一步将定时事件绑定到 (time, Waker)，使 timer 到期后能够直接唤醒具体 Future 任务，从而更贴合 Rust Future/Waker 异步执行模型。针对 rCore-N 中多核共享 ready_queue 和 S Timer Trap 上下文约束，本文没有直接恢复 blocked sleep，也没有直接照搬 Embassy 的中断执行器，而是采用 suspend-loop sys_sleep 与 S Timer Trap 内统一 wake/poll 的折中实现，保证了 OS 时间片调度路径与任务切换上下文的一致性。

第三，本文对异步 timer 模块进行了实验验证。多核下 sleep 响应测试通过基础响应误差测试、并发 sleep 与 CPU load 压力测试、不同到期顺序测试，验证了多个 sleep_blocking 任务能否正确注册 timer、等待到期、被 Waker 唤醒并恢复执行。实验结果表明，本文实现的异步 timer 原型能够接入 rCore-N 的 S Timer 路径，并在多核环境下完成基本的定时等待与任务恢复。

综上，本文完成了一个面向 rCore-N 的 Rust 异步 timer 原型实现，验证了

Future/Waker 机制接入操作系统内核 timer 路径的可行性。该实现虽然仍属于原型性质，但已经形成了从 Future 挂起、Waker 注册、timer 到期、中断唤醒到 executor 重新 poll 的完整链路，为后续在 rCore-N 中继续探索异步驱动、异步 I/O 和多核异步调度机制提供了基础。

5.2 目前实现的问题与展望

5.2.1 异步 timer 的问题

本文在 rCore-N 中实现了一个基于 Rust Future/Waker 的异步 timer 原型，并将其接入 S Timer Trap 路径，用于支持后续 sys_sleep 实验。实验结果能够说明该机制已经具备基本可用性：Timer Future 能够注册等待事件，S Timer 中断能够触发 Waker，执行器能够重新 poll 已就绪任务，用户态 sleep_blocking 也能够目标时间附近返回。然而，从系统完整性和性能角度看，当前实现仍然存在若干局限。

第一，当前 sys_sleep 仍然是 suspend-loop 式折中实现，并不是真正意义上的 blocked sleep。理想情况下，调用 sleep 的任务应当从 ready_queue 中移除，并在 timer 到期后由内核重新唤醒。但在 rCore-N 的多核共享调度队列结构下，timer 中断路径中直接访问 ready_queue 并持锁会带来稳定性风险，因此本文当前实现选择让 sleep 任务循环调用 suspend_current_and_run_next() 主动让出 CPU。该方案可以避免在中断上下文中直接操作共享调度队列，但 sleep 任务并没有真正退出就绪调度体系，仍可能在等待期间被反复调度，从而产生额外开销。

第二，当前 timer queue 使用 Vec 保存等待项，插入去重和到期检查都需要线性扫描。具体而言，注册 timer 时需要通过 Waker::will_wake() 遍历已有项以避免重复插入；处理到期事件时，也需要扫描整个队列以唤醒到期任务并计算下一次最近到期时间。该实现简单直观，适合当前原型验证，但在大量 timer 并发注册的情况下，其 $O(n)$ 开销可能成为瓶颈。后续可以考虑使用二叉堆、BTreeMap 或 Embassy 式 intrusive timer queue 优化定时事件管理。

第三，当前异步任务生命周期管理仍然较为原型化。本文使用 TaskRef 封装 TaskHeader 原始指针，并将其放入 RawWaker 中用于唤醒路径。这种设计降低了 Waker 唤醒路径中的查找成本，但也使任务释放、重复唤醒和取消 timer 等场景更加复杂。当前实现主要面向定时任务正常完成的实验路径，尚未完整处理任务取消、Future drop、timer 项撤销和任务资源回收等问题。后续若将该机制扩展为通用异步运行时，需要进一步完善任务生命周期管理。

第四，当前 TimerDriver 和 Executor 均采用每 hart 独立实例的设计。这降低

了跨 hart 同步复杂度，也符合本文实验中 timer 任务在本 hart 上注册和处理的假设。但该设计也意味着异步任务当前绑定在创建时所在 hart 的 executor 上，不支持跨 hart 迁移和负载均衡。若后续希望在多核环境下支持更通用的异步任务调度，就需要进一步设计跨 hart wake、任务迁移或 per-hart work stealing 等机制。

第五，当前 time driver 内部使用 Cell、RefCell 等结构管理 alarm 状态和 timer queue，并通过静态数组为每个 hart 保存独立驱动实例。在当前实现假设下，这些结构主要由对应 hart 的 S Timer 路径访问，因此能够支撑原型实验。但如果后续支持跨 hart 注册 timer、远程唤醒或任务迁移，那么现有同步假设将不再充分，需要重新审查 time driver 的并发安全性，并引入更严格的锁或无锁同步设计。

第六，本文最终将 wake 与 poll 都放在 S Timer Trap 上下文中执行，这是为了保证 OS 时间片调度路径与 suspend_current_and_run_next() 的上下文一致性。然而，这也意味着 executor poll 的执行时间会出现在 timer interrupt 路径中。如果一次中断中存在较多 Ready 异步任务，或者某些 Future 的 poll 开销较大，就可能延长 S Timer Trap 的处理时间，进而影响系统响应。后续可以进一步探索更合理的上下半部拆分方式，例如保留 S Timer Trap 中的最小唤醒逻辑，再通过 deferred execution 或 per-hart 内核线程执行较重的 poll 工作，同时仍保证 OS tick 调度路径的上下文正确性。

综上，本文当前实现更适合作为 rCore-N 中引入 Rust 异步机制的原型验证，而不是最终形态的高性能异步 timer 子系统。后续优化的重点应包括：设计不在中断上下文直接持有共享 ready_queue 锁的 deferred wake 机制；将 suspend-loop sleep 改造为真正的 blocked sleep；优化 timer queue 的数据结构；完善异步任务生命周期管理；以及探索适合多 hart 内核环境的异步执行器调度模型。

5.2.2 后续推进设想

针对当前实现中暴露出的问题，后续可以从以下几个方向继续推进。

第一，设计 deferred wake 机制，实现真正的 blocked sleep。当前 sys_sleep 采用 suspend-loop 折中实现，虽然避免了在 timer 中断上下文中直接持有共享 ready_queue 锁，但 sleep 任务没有真正退出就绪调度体系，仍可能在等待期间被反复调度。后续可以考虑将 timer 中断中的唤醒操作拆分为两个阶段：中断上下文只记录“某个任务需要被唤醒”的事件，并将其放入 per-hart deferred wake 队列；真正修改任务状态和访问全局 ready_queue 的操作则延后到安全的调度上下文或专门的内核线程中执行，这一思路与内核中常见的 deferred work / workqueue 机制具有相似性^[28]。这样既可以避免中断上下文复杂持锁，又可以实现接近传统

blocked sleep 的等待语义。

第二，优化 timer queue 的数据结构。当前异步 timer 原型使用 Vec 保存 (at, waker) 项，插入去重和到期检查都需要线性扫描。对于少量 timer 事件，该方案实现简单、调试方便；但当并发 timer 数量增加时， $O(n)$ 扫描开销可能影响 timer interrupt 处理时间。后续可以考虑使用二叉堆、BTreeMap、时间轮或 Embassy 式 intrusive timer queue 来优化 timer 事件管理^[11,29]。其中，二叉堆适合高效获取最早到期事件，BTreeMap 适合按时间戳组织多个 timer，时间轮适合大量近似粒度定时事件，而 intrusive queue 则可以进一步减少动态内存分配和额外节点开销。

第三，完善异步任务生命周期管理。当前 TaskRef 通过原始指针连接 TaskHeader 和 RawWaker，适合验证 Waker 唤醒链路，但对于完整异步运行时而言，还需要处理 Future 完成后的资源释放、任务取消、timer 撤销、重复 wake、迟到 wake 等复杂场景。后续可以为异步任务引入更明确的生命周期状态，例如 Completed、Cancelled 等，并设计安全的任务释放流程，避免 Waker 持有已释放任务指针或 timer queue 中残留过期 Waker。

第四，重新设计 wake 与 poll 的上下文拆分。本文将 wake 与 poll 都放在 S Timer Trap 中执行，是为了保证 OS tick 对应的任务切换逻辑运行在正确的 S Timer Trap 上下文中。但这一设计也可能导致 timer interrupt 处理时间变长，尤其当一次中断中有较多 Ready 异步任务时，executor poll 会延长 trap 路径。后续可以探索更合理的上下半部结构：S Timer Trap 只完成最小化的到期事件检查和唤醒标记，而较重的 executor poll 交由 per-hart 内核线程、软中断或 deferred execution 机制执行。同时，需要为 OS tick 这类特殊任务保留正确的调度上下文，避免破坏 suspend_current_and_run_next() 的现场语义。

第五，探索跨 hart 的异步任务调度。当前实现中，每个 hart 拥有独立的 TimerDriver 和 Executor，异步任务绑定在创建时所在 hart 上。这种设计降低了跨 hart 同步复杂度，但也限制了负载均衡能力。后续可以研究跨 hart wake 和任务迁移机制，例如当某个 timer 在 hart A 上到期，但目标任务需要在 hart B 上执行时，可以通过 IPI 或 per-hart 唤醒队列通知目标 hart；也可以进一步设计 work stealing，使空闲 hart 能够协助执行其他 hart 上积压的异步任务^[30]。

第六，将异步 timer 机制推广到更多驱动场景。Timer 是异步驱动中较适合作为切入点的设备，因为其事件来源明确、状态较少、唤醒条件清晰。后续可以在此基础上继续探索异步串口、异步块设备或异步网络驱动。例如，对于串口输入，可以在没有数据时返回 Poll::Pending 并注册 Waker，在 UART 中断到来后唤醒等待任务；对于块设备，可以在请求提交后挂起 Future，等待设备完成中断后恢复

执行。这样可以逐步形成一套面向 rCore-N 的异步驱动抽象。

第七，补充更系统的性能评估。本文当前实验主要验证异步 timer 的基本可用性和响应误差，尚未对 CPU 利用率、上下文切换次数、timer queue 长度、trap 处理耗时、executor poll 耗时等指标进行系统量化。后续可以加入内核级统计计数器，分别记录 timer interrupt 次数、Waker 唤醒次数、executor poll 次数、任务重新调度次数以及 sleep 期间的 CPU 消耗，并与 busy-wait sleep 和理想 blocked sleep 进行对比，从而更全面地评估异步 timer 的性能收益与开销来源。

第八，进一步清理和抽象模块接口。当前异步 timer 仍然是面向实验目标的原型实现，与 rCore-N 原有 timer、trap 和调度路径耦合较强。后续可以将 Future 执行器、Waker、timer queue、time driver 和 OS tick 处理进一步分层，形成更清晰的接口边界。例如，time driver 只负责硬件时间和 timer queue，executor 只负责异步任务调度，OS tick 作为一种特殊 timer 事件单独适配调度器。这样有助于后续将 async timer 扩展为更通用的异步运行时子系统。

总体而言，后续工作的核心方向是从“可运行的异步 timer 原型”走向“可扩展、可维护、可评估的内核异步运行机制”。本文已经验证了 Future/Waker 接入 rCore-N S Timer 路径的基本可行性，但要真正形成高效可靠的异步驱动框架，还需要在 blocked sleep、deferred wake、任务生命周期、timer 数据结构、多核调度和性能评估等方面继续完善。

参考文献

- [1] LAUER H C, NEEDHAM R M. On the duality of operating system structures[J]. ACM SIGOPS Operating Systems Review, 1979, 13(2): 3-19. DOI: 10.1145/850657.850658.
- [2] WELSH M, CULLER D, BREWER E. SEDA: an architecture for well-conditioned, scalable Internet services[C]//Proceedings of the 18th ACM Symposium on Operating Systems Principles. New York: Association for Computing Machinery, 2001: 230-243. DOI: 10.1145/502034.502057.
- [3] ADYAA A, HOWELL J, THEIMER M, et al. Cooperative task management without manual stack management[C]//Proceedings of the USENIX Annual Technical Conference. Berkeley, CA: USENIX Association, 2002: 289-302.
- [4] RUST PROJECT DEVELOPERS. Asynchronous programming in Rust[EB/OL]. [2026-05-28]. <https://rust-lang.github.io/async-book/>.
- [5] RUST PROJECT DEVELOPERS. Future trait in core::future[EB/OL]. [2026-05-28]. <https://doc.rust-lang.org/core/future/trait.Future.html>.
- [6] RUST PROJECT DEVELOPERS. Waker in std::task[EB/OL]. [2026-05-28]. <https://doc.rust-lang.org/std/task/struct.Waker.html>.
- [7] VON BEHREN R, CONDIT J, BREWER E. Why events are a bad idea (for high-concurrency servers)[C]//Proceedings of the 9th Workshop on Hot Topics in Operating Systems. Berkeley, CA: USENIX Association, 2003: 19-24.
- [8] VON BEHREN R, CONDIT J, ZHOU F, et al. Capriccio: scalable threads for Internet services[C]//Proceedings of the 19th ACM Symposium on Operating Systems Principles. New York: Association for Computing Machinery, 2003: 268-281. DOI: 10.1145/945445.945471.
- [9] EMBASSY PROJECT. Embassy[EB/OL]. [2026-05-28]. <https://embassy.dev/>.
- [10] EMBASSY-RS. embassy[EB/OL]. [2026-05-28]. <https://github.com/embassy-rs/embassy>.
- [11] EMBASSY-RS. embassy-time[EB/OL]. [2026-05-28]. <https://github.com/embassy-rs/embassy/tree/main/embassy-time>.
- [12] RCORE-OS COMMUNITY. rCore Tutorial Book v3[EB/OL]. [2026-05-28]. <https://rcore-os.cn/rCore-Tutorial-Book-v3/>.
- [13] RCORE-OS COMMUNITY. rCore-Tutorial-v3[EB/OL]. [2026-05-28]. <https://github.com/rcore-os/rCore-Tutorial-v3>.
- [14] DUSKMOON314. rCore-N[EB/OL]. [2026-05-28]. <https://github.com/duskmoon314/rCore-N>.
- [15] MAN7. epoll(7) — Linux manual page[EB/OL]. [2026-05-28]. <https://man7.org/linux/man-pages/man7/epoll.7.html>.
- [16] KERRISK M, AXBOE J. io_uring paper[EB/OL]. [2026-05-28].

https://kernel.dk/io_uring.pdf.

- [17] ARPACI-DUSSEAU R H, ARPACI-DUSSEAU A C. Operating systems: three easy pieces[EB/OL]. [2026-05-28]. <https://pages.cs.wisc.edu/~remzi/OSTEP/>.
- [18] TOKIO TEAM. Asynchronous programming with Tokio[EB/OL]. [2026-05-28]. <https://tokio.rs/tokio/tutorial/async>.
- [19] SRINIVASAN S, MYCROFT A. Kilim: isolation-typed actors for Java[C]//Proceedings of the 22nd European Conference on Object-Oriented Programming. Berlin: Springer, 2008: 104-128.
- [20] RUST PROJECT DEVELOPERS. Pin in std::pin[EB/OL]. [2026-05-28]. <https://doc.rust-lang.org/std/pin/struct.Pin.html>.
- [21] EMBASSY-RS. embassy-executor[EB/OL]. [2026-05-28]. <https://github.com/embassy-rs/embassy/tree/main/embassy-executor>.
- [22] EMBASSY-RS. multiprio.rs example[EB/OL]. [2026-05-28]. <https://github.com/embassy-rs/embassy/blob/main/examples/rp/src/bin/multiprio.rs>.
- [23] RISC-V INTERNATIONAL. The RISC-V instruction set manual, privileged architecture[EB/OL]. [2026-05-28]. <https://docs.riscv.org/reference/isa/priv/priv-intro.html>.
- [24] RUSTSBI COMMUNITY. rustsbi[EB/OL]. [2026-05-28]. <https://github.com/rustsbi/rustsbi>.
- [25] RISC-V NON-ISA. riscv-sbi-doc[EB/OL]. [2026-05-28]. <https://github.com/riscv-non-isa/riscv-sbi-doc>.
- [26] LINUX KERNEL COMMUNITY. Timers subsystem documentation[EB/OL]. [2026-05-28]. <https://www.kernel.org/doc/html/latest/timers/index.html>.
- [27] EMBASSY-RS. time_driver.rs in embassy-rp[EB/OL]. [2026-05-28]. https://github.com/embassy-rs/embassy/blob/main/embassy-rp/src/time_driver.rs.
- [28] LINUX KERNEL COMMUNITY. Workqueue documentation[EB/OL]. [2026-05-28]. <https://www.kernel.org/doc/html/latest/core-api/workqueue.html>.
- [29] VARGHESE G, LAUCK T. Hashed and hierarchical timing wheels: data structures for the efficient implementation of a timer facility[C]//Proceedings of the 11th ACM Symposium on Operating Systems Principles. New York: Association for Computing Machinery, 1987: 25-38. DOI: 10.1145/41457.37504.
- [30] BLUMOF R D, LEISERSON C E. Scheduling multithreaded computations by work stealing[J]. Journal of the ACM, 1999, 46(5): 720-748. DOI: 10.1145/324133.324234.

附录 A 外文资料书面翻译 1

无须手动栈管理的协作式任务管理

摘要

协作式任务管理能够让程序架构师更容易推理并发问题。支持“事件驱动”编程而反对“多线程”编程的人常常会强调这一性质。然而，这两个术语混淆了若干不同的问题。本文澄清这些问题，并说明如何同时取得两方面的优点：既像“事件驱动”编程的倡导者所建议的那样简化并发推理，又保留“多线程”编程中代码可读性和可维护性的优势。

我们指出，这两种编程风格之间产生混淆的根源，在于两个概念被混在了一起：任务管理和栈管理。这两个问题构成了一个二维设计空间，在这个空间中，“多线程”编程和“事件驱动”编程位于对角相对的位置；而空间中还存在第三个“甜点区域”，能够结合两种编程风格的优势。本文指出手动栈管理和自动栈管理各自可能存在的陷阱，并给出减轻自动栈管理风险的技术。最后，本文展示了适配器机制，使自动栈管理代码和手动栈管理代码能够在同一代码库中互操作。

A.1 引言

我们的团队启动了一个新项目，并面临选择何种编程模型的问题。团队中的每个成员过去都曾被并发问题困扰过，遇到过难以复现、更难定位和修复的错误。我们起初选择遵循自己理解中的社区共识，即“事件驱动”编程模型能够通过减少竞态条件和死锁出现的机会来简化并发问题。然而，随着经验积累，我们意识到流行术语“事件驱动”实际上混淆了若干不同概念；更重要的是，它暗示若想获得更容易的并发推理，就必须接受繁琐的手动栈管理。通过分离这些问题，我们得以实现“两全其美”的效果。

第 2 节定义了两个被混淆后会带来问题的概念，并简要讨论另外三个相关概念，以避免它们与核心思想混为一谈。核心观点是：人们可以选择协作式任务管理带来的推理优势，而不必牺牲自动栈管理所带来的可读性和可维护性。第 3 节聚焦栈管理，说明软件演化如何加剧手动栈管理代码和自动栈管理代码各自的问题，并展示如何缓解自动栈管理中最隐蔽的问题。第 4 节提出一种混合式栈管理

模型，使采用自动栈管理的代码能够与采用手动栈管理的代码共存并在同一程序中互操作；这一模型也帮助我们在开发团队中调和了不同开发者对栈管理方式的分歧。第 5 节讨论我们在两个不同系统中实现这些思想的经验。第 6 节将我们的观察与相关工作进行比较，第 7 节总结本文结论。

A.2 定义

本节定义并描述五个不同概念：任务管理、栈管理、I/O 响应管理、冲突管理和数据划分。这些概念并非完全正交，但将它们分别考虑，有助于理解它们在完整设计中的交互方式。我们首先将这些问题拆开分析，然后再回到它们在常见说法中如何被混淆这一问题上。

A.2.1 任务管理

程序所做的工作通常可以被划分为若干概念上相互独立的任务：每个任务封装一条控制流，所有任务又访问某些公共的共享状态。高性能程序经常采用抢占式任务管理，在这种方式下，任务的执行可以在单处理器上交错进行，也可以在多处理器上并行重叠。与之相反的方法是串行任务管理，即每个任务运行完成之后才开始下一个任务。串行任务管理的优点是不会发生对共享状态的访问冲突；可以在共享状态上定义任务间不变量，并确信当前任务运行期间没有其他任务会破坏这些不变量。然而，当系统希望利用多处理器并行性，或者某些慢任务不能长时间推迟后续任务执行时，这种策略就不合适。

折中的方法是协作式任务管理。在这种方法中，一个任务的代码只会在执行过程中的明确定义点让出控制权；通常只有当该任务必须等待耗时较长的 I/O 时才会让出。若任务之间必须交错执行以避免彼此等待 I/O，但多处理器并行性又不是获得良好应用性能的关键，则这种方法具有价值。

协作式任务管理保留了串行任务管理的部分优点：全局状态上的不变量只需要在任务显式让出控制权时恢复，并且当任务恢复运行时可以假定这些不变量仍然有效。协作式任务管理比串行任务管理更困难的一点在于，如果任务在让出之前的局部状态依赖于全局状态，那么当任务恢复时，这些局部状态可能已经失效。在抢占式任务管理中，如果为了执行慢速 I/O 操作而暂时释放锁，也会出现同样的问题。

采用协作式任务管理的一个代价是：所调用的每个 I/O 库函数都必须被包装，使其不再阻塞，而是发起 I/O 后将控制权让给另一个任务。包装函数还必须安排当前任务在 I/O 完成时重新变为可调度。

A.2.2 栈管理

实现协作式任务管理的一种常见方式，是将程序组织为一组事件处理器。例如，假设一个任务包括接收网络消息、从磁盘读取一个块，然后回复该消息。消息接收是一个事件；一个过程处理该事件并发起磁盘 I/O。磁盘 I/O 结果到达是第二个事件；另一个过程处理该事件并构造网络回复消息。由于当前任务等待磁盘 I/O 时其他任务可以继续推进，因此所需的任务管理效果得以实现。

我们将上述方法称为手动栈管理。正如第 3.1 节所讨论的，问题在于单个概念任务的控制流及其任务特有状态被拆散到多个语言层面的过程中，从而实际上丢弃了语言提供的作用域机制。这个问题很隐蔽，因为它在软件演化时最容易造成麻烦。需要强调的是，人们可以选择协作式任务管理带来的好处，同时仍然利用结构化编程语言提供的自动栈管理。第 3.3 节将说明如何做到这一点。

一些语言内置了透明构造闭包的设施，例如 Scheme 的 `call-with-current-continuation`。在这样的语言中，手动栈管理这一概念基本上不再必要。本文关注的是传统系统语言中的栈管理问题，这类语言通常没有优雅的闭包机制。

A.2.3 I/O 管理

尽管本文主要关注前两个轴，但我们仍然明确提出另外三个轴，以免它们与前两个概念混淆。第一个是同步 I/O 管理与异步 I/O 管理的问题，这与任务管理轴是正交的。如果调用任务在调用点看起来会阻塞，直到 I/O 完成后才继续执行，那么这个 I/O 编程接口就是同步的。异步接口调用则看起来会立即将控制权返回给调用者。调用代码可以发起若干个重叠的异步操作，然后在稍后等待结果以任意顺序到达。

这种并发形式不同于任务管理，因为 I/O 操作可以独立于与之重叠的计算来考虑，原因在于 I/O 本身并不访问计算中的共享状态。遵循任何任务管理方式的代码都可以调用任意一种 I/O 接口。此外，只要有合适的原语，人们可以在异步接口之上构造同步接口，也可以在同步接口之上构造异步接口；我们在自己的系统中确实这样做了。

A.2.4 冲突管理

不同的任务管理方式为共享状态提供了不同粒度的原子性。冲突管理关注如何将可用的原子性转化为避免资源冲突的有意义机制。例如，在串行任务管理中，整个任务对于共享状态而言都是一个原子操作，因此不需要显式机制来避免任务间对共享资源的冲突。在抢占式任务管理的极端情况下，由于其他任务正在并发执行，任务必须确保共享状态上的不变量始终成立。

解决这一问题的一般方法是使用锁、信号量和监视器等同步原语。基于机器或运行时环境提供的小粒度原子操作，同步原语使我们能够构造机制来维护共享状态上的复杂不变量，使其始终成立。同步机制可以是悲观的，也可以是乐观的。悲观机制会将其他任务排除在完成计算所需资源之外。乐观原语则先进行推测性计算；如果计算结果与并发任务发生冲突，机制会重试，必要时在无法取得前进进展时回退到悲观机制。

协作式任务管理或串行任务管理实际上提供了任意大的原子操作：两个显式让出点之间执行的所有代码都是原子执行的。因此，安全地构建许多复杂不变量是直接的。这种方法类似于单处理器操作系统内核中通过屏蔽中断构造原子序列。第 3.3 节将讨论如何在软件演化过程中确保依赖原子性的代码仍然保持原子性。

A.2.5 数据划分

任务管理和冲突管理共同处理对共享状态的潜在并发访问问题。通过划分状态，可以减少发生冲突的机会。例如，任务特有状态不需要考虑并发问题，因为它已经被明确地从共享状态中划分出来。数据在不同分区之间通过值传递；但对于隐式引用，例如一个实际上作为数组索引的值，必须谨慎处理。

显式引入数据分区能够减少共享状态的共享程度，使得在每个分区上编写和推理不变量更加容易；数据划分是相对于前面几种方法而言正交的一种思路。

A.2.6 概念之间的关系

我们已经描述了五个不同概念。它们并非全都精确正交，但分别考虑每个维度选择的影响是有帮助的。就本文而言，最重要的是任务管理和栈管理这两个轴确实是正交的，如图 1 所示。

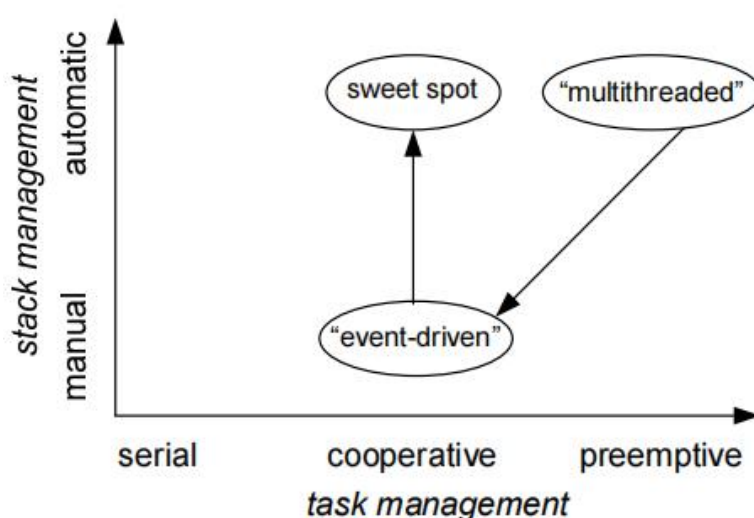


图 1 经常被混淆的两个轴

图 1 的含义是：传统并发编程使用抢占式任务管理，并利用标准语言的自动栈管理。这个设计空间中的点通常被称为“线程式编程”。第二个有趣的点是“事件驱动编程”，其中协作式任务被组织为事件处理器，任务通过将控制权返回给事件调度器来让出执行，并手动展开栈。本文围绕这样一个观察展开：人们可以选择协作式任务管理，同时保留使编程语言具有“结构化”特征的自动栈管理；在图中，这一点被标记为“甜点区域”。

A.3 栈管理

根据前面的图示，人们可能会问：“两种栈管理形式各有什么优缺点？”本节回答这一问题。我们提出每种形式的主要优点和缺点，并重点说明软件演化如何加剧这些缺点。同时，我们还提出一种技术，用于缓解自动栈管理的主要缺点。

A.3.1 自动栈管理与手动栈管理

程序员可以使用自动栈管理或手动栈管理来表达一个任务。使用自动栈管理时，程序员在源语言中将每个完整任务表达为单个过程。这样的过程可以调用会阻塞在 I/O 操作上的函数，例如磁盘请求或远程请求。当任务等待阻塞操作时，其当前状态被保存在过程程序栈上的数据中。这种控制流风格通常与“过程式”这一术语相关联。

相反，手动栈管理要求程序员将任意给定任务的代码拆分成多个事件处理器，这些事件处理器运行到结束而不阻塞。事件处理器是可以由事件处理调度器

响应事件而调用的过程，例如任务启动事件或先前请求的 I/O 返回事件。为了发起 I/O，事件处理器 E1 会提交操作请求，但不会等待回复。相反，E1 会向事件处理调度器注册一个称为续体的任务特有对象。该续体打包了表示 E1 在任务中执行到何处的状态，以及一个指向另一个事件处理过程 E2 的引用，E2 编码了请求的 I/O 完成后应当执行的操作。发起 I/O 并注册续体之后，E1 将控制权返回给事件处理调度器。当表示 I/O 完成的事件发生时，事件处理调度器调用 E2，并将 E1 打包的状态作为参数传入。这种控制流风格通常与“事件驱动”这一术语相关联。

为了说明这两种栈管理风格，考虑一个函数 `GetCAInfo`。该函数在内存哈希表中查找指定证书颁发机构 `id`，并返回指向相应对象的指针。证书颁发机构是签发证书的实体，例如为文件系统用户签发证书。

```
CAInfo GetCAInfo(CAID caId) {
    CAInfo caInfo = LookupHashTable(caId);
    return caInfo;
}
```

假设该函数最初被设计为处理少数全局已知的证书颁发机构，因此所有 CA 记录都可以存储在内存中。我们将这种函数称为纯计算函数：由于它不会为 I/O 暂停，因此不需要考虑其栈如何跨越 I/O 调用被管理，编译器提供的自动栈管理总是合适的。

现在假设该函数演化为支持大量 CA 对象。我们可能希望将哈希表转换为磁盘上的结构，并在内存中缓存正在使用的条目。`GetCAInfo` 变成了一个可能必须为 I/O 让出执行的函数。代码如何演化，取决于它采用自动栈管理还是手动栈管理。下面是使用自动栈管理实现修改后函数的代码：

```
CAInfo GetCAInfoBlocking(CAID caId) {
    CAInfo caInfo = LookupHashTable(caId);
    if (caInfo != NULL) {
        // 在哈希表中找到节点
        return caInfo;
    }
    caInfo = new CAInfo();
    // DiskRead 阻塞等待磁盘 I/O 完成
    DiskRead(caId, caInfo);
    InsertHashTable(caId, caInfo);
    return caInfo;
}
```

若使用手动栈管理实现同样目标，则需要将单个概念函数 `GetCAInfoBlocking` 拆成两个源语言函数，以便第二个函数能够在磁盘 I/O 完成后由事件处理调度器调用并继续执行。下面是保存打包状态和函数指针的续体对象：

```

class Continuation {
    // 当该续体被调度运行时调用的函数
    void (*function)(Continuation cont);

    // 由 I/O 操作设置的返回值，传递给续体
    void *returnValue;

    // 打包的状态
    void *arg1, *arg2, ...;
}

```

下面是原始函数被拆成两个事件处理器后的形式：

```

void GetCAInfoHandler1(CAID caId, Continuation *callerCont) {
    // 如果在缓存中，立即返回结果
    CAInfo *caInfo = LookupHashTable(caId);
    if (caInfo != NULL) {
        // 用结果调用调用者的续体
        (*callerCont->function)(caInfo);
        return;
    }

    // 为磁盘读取分配缓冲空间
    caInfo = new CAInfo();

    // 保存返回地址和活跃变量
    Continuation *cont = new Continuation(&GetCAInfoHandler2,
                                           caId, caInfo, callerCont);

    // 发送请求
    EventHandle eh = InitAsyncDiskRead(caId, caInfo);

    // 通过注册续体，安排收到回复后运行的事件处理器
    RegisterContinuation(eh, cont);
}

void GetCAInfoHandler2(Continuation *cont) {
    // 恢复活跃变量
    CAID caId = (CAID) cont->arg1;
    CAInfo *caInfo = (CAInfo*) cont->arg2;
    Continuation *callerCont = (Continuation*) cont->arg3;

    // 将 CAInfo 对象放入哈希表
    InsertHashTable(caId, caInfo);

    // 现在将结果“返回”给原始调用者
    (*callerCont->function)(callerCont);
}

```

注意，GetCAInfo 的签名与 GetCAInfoHandler1 的签名不同。由于原本由 GetCAInfo 返回的结果要等到稍后 GetCAInfoHandler2 运行时才可用，因此 GetCAInfoHandler1 的调用者必须传入一个续体，供 GetCAInfoHandler2 稍后调

用，以通过续体记录返回所需结果。换言之，在手动栈管理中，向调用者返回控制权和返回值的语句必须通过调用续体过程来模拟。

A.3.2 栈撕裂

在 C++ 等没有闭包支持的传统系统语言中，程序员必须进行大量手动栈管理，才能为 I/O 操作让出执行。注意，上一节中的函数仅仅因为一个 I/O 调用就被拆成了两个部分。如果存在更多 I/O 调用，代码中就会出现更多拆分。当出现 for 循环等控制结构时，情况会进一步恶化。程序员必须解构语言栈，在堆上重建它，并在此过程中降低代码可读性。

此外，调试也会受到影响。当调试器停在 `GetCAInfoHandler2` 中时，调用栈只显示当前事件处理器的状态，并不能提供该被拆分任务到达当前事件处理器调用之前所经历事件序列的信息。理论上，可以通过追踪续体对象手动恢复调用栈；但在实践中，我们观察到程序员会手动优化掉尾调用，从而导致大量栈信息丢失。

总之，对于每个被拆分的例程，程序员都必须手动管理通常由编译器处理的过程式语言特性：

- 函数作用域：现在两个或更多语言函数表示同一个概念函数。
- 自动变量：原本由语言分配在栈上的变量，必须移动到新的状态结构中，并存放在堆上，以便跨越让出点继续存在。
- 控制结构：任何可能阻塞的函数所在基本块的入口点，都必须能够从续体到达，因此必须成为独立的语言层函数。也就是说，包含循环的概念函数必须被拆分成两个以上的片段。
- 调试栈：调试时必须手动恢复调用栈，而对尾调用的手动优化可能使其无法恢复。

软件演化会显著放大函数拆分问题：当一个函数从纯计算函数演化为可能让出执行的函数时，从并发语义发生变化的函数到调用图根部之间每一条路径上的所有函数，都可能必须被拆成两部分。更准确地说，调用图某一分支上的所有函数都必须被拆分，直到遇到一个已经以续体传递形式进行调用的函数为止。我们将这种现象称为“栈撕裂”，并认为它是手动栈管理的主要缺点。注意，和所有全局演化一样，调用图上的函数可能由不同人员维护，这使得修改更加困难。

A.3.3 隐藏的并发假设

手动栈管理的巨大优点是，调用图每一层中的每个让出点都在代码中显式可见。相反，GetCAInfo 中的 DiskRead 调用隐藏了潜在并发。在 DiskRead 调用之前从共享状态中提取的局部状态，可能需要在调用之后重新检查。如果没有注释，程序员无法判断哪些函数调用可能让出执行，也无法判断哪些局部状态因此需要重新验证。

与手动栈管理类似，软件演化也会使这种情况更加糟糕。昨天不会让出执行的调用，明天可能被修改为为了 I/O 而让出。然而，当采用手动栈管理的函数演化为会为 I/O 让出时，其签名会随新的结构发生变化，编译器会提醒那些尚未意识到该演化的调用者。采用自动栈管理时，这种变化在语法上不可见，但它却影响直接或间接调用该演化函数的每个函数的语义。

自动栈管理危险的一面在于，被调用过程的一个语义属性，即是否会让出执行，会极大影响调用过程应该如何编写，但却没有机制检查调用过程是否遵守了这一属性。幸运的是，并发假设可以被显式声明，并以静态或动态方式检查。

静态检查是理想的，因为它能在编译期发现违规。会让出执行的函数用 yielding 属性标记，调用函数中假定不会让出的代码块被标记为 atomic。编译器或静态工具检查调用 yielding 函数的函数自身是否也被标记为 yielding，并检查 atomic 块内部是否没有对 yielding 函数的调用。事实上，人们甚至可以适度滥用异常声明机制来达到这一目的。

动态检查不如静态检查理想，因为违规只有在运行时发生时才会被发现。它仍然有用，因为违规会导致立即失败，而不是以难以追溯原因的方式微妙地破坏系统状态。我们选择动态检查，是因为它实现起来快速而简单。每个依赖原子性的代码块以 startAtomic() 调用开始，以 endAtomic() 调用结束。startAtomic() 函数递增一个私有计数器，endAtomic() 将其递减。当任何函数试图阻塞于 I/O 时，yield() 会断言该计数器为零，否则生成 core dump。

注意，在演化采用自动栈管理的代码时，也可能需要修改从并发语义发生变化的函数向上沿调用图每一条路径延伸的所有函数。然而，手动栈管理意味着每个受影响函数都必须被拆成多个片段；而自动栈管理代码可能不需要修改，或者只需要远不那么侵入性的修改。如果某个函数的局部状态不依赖被调用函数的让出行为，那么调用函数不需要修改。如果调用函数的局部状态受到影响，则必须修改该函数以重新验证其状态；这种手术通常是局部的，不需要大量代码重构。

A.4 混合方法

在我们的项目中，两种栈管理风格各有坚定支持者。有一种混合方法能够使两种风格在同一代码库中共存，并通过适配器将它们连接起来。这种混合方法还使得一个项目可以用一种风格编写，同时包含另一种风格编写的遗留代码。

在 Windows 操作系统中，“线程”采用抢占式调度，而“纤程”采用协作式调度。我们的实现通过在单个线程上调度多个纤程来实现协作式任务管理；任一时刻只有一个纤程处于活动状态。

在我们的设计中，调度器运行在一个特殊纤程 `MainFiber` 上，并同时调度手动栈管理代码，即事件处理器，以及自动栈管理代码。使用自动栈管理编写并希望为 I/O 阻塞的代码，总是运行在 `MainFiber` 之外的纤程上；当它阻塞时，总是将控制权让回 `MainFiber`，由调度器选择下一个要调度的任务。纯计算函数当然可以运行在任意纤程上，因为它们可以从任一上下文自由调用。

两种栈管理代码由同一个调度器调度，原因是 Windows 纤程包只支持从一个纤程显式切换到另一个指定纤程，并没有调用默认纤程调度器的通用 `yield` 操作。实现组合调度器还使我们避免了两个可能相互冲突的调度器并行运行的问题：一个用于事件处理器，另一个用于纤程。

使两种代码风格交互还有其它方法。我们的目标是简化设计，并保留使用手动栈管理的既有代码库。我们的方案保证，任一种风格编写的代码都可以调用另一种风格实现的函数，而无需意识到另一种栈管理规则的存在。

为了说明混合方法，我们给出一个包含两种方向跨风格调用的示例。该示例涉及四个函数：`FetchCert`、`GetCertData`、`VerifyCert` 和 `GetCAInfo`。`FetchCert` 使用 `GetCertData` 获取安全证书，然后调用 `VerifyCert` 验证其有效性。`VerifyCert` 又调用 `GetCAInfo` 获取一个 CA，用于验证证书。若采用串行任务管理，代码大致如下：

```
bool FetchCert(User user, Certificate *cert) {
    // 从可能执行 I/O 的函数获取证书数据
    certificate = GetCertData(user);
    if (!VerifyCert(user, cert)) {
        return false;
    }
}

bool VerifyCert(User user, Certificate *cert) {
    // 获取证书颁发机构信息，然后验证证书
    ca = GetCAInfo(cert);
    if (ca == NULL) return false;
    return CACheckCert(ca, user, cert);
}
```

```

}

Certificate* GetCertData(User user) {
    // 在内存缓存中查找证书并返回结果；否则从磁盘或网络获取
    if (Lookup(user, cert))
        return certificate;
    certificate = DoIOAndGetCert();
    return certificate;
}

```

当然，我们希望重写代码以使用协作式任务管理，使其他任务能够在 I/O 暂停期间运行，同时不同函数可以遵循不同形式的栈管理。假设 `VerifyCert` 使用自动栈管理编写，其余函数 `FetchCert`、`GetCertData`、`GetCAInfo` 使用手动栈管理并采用续体实现。下面我们定义适配器函数，用于在两种风格之间路由控制流。

A.4.1 手动栈管理代码调用自动栈管理代码

图 2 是一个序列图，说明手动栈管理代码如何调用自动栈管理代码。在图中，反方向调用的细节暂时隐藏在虚线框之后。`FetchCert1` 的第一个事件处理器调用 `GetCertData1`，后者发起一个 I/O 操作，整个栈按照手动栈管理方式展开。稍后，当 I/O 回复到达时，调度器执行 `GetCertData2` 续体，它通过函数调用“返回”到 `FetchCert` 的第二个处理器。这是纯粹的手动栈管理。

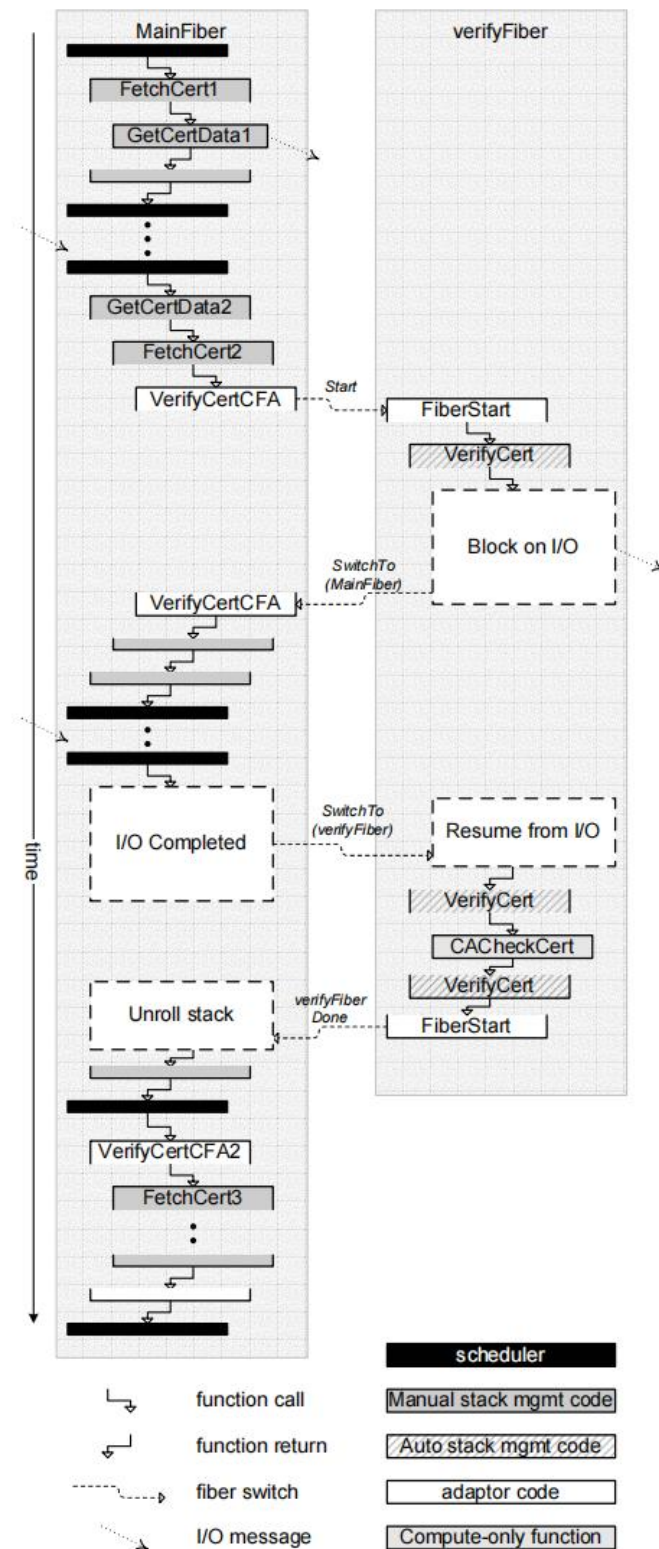


图 2 手动栈管理代码 GetCertData 调用采用自动栈管理的函数 VerifyCert

当使用手动栈管理的函数调用自动栈管理代码时，必须调和两种风格。调用方代码预期自己绝不会阻塞在 I/O 上，而被调用方则预期自己总是可以阻塞 I/O。为调和这些风格，我们创建一个新的纤程，并在该纤程上执行被调用方代码。调用方会在该纤程第一次执行爆发完成后恢复，以便手动展开其栈。该纤程可以多次运行并阻塞于 I/O；当它代表调用方完成工作后，会执行调用方的续体，以恢复调用方任务。因此，调用方代码不会阻塞，而被调用方代码可以按需阻塞。

在我们的示例中，手动栈管理函数 `FetchCert2` 通过适配器调用自动栈管理函数 `VerifyCert`。`FetchCert2` 传入一个指向 `FetchCert3` 的续体，以便最终重新获得控制并执行其实现的最后部分。下面是 CFA 适配器的代码，它被拆成调用部分和返回部分；CFA 表示“Continuation-To-Fiber adaptor”，即续体到纤程适配器。

```
void VerifyCertCFA(CertData certData, Continuation *callerCont) {
    // 在 MainFiber 上执行
    Continuation *vcaCont = new Continuation(VerifyCertCFA2,
    callerCont);
    Fiber *verifyFiber = new VerifyCertFiber(certData, vcaCont);

    // 在 verifyFiber 上开始执行 VerifyCertFiber::FiberStart
    SwitchToFiber(verifyFiber);

    // 当 verifyFiber 阻塞于 I/O 时，控制返回这里
}

void VerifyCertCFA2(Continuation *vcaCont) {
    // 在 MainFiber 上执行，在 verifyFiber 完成后被调度
    Continuation *callerCont = (Continuation*) vcaCont->arg1;
    callerCont->returnValue = vcaCont->returnValue;

    // “返回”给原始调用者 FetchCert
    (*callerCont->function)(callerCont);
}
```

第一个适配器函数接受被适配函数的参数以及调用任务的续体，也即“栈帧”。它构造自己的续体 `vcaCont`，并创建名为 `verifyFiber` 的对象来表示新纤程；该对象记录函数参数和 `vcaCont`，以便在 `verifyFiber` 的工作完成时将控制权转移到 `VerifyCertCFA2`。最后，它执行到 `verifyFiber` 的纤程切换。当 `verifyFiber` 开始运行时，它执行胶水例程 `VerifyCertFiber::FiberStart` 来解包参数并传递给 `VerifyCert`，后者可能会阻塞于 I/O：

```
VerifyCertFiber::FiberStart() {
    // 在 MainFiber 之外的纤程上执行
    // 下面的调用可能阻塞于 I/O，执行实际验证
    this->vcaCont->returnValue = VerifyCert(this->certData);

    // 验证完成，调度 VerifyCertCFA2
}
```

```
scheduler->schedule(this->vcaCont);  
SwitchTo(MainFiber);  
}
```

这个启动函数只是调用 `VerifyCert`。在某个时刻，当 `VerifyCert` 为 I/O 让出执行时，它会在 I/O 函数中通过 `SwitchTo` 调用将控制权切换回 `MainFiber`，而不是在上面 `FiberStart()` 例程所示的调用点处切换。控制权恢复到 `VerifyCertCFA`，后者展开续体栈，即 `GetCertData2` 和 `FetchCert2`，回到调度器。因此，混合任务已经为自动栈管理代码发起的 I/O 阻塞，同时确保事件处理器 `FetchCert2` 不会阻塞。

稍后，当 I/O 完成时，`verifyFiber` 被恢复。`VerifyCert` 完成最后的工作后，控制权返回到 `FiberStart`。`FiberStart` 将返回值放入 `VerifyCertCFA2` 的续体，将其调度执行，并最后一次切换回 `MainFiber`。此时 `verifyFiber` 被销毁。当 `VerifyCertCFA2` 执行时，它以手动栈管理代码通常采用的函数调用方式，将 `VerifyCert` 的返回值“返回”给适配器调用者的续体 `FetchCert3`。

A.4.2 自动栈管理代码调用手动栈管理代码

现在讨论采用自动栈管理的函数调用手动管理其栈的函数时，代码之间如何交互。在这种情况下，前者需要为 I/O 阻塞，而后者只是调度 I/O 并返回。为调和这些要求，我们提供一个适配器：它使用一个特殊续体调用手动栈管理代码，并将控制权让给 `MainFiber`，从而使适配器的调用者保持阻塞。当 I/O 完成时，该特殊续体在 `MainFiber` 上运行，并恢复被阻塞适配器所在的纤程，后者再恢复等待 I/O 结果的原始函数。

图 3 填补了图 2 中缺失的细节，用于说明这种交互。在此示例中，VerifyCert 在调用手动栈管理函数 GetCAInfo 时阻塞于 I/O。VerifyCert 调用适配器 GetCAInfoFCA，后者隐藏 GetCAInfo 的手动栈管理性质，其中 FCA 表示“Fiber-to-Continuation Adaptor”，即纤程到续体适配器：

```
Boolean GetCAInfoFCA(CAID caid) {
    // 在 verifyFiber 上执行
    // 获取一个续体；当它在 MainFiber 上被调用时，会切换回此纤程
    FiberContinuation *cont = new FiberContinuation(FiberContinue,
this);
    GetCAInfo(caid, cont);
    if (!cont->shortCircuit) {
        // GetCAInfo 确实阻塞了
        SwitchTo(MainFiber);
    }
    return cont->returnValue;
}

void FiberContinue(Continuation *cont) {
    if (!Fiber::OnMainFiber()) {
        // 手动栈管理代码没有执行 I/O：只标记为短路
        FiberContinuation *fcont = (FiberContinuation*) cont;
        fcont->shortCircuit = true;
    } else {
        // I/O 后恢复：直接切换到原始纤程
        Fiber *f = (Fiber*) cont->arg1;
        f->Resume();
    }
}
```

适配器 GetCAInfoFCA 设置一个特殊续体，稍后通过 FiberContinue 中的代码恢复 verifyFiber。随后它将该续体传递给 GetCAInfo，GetCAInfo 发起 I/O 操作，并立即返回到它以为的事件处理调度器；当然，在此处控制权实际上返回到 GetCAInfoFCA。由于 I/O 已被调度且未发生短路，GetCAInfoFCA 必须确保控制权尚不返回 VerifyCert；为实现这一效果，它将控制权切换到 MainFiber。

在 MainFiber 上，启动这次纤程执行爆发的续体代码 VerifyCertCFA 多次返回以展开其栈，随后调度器再次运行。最终，I/O 结果到达，调度器执行 GetCAInfo 的剩余工作 GetCAInfo2。GetCAInfo2 填充本地哈希表，并通过调用续体“返回”控制权。在这个例子中，它调用的是传递给 GetCAInfo 的续体 FiberContinue。

FiberContinue 注意到 verifyFiber 确实已经阻塞，于是将控制权切换回该纤程。在该纤程上，适配器 GetCAInfoFCA 的后半部分提取返回值，并将其传递给调用它的自动栈管理代码 VerifyCert。

示例中未走到的短路分支处理这样一种情况：GetCAInfo 无需等待 I/O 即可立即返回结果。在这种情况下，它不能允许控制权传递给调度器。这样做是必要的，因为调用者可以选择性地判断某个例程是否让出了控制权，并据此判断是否需要重新验证局部状态。如果没有短路路径，就无法实现这一重要优化以及第 5 节所描述的相关设计模式。图 4 展示了短路序列：短路代码检测到 GetCAInfo 在本地运行、没有执行 I/O、并立即执行，即“返回到”当前续体。FiberContinue 检测到它不是由调度器直接执行的，于是设置 shortCircuit 标志，防止适配器切换到 MainFiber。

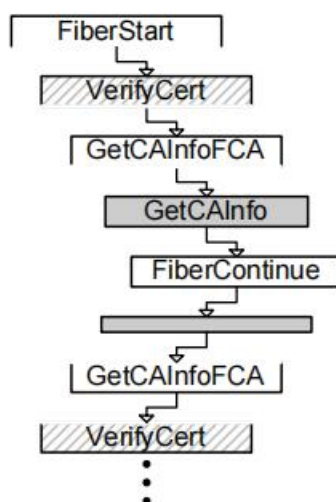


图 4 GetCAInfo 无需执行 I/O 时的变体

A.4.3 讨论

一个重要观察是，有了适配器之后，每种风格的代码都意识不到另一种风格的存在。采用自动栈管理的函数看到的是它所期望的情形：在其栈的深处，控制权可能转移出去，并在稍后带着完整栈返回。同样，事件处理调度器也无法分辨自己调用的是否只是普通的手动栈管理续体序列：适配器巧妙地交换纤维栈，同时看起来像任何其他续体一样。因此，集成两种风格的代码是直接的：对于事件驱动代码来说，纤维执行看起来像一个续体；对于过程式代码来说，续体调度器看起来像任何其他纤维。这种适应性使自动栈管理程序员能够与手动栈管理程序员协作，也使手动栈管理代码库能够通过自动栈管理函数演化，反之亦然。

A.5 实现经验

我们在两个系统中采用了协作式任务管理：Farsite，一个运行在桌面计算机上的分布式、安全、无服务器文件系统；以及 UCoM，一个面向 PDA 等手持设备的无线电话应用。Farsite 代码被设计为在 Windows NT 操作系统上作为守护进程运行，负责处理文件请求。UCoM 系统则为 Windows CE 操作系统设计，是一个带有用户界面和音频支持的客户端应用。

Farsite 系统代码最初采用事件驱动风格，即协作式任务管理和手动栈管理，以便简化对系统并发条件的推理。随着代码库在两年中不断增长和演化，我们逐渐认识到采用手动栈管理的成本，并设计了上一节讨论的混合方法，以将自动栈管理代码引入系统。UCoM 系统则完全使用自动栈管理。

Farsite 使用 Windows NT 提供的协作式线程设施，即纤程。在 Windows 纤程中，每个任务的状态由一个栈表示，控制权通过简单地交换一个栈指针与另一个栈指针来转移，类似 `setjmp` 和 `longjmp`。由于 Windows CE 操作系统没有纤程，UCoM 使用抢占式线程和条件变量来实现协作式线程设施：每个线程阻塞在自己的条件变量上，调度器保证任意时刻最多只有一个条件变量被唤醒。当线程让出时，它阻塞在自己的条件变量上，并通知调度器继续；调度器选择一个就绪线程并通知其条件变量。

我们通过一系列机械生成的宏来实现两个方向上的混合适配器。宏分为两组，分别对应两个适配方向。每组内部又有多个变体，以处理不同数量的参数、返回类型是否为 `void`，以及被调用函数是静态函数还是对象方法等情况；需要多个宏来生成相应语法变体。每个宏都以被适配函数的签名作为参数。宏会声明并创建合适的 Fiber 对象和 Continuation 对象。

我们在两个系统上的经验都是积极的。主观上，我们认为以协作式任务管理作为工作的基础，使我们能够预先避免许多微妙的并发问题。虽然包装 I/O 函数的工作可能繁琐，但它可以自动化；我们发现，预先付出成本以减少微妙竞态条件是一项值得的投入。

两个系统都使用额外线程将阻塞 I/O 操作转换为非阻塞操作，并调度 I/O 操作，许多其他系统也采用类似做法。数据划分避免了 I/O 线程与协作式管理任务共享状态之间的同步问题。

只有当任务无需将控制权让给其他任务就能完成时，协作式任务管理才能避免锁带来的并发问题。为了处理需要执行 I/O 的任务，我们发现通常可以通过采用一种特定设计模式来避免对锁的需求。我们称该模式为“固定模式”。在该模式

中，I/O 操作用于将资源固定在内存中，使其可以在不让出的情况下被操作。注意，固定并不意味着独占：固定的资源被保持在内存中，以避免访问它时阻塞于 I/O；但当其他任务运行时，它们仍然可以自由操作其中包含的数据结构。函数被组织为两个阶段：一个循环，反复尝试执行所有可能让出的操作，直到它们都能在不让出的情况下完成；以及一个原子块，计算结果并将其写入共享状态。

该设计模式的一个重要细节是，可能让出的操作之间可能存在依赖关系。函数可能需要根据先前固定资源的结果进行计算，以决定接下来要固定哪个资源；例如，在 Farsite 中遍历目录树路径时就会发生这种情况。因此，在该设计模式的完全通用版本中，每个可能让出的操作之后都需要检查该操作是否确实让出了执行；如果确实让出，则从循环顶部重新开始。一旦整个循环在没有中断的情况下执行完成，我们就知道固定在内存中的这组资源之间具有我们期望的关系，因为循环的最后一次遍历是原子执行的。

A.6 相关工作

Birrell 对采用抢占式任务管理的传统线程式编程模型给出了很好的概述。在他提出的使用并发的理由中，协作式任务管理除了不能利用多处理器之外，几乎都能提供帮助，这一缺点我们在第 2.1 节中已经提到。Birrell 建议“必须极其细致地将每一份数据与一个且仅一个互斥锁关联起来”；可以将协作式任务管理看作这一建议的极限情形。复杂之处在于，每当任务让出时，它实际上释放了全局互斥锁，并且在恢复时必须重新建立其不变量。不过，即使在抢占式任务管理下，Birrell 也指出“在调用可能阻塞或执行很长时间的低层抽象之前，可能需要先解锁互斥锁”；因此，这种复杂性并不是由选择协作式任务管理引入的。

Ousterhout 指出了抢占式任务管理的陷阱，例如微妙的竞态条件和死锁。我们认为，他所谓的“线程式”模型混淆了抢占式任务管理与自动栈管理，而他的“事件驱动”模型混淆了协作式任务管理与手动栈管理。我们希望说服设计者，这些选择是正交的；Ousterhout 的论点实际上针对的是任务管理决策，而程序员应当利用协作式任务管理易于推理的好处，同时通过自动栈管理利用编程语言的结构化特性。

其他系统设计者提倡非线程式编程模型，是因为他们观察到，对于某类高性能系统，例如文件服务器和 Web 服务器，通过减少上下文切换并精心实现特定应用的缓存敏感任务调度，可以获得显著性能提升。在高负载情况下，这些因素尤其明显，因为线程数量可能变得很大，以至于系统在试图公平分配资源给每个线

程时开始抖动。我们的观点是，用户级线程，即纤程，的上下文切换开销事实上相当低；我们测得纤程包中的切换成本不到过程调用成本的十倍。此外，采用协作式任务管理和自动栈管理同样能够实现特定应用的缓存敏感任务调度：调度器获得的调度机会与事件驱动代码中完全相同；唯一区别在于栈状态是保存在栈上，还是保存在堆上的续体链中。

对于上述类型的应用，处理过程通常被划分为若干阶段。将系统状态划分为不相交阶段是一种数据划分形式，它在粗粒度上处理并发问题。在每个阶段内部，系统设计者仍然必须选择冲突管理、任务管理和栈管理的形式。精心构造阶段可以避免阶段内部出现 I/O 调用；在这种情况下，阶段内部的协作式任务管理退化为串行任务管理，栈管理方面也不会出现区别。在实践中，在阶段之间，一个任务通常会串过多个阶段，读起来类似手动栈管理。通常这些阶段是单调的：任务一旦离开某个阶段，就不会返回该阶段。这至少避免了与循环控制结构相关的撕裂问题。

Lauer 和 Needham 展示了两种编程模型在语法替换意义上等价。用本文的轴来描述他们的模型：其过程式系统采用抢占式任务管理、自动栈管理，使用监视器进行冲突管理，并用一个由这些监视器保护的大数据分区。他们的消息式系统采用手动栈管理，任务状态通过消息传递；由于状态被划分成许多分区，实际上从不并发共享，因此没有冲突需要管理。

值得注意的是，他们在讨论消息式系统时说“过程式接口和全局命名机制都不是很有用”，也就是说，手动栈管理削弱了语言的结构化特性。按照我们的理解，这两种模型都没有采用协作式任务管理，因为两种模型都需要同样细致地推理冲突管理。因此，他们的比较并不是我们所说的多线程编程模型与事件驱动编程模型之间的比较。

A.7 结论

本文通过识别两个可分离的问题，即任务管理和栈管理，澄清了关于“事件驱动”与“线程式”编程模型的持续争论。在将二者分离之后，本文假定采用协作式任务管理，并聚焦在这一语境下的栈管理问题。任务管理策略的选择是基础性的，而栈管理的选择则可以留给个人偏好。不幸的是，“事件驱动编程”这一术语同时混合了协作式任务管理和手动栈管理两个概念。这使许多人没有考虑将易读的自动栈管理编码风格与协作式任务管理结合使用。

软件演化是影响任务管理策略选择的重要因素。当并发假设发生演化时，可能需要对应用实现进行全局的、破坏抽象边界的修改。手动栈管理代码的演化会带来繁琐的代码重构负担，即栈撕裂；而任一种风格代码的演化，都需要重新审视不变量逻辑，因为并发假设发生变化，有时还需要对函数进行局部修改，以重新验证局部状态。

最后，混合模型能够在自动栈管理代码与手动栈管理代码之间进行适配，从而支持不同偏好的程序员协作，并支持不同代码库之间的软件演化。

书面翻译对应的原文索引

- [1] Adya A, Howell J, Theimer M, Bolosky W J, Douceur J R. Cooperative Task Management Without Manual Stack Management[C]//Proceedings of the 2002 USENIX Annual Technical Conference. Monterey, CA: USENIX Association, 2002.

附录 B 外文资料书面翻译 2

物联网与嵌入式系统中使用协程进行异步编程的综述

摘要

许多物联网与嵌入式项目都采用事件驱动架构，因此天然需要异步与并发编程。随着 C++20 相关提案提出在语言层面对协程提供原生支持，现在正是重新审视协程在嵌入式系统开发中使用现状的合适时机。本文考察了既有研究中在资源受限平台上使用或讨论协程的工作，并从软件平台、硬件平台与容量、使用场景、预期收益以及协程应用程序编程接口设计等维度进行了分析。

作者采用系统性映射研究方法，筛选了 2007 至 2018 年间围绕资源受限平台上协程应用的原创研究。在线数据库最初共得到 566 篇候选论文，经筛选后仅保留 35 篇，并据此形成如下分类图景：35 篇论文中有 22 篇使用 C 或 C++；在硬件方面，16 项研究使用 8 位或 16 位处理器，13 项研究使用 32 位处理器；最常见的四类使用场景分别是并发、网络通信、传感器读数与数据流；最主要的预期收益则是代码风格与简洁性、调度能力以及效率。现有工作采用了从原生宏、附加工具链步骤、新语言特性到不可移植汇编代码等多种协程实现技术。本文最终认为，资源受限设备对协程存在广泛而明确的需求；如果 C++ 能提供一种形式化、稳定、支持完善并充分考虑资源受限设备特殊需求的协程实现，那么它将为这类设备带来尤为显著的收益。

关键词：嵌入式系统；资源受限；异步编程；协程；直接式编程风格；调度

B.1 引言

物联网（IoT）正在持续扩张，不仅接入设备的规模和种类不断增加，围绕这些设备构建的应用数量也在上升。与之相伴，作为异构 IoT 应用常见组成部分的资源受限嵌入式系统，其软件工程问题愈发值得关注；而安全性、可靠性与隐私等新挑战的出现，又进一步提升了这一关注的紧迫性。

大量 IoT 与嵌入式系统具有事件驱动架构，因此其软件通常采用异步编程风格：多个任务同时等待外部事件。异步代码之所以难写，在于应用逻辑会被拆散到“发起请求的函数”和“响应就绪时被调用的事件处理函数”之间。随着开发者不

断引入更多事件源（例如超时）及其对应的处理器，这种“分阶段（split-phase）”结构会迅速复杂化；不同分阶段事件之间还可能发生相互作用，导致状态模型的自由度上升。结果是，针对单个事件的相关代码越来越可能分散在多个位置，阅读者必须在不同位置之间来回跳转。应用逻辑被这种分裂式组织方式遮蔽，系统设计与源代码表示之间形成落差，使代码更难理解，也更难维护。

在桌面软件领域，一个重要的解决思路是利用语言级协程和 `promise` 等机制。例如在 C#、JavaScript 和 Python 中，开发者可以用 `await` 关键字等待外部事件，从而以几乎与同步阻塞代码同样清晰的方式编写异步代码。然而，资源受限嵌入式系统仍主要使用 C 或 C++ 开发，而这两种语言长期以来都缺乏对 `await` 模式的原生支持。

当前 C++ 标准化委员会正在讨论将协程纳入语言标准，并已出现至少两套竞争性方案。协程一旦进入 C++，就有机会显著简化嵌入式系统代码。不过，既有 C++ 协程研究和实现未必充分考虑了嵌入式系统的特殊需求，因为最初的实现多依赖并不面向这类平台的编译器。本文特别关注那些内存不足以运行 Linux 或其他通用操作系统的小型嵌入式系统。作者认为，如果 C++ 要引入 `async/await` 与协程模式，就必须同时考虑资源受限平台的需求。

本文通过对学术数据库进行检索并人工审阅每一篇符合条件的论文，对嵌入式系统中协程使用情况进行了系统性映射，从而为“协程为何被使用、如何被使用”提供较为完整的研究图景。基于这一映射，作者进一步从平台、使用场景和实现方式等维度构建了既有研究的分类体系。文章其余部分的安排如下：第 2 节介绍背景，包括资源受限设备上 C/C++ 开发环境、开发者常见问题以及已有解决思路，并讨论 C 与 C++ 中的协程；第 3 节介绍映射研究的方法、方法学选择依据及相关工作；第 4 节展示结果并给出主要观察；第 5 节讨论结果并分析研究空白；第 6 节总结全文并展望未来研究。

B.2 背景

B.2.1 Async/Await 模式

在 IoT 与嵌入式设备程序中，程序流程往往天然具有异步性，例如软件需要等待远端设备或传感器返回结果。若采用朴素方式实现这类流程，通常会得到有限状态机（FSM）与多段分散代码的复杂组合，最终导致源代码复杂、脆弱且缺乏灵活性。另一种常见替代模式是延续传递风格（`continuation-passing style`，

CPS)，这在 JavaScript 中十分常见；但当多个顺序操作叠加时，又容易演变为著名的“厄运金字塔”或“回调地狱”。

相比之下，更优雅的做法是 `async/await` 模式。它能够把 CPS 风格的代码转写成直接式编程风格，让一系列异步步骤在同一个代码块中以自然、顺序化的形式书写。该模式已经在 C#、JavaScript 等语言中取得成功；在 C++ 方向，相关提案则考虑通过 `co_await`、`co_yield` 与 `co_return` 等关键字，或等价语法，将其纳入 C++20 标准。借助 `async/await`，程序员可以像编写同步逻辑那样，以一段连续语句描述实际会异步执行的流程，并同时避免显式使用全局变量。

B.2.2 协程

协程是在函数概念基础上的一种扩展，它额外提供了挂起（`suspend`）与恢复（`resume`）操作。协程可以用于多种目的，例如事件处理器、数据流组织、协作式多任务，以及 `async/await` 模式本身。

当协程挂起时，实现通常会保存其执行位置，并且往往还会保存局部变量状态，但这并非必然。例如 `Protothreads` 是一种面向嵌入式系统的协程实现，它不会保留局部变量状态；相反，协程内部变量必须以静态方式分配。这样的策略减少了上下文切换开销，并带来了可预测的内存占用，但也会使协程失去可重入性；同时，当程序员必须显式管理协程状态时，出错概率也会升高。本文会在嵌入式系统语境下同时考察这两类协程。

协程实现还可以进一步划分为有栈（`stackful`）和无栈（`stackless`）两类。有栈协程拥有自己独立于调用者的栈，因此局部变量能够在挂起期间保存在该栈上。相对地，无栈协程在挂起时会像普通函数返回那样把状态弹出调用栈，因此必须借助额外机制保留状态，例如把局部变量转移到全局或其他存储区域。

此外，无栈协程通常只能从协程自身内部挂起，而不能在其调用的子程序内部挂起。以 C++ 提案 N4680 为例，它采用的是无栈模型，并要求所有 `yield` 或 `return` 语句都必须位于协程本体内部。

无论是有栈还是无栈，都不能被视为适用于所有 C++ 使用场景的普适答案。一些替代技术，例如 `stack slicing`，也曾被用于在无栈实现中保存状态，并进一步支持单线程协作式多任务。

B.2.3 资源受限平台上的既有协程实现

早期的许多实现都借助 C 语言中的宏来为程序加入类似协程的能力。典型例子是 `Duff's device`：它利用了 C 语言 `switch/case` 在缺少 `break` 时的贯穿（`fall-`

through) 行为, 甚至允许将 do ... while 这类代码块交错嵌入 switch 结构中。Tatham 曾在 C 中给出一种协程方案, 利用 Duff's device 通过宏高效实现协程, 而无需手工编写状态机。但他也指出, 这种技巧“几乎违反了书中所有编码规范”; Duff 本人也把它称为“用 switch 实现中断驱动状态机的一种令人反感的方式”。后来 Dunkels 等人在此基础上提出了 Protothreads, 它能在内存受限系统上提供条件阻塞操作, 而无需多栈支持, 并成为广泛使用的实时操作系统 Contiki 的核心组成部分。

Protothreads 以及所有基于 Duff's device 的解决方案都存在两个严重缺陷。第一, 它给 C 程序带来了明显限制: 一旦程序使用 Protothreads, 就无法再安全地使用 switch 语句; 这类错误编译器未必会报出, 但运行时行为可能不可预测。第二, 它不会代表程序员自动管理局部变量状态; 任何需要在多次调用之间保持值的协程内部变量, 都必须声明为 static (即全局存储)。这不仅影响可重入性, 也会影响代码质量。不过, 从编码工作量、内存使用与运行速度来看, 这类方案又极其廉价, 而且由于只用纯 C 实现, 因此具有良好的可移植性。原始库以 C 编写, 之后也被移植到了 C++。

代码清单 1 使用 Protothreads 实现异步生产者/消费者线程的代码片段

```
static struct pt_sem full, empty;

static
PT_THREAD(consumer(struct pt *pt))
{
    static int consumed;
    PT_BEGIN(pt);
    for (consumed = 0; consumed < NUM_ITEMS;
        ++consumed) {
        PT_SEM_WAIT(pt, &empty);
        consume_item(get_from_buffer());
        PT_SEM_SIGNAL(pt, &full);
    }
    PT_END(pt);
}
```

代码清单 2 使用 co_await 的 C++ 异步生产者/消费者代码片段

```
task <> consumer(semaphore& sem) {
    auto producer = async_producer(sem, NUM_ITEMS);
    for co_await(auto consumed : producer) {
        consume_item(get_from_buffer());
    }
}
```

作者指出, 这两个代码清单体现出若干显著差异。与使用 Protothreads 的实现相比, 使用当前 C++ 标准化提案中 co_await 关键字的实现代码行数更少, 不再依

赖宏，也不要求把局部变量声明为 `static`。虽然这两类代码都与同步版本存在概念差异，但作者认为，基于 C++ 协程语法的写法在表达上更加透明，也更便于理解。

回顾相关工作可见，协程在资源受限平台上的探索由来已久。1992 年，Gupta 等人就在比较不同并发模型时考察了基于协程的并发模型；2000 年，Engelschall 总结了基于 `setjmp/longjmp` 的多种技术；约在 2003 年起发展的 FreeRTOS 则包含一个协程调度器，但不保存局部变量状态。2006 年，Rossetto 与 Rodriguez 描述了一种作为 TinyOS 扩展实现的新并发模型，其中以协程为整合基础，该实现属于有栈模型，并能保持局部变量状态。之后，Schimpf 为 Protothreads 增加了基于优先级的调度器；Cohen 等人为 TinyOS 提供了基于协程的调度器，用以实现类似 RPC 的接口，从而让 nesC 编写的通信代码呈现直接式编程风格；Riedel 等人则面向多平台生成 C 代码，其中一个版本利用协程实现并发；Susilo 等人用基于协程的调度器实现了“无中断的实时多任务”；Andersen 等人则拒绝在受限平台上使用 C++ futures，因为他们需要处理的是事件流而非单一事件，这会带来不可确定的内存使用，因此最终仍然选择以回调方式处理 C++ 事件。

除 C/C++ 体系外，Lua 自身就具备协程实现，并已经成功运行在微控制器上；MicroPython 作为面向微控制器的 Python 3 变体，同样支持生成器与协程。

B.2.4 编程语言：C 与 C++

根据 2017 年的嵌入式市场研究，约 78% 的嵌入式系统使用 C 或 C++ 编写。C 仍然是最流行的语言，但其占比正在缓慢下降，C++ 及其他语言的占比则在上升。2015 到 2017 年间，采用 C++ 的嵌入式项目比例从 19% 上升到 22%，而 C 的比例则从 66% 降至 56%。由于协程被提议加入的是 C++ 而非 C，因此嵌入式开发者若想使用这些特性，就必须转向 C++。作者据此认为，C++ 在嵌入式领域的使用还会继续增长，因此 C++ 协程设计必须认真考虑嵌入式软件的约束。

从 C 迁移到 C++ 并不一定意味着剧烈变化。C++ 几乎可以视为 C 的超集；只要编译器支持得当，许多嵌入式代码库甚至只需切换一个编译选项即可完成迁移。当然，这一迁移也可能带来问题，例如代码体积更大、运行更慢，或者更不利于将代码块放入 ROM。此外，一个更隐蔽的问题是：代码也许会更难接受最坏情况执行时间（WCET）分析。Goldthwaite 指出，即便采用防御性编程，动态类型转换、动态内存分配和异常机制这三方面仍可能在时序分析上带来困难。

不过，促使团队从 C 转向 C++ 的因素一直存在，例如命名空间、封装与内联函数等前端语言特性，它们在提升代码清晰度方面有明显优势，却几乎不会带来额外的代码尺寸或速度成本。若再考虑使用新的、被广泛支持的语言标准，把分阶段函数重新组织为直接式编程风格，那么对嵌入式开发者而言，迁移到 C++ 的吸引力就更强了。问题在于，这类特性能否在嵌入式系统中落地，同时又不引入 Goldthwaite 所担心的三个特征中的两个——动态内存分配与异常。

B.3 系统性映射研究

B.3.1 概览

本文的系统性映射研究遵循 Kitchenham、Kitchenham 与 Charters，以及 Petersen 等人的相关指南。研究流程如图 1 所示。作者从六个在线数据库中检索论文，这些数据库是根据相关性与可获得性选取的；检索条件要求论文标题、摘要或关键词中同时包含表 1 两组字符串中的至少一项。为保证检索完整性，作者还通过滚雪球法迭代测试，并针对不同数据库在复数形式、拼写和缩写方面的差异进行了细致处理。

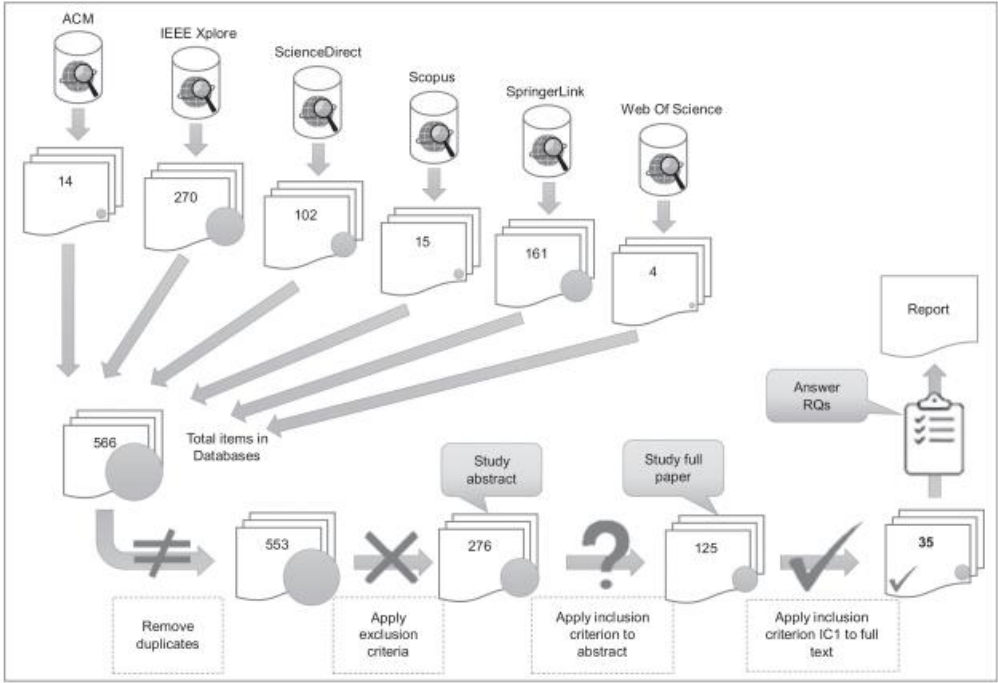


图 1 检索与筛选流程摘要

Part 1: Pattern	Part 2: Platform
coroutine OR "lightweight AND thread"	IoT OR "Internet of Things" OR "Cyber Physical Systems" OR RTOS OR "Real-time Operating Systems" OR "Embedded Systems" OR WSN OR "Wireless sensor networks" OR WSN

表 1 在线数据库检索字符串

第 1 部分（Pattern）：coroutine OR “lightweight thread”

第 2 部分（Platform）：IoT OR “Internet of Things” OR “Cyber Physical Systems” OR RTOS OR “Real-time Operating Systems” OR “Embedded Systems” OR WSN OR “Wireless sensor networks” OR WSN

B.3.2 检索过程

最主要的纳入标准是：论文必须包含关于“在资源受限平台上应用协程”的原创研究（IC1）。这一标准排除了大量只在模拟环境中使用协程来模拟资源受限平台、却并未真正部署在目标平台上的工作。

排除标准参考了既有系统综述方法学。若论文缺少 DOI 或 ISBN 等学术标识（EC1）、没有摘要（EC2）、发表于 2007 年以前（EC3）、并非英文（EC4）、评审者无法获得全文（EC5）、只是另一篇论文的早期版本（EC6）、并非一手研究（EC7），或不属于作者选定的出版类型（期刊论文、会议论文或书籍章节）（EC8），则会被排除。

作者分别在 2017 年 10 月与 2018 年 9 月对所有数据库进行了两轮检索，共得到 187 篇期刊论文、224 篇会议论文和 155 个书籍章节。这一结果促使作者把三类出版物全部纳入检索范围。作者同时决定仅保留 2007 年及之后发表的研究，这一条件大约排除了原始检索结果中的 43%。检索字符串、纳入与排除标准、具体流程以及下载脚本的详细信息，都放在了补充材料中。

B.3.3 其他系统综述与映射研究

按照 Kitchenham 与 Charters 的建议，作者首先做了一项三级研究，即先回顾该研究领域已有的综述与映射研究。结果表明，在论文写作时点上，本文似乎是第一篇系统性映射“资源受限系统（无论是嵌入式系统还是 IoT 组成系统）中协程使用情况”的研究。

B.3.4 研究问题

推动本研究的重要动机之一，是为在资源受限平台上以协程实现 `await/async` 与生成器模式奠定基础。因此，研究问题围绕以下方面展开：硬件与软件平台、开发者偏好、使用场景、预期收益以及应用程序编程接口（API）。

其中，RQ1 考察软件平台，包括编程语言、用于实现相关语言特性的具体方法以及所使用的操作系统；RQ2 考察硬件平台，包括内存大小与处理器家族；RQ3 与 RQ4 分别考察协程的使用场景和预期收益；RQ5 则聚焦编程接口本身。

Code	Research question
RQ1	What was the software platform?
RQ1a	What was the programming language used?
RQ1b	What method was used to implement coroutines?
RQ1c	What was the operating system used (if any)?
RQ2	What was the hardware platform?
RQ2a	What was the class of hardware platform?
RQ2b	How much read-only or flash memory (ROM) was available?
RQ2c	How much random-access memory (RAM) was available?
RQ2d	What was the processor family?
RQ2e	Was the processor 8-bit, 16-bit or 32-bit?
RQ2f	What was the processor's instruction set?
RQ3	What were the use cases?
RQ4	What were the intended benefits of using coroutines in this context?
RQ5	What is the API of the coroutine?
RQ5a	Does the paper discuss an implementation of coroutines?
RQ5b	Is the control flow managed on behalf of the developer?
RQ5c	Is the state of local variables automatically managed?
RQ5d	Is the coroutine implementation stackless or stackful?
RQ5e	How is the coroutine state allocated?

表 2 研究问题

RQ1: 软件平台是什么？

RQ1a: 使用的编程语言是什么？

RQ1b: 采用了什么方法来实现协程？

RQ1c: 使用了什么操作系统（如果有）？

RQ2: 硬件平台是什么？

RQ2a: 硬件平台属于哪一类？

RQ2b: 可用的只读存储器或闪存（ROM）有多大？

RQ2c: 可用的随机存取存储器（RAM）有多大？

RQ2d: 处理器属于哪个家族？

RQ2e: 处理器是 8 位、16 位还是 32 位？

RQ2f: 处理器采用什么指令集？

RQ3: 协程的使用场景是什么?

RQ4: 在这一语境下使用协程的预期收益是什么?

RQ5: 协程的 API 是怎样的?

RQ5a: 论文是否讨论了某种协程实现?

RQ5b: 控制流是否由实现代表开发者管理?

RQ5c: 局部变量状态是否自动管理?

RQ5d: 该协程实现是无栈还是有栈?

RQ5e: 协程状态是如何分配的?

通过考察硬件与软件特征, 作者希望识别一个协程实现必须工作的环境所具备的关键属性; 通过分析使用场景与期望结果, 作者希望确定成功实现所需的一些必要特征; 最后, 通过分析编程接口, 作者希望观察既有研究如何处理实现设计中的权衡。

B.3.5 有效性威胁

数据提取过程遵循 Petersen 等人关于可重复性的建议。本文结果面临多种效度威胁, 尤其集中在三个方面: 研究选择、数据提取与分类过程。

在研究选择阶段, 作者详细记录了检索过程, 并对检索字符串进行了可重复性与跨数据库一致性测试。滚雪球法是指通过递归地查找已选论文的参考文献及其被引文献来扩展检索结果。虽然最终检索过程本身未直接采用滚雪球法, 但作者在前期建立检索字符串时确实利用了它, 并据此修订了一些检索式。在应用纳入与排除标准时, 只要评审者之间出现分歧, 就会共同讨论; 同时, 他们还定期借助争议样例与随机测试论文来比较各自的判断过程。

在数据提取方面, 作者按照 Petersen 等人的建议在 Excel 中构建了统一的数据收集表, 并在整个过程中持续使用它, 以提升可重复性与准确性, 减少主观性。为提高分类一致性, 作者还让两位评审者共同检查了部分论文, 并对分类结果进行比较与讨论; 这一过程反复进行, 直到分类依据完全明确, 所有争议样例都得到解决。

B.3.6 数据集

初始检索共得到 566 条结果; 去重后剩下 553 篇文献。其中超过一半在排除标准阶段即被淘汰, 最终有 276 篇论文进入摘要审查。

在剩余论文中, 大约 55% 会在摘要层面直接因不满足纳入标准而被剔除, 留下 125 篇进入全文审查。进一步依据全文应用纳入标准后, 又有约 72% 被排除,

最终保留 35 篇研究；其中 21 篇包含对协程实现本身的讨论。图 1 的下半部分展示了这一筛选过程。作者认为，这些被选中的论文共同表明：即便缺少主流语言级支持，研究者仍然敏锐地识别到了资源受限平台对协程的需求，并通过各类原型实现展示了这些特性的潜在收益。随着 C++ 为异步编程加入原生支持，这种来自嵌入式开发者的需求很可能只会继续增强。

B.4 结果

B.4.1 概览

本研究共识别出 35 篇相关论文，其中 21 篇讨论了协程实现，涉及 7 种不同的编程语言。更细粒度的结果列表可参见补充材料。

B.4.2 编程语言

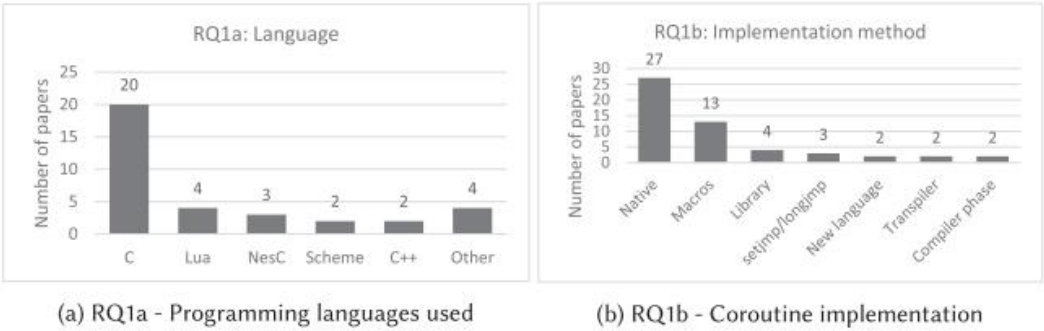


图 2 编程语言与协程实现方式结果

如图 2(a) 所示，C 是占主导地位的编程语言。在全部研究中，有 20 篇（57%）使用 C；再加上使用相关语言 C++ 与 NesC 的 5 篇（14%），类 C 语言占据了绝对主流。除此之外，Lua 是出现次数第二多的语言，共有 4 篇论文采用。

B.4.3 协程实现方式

在协程实现方式上，27 篇论文（77%）采用了原生方式，也就是尽量避免依赖需要新增或修改工具链的技术。在这些原生实现中，13 篇使用了宏，其中 7 篇基于 Duff's device；4 篇使用库；还有 3 篇使用了 C 语言中的 setjmp/longjmp 机制。

与此同时，也有若干研究选择扩展工具链或创建新工具：两篇论文提出了新语言，一篇论文给出了语言扩展；两篇论文使用了从一种语言到另一种语言的转译器；另有研究分别通过预编译器与新的编译优化阶段来支持协程。还有两项工作借助其他语言来实现协程：一项工作在 Lua 中通过 C API 直接操纵宿主环境，另一项则使用了不可移植的汇编语言。总体而言，这些结果表明，资源受限平台上的协程需求非常真实，但在缺乏统一语言支持时，开发者不得不诉诸各种各样的工程手段。

B.4.4 操作系统

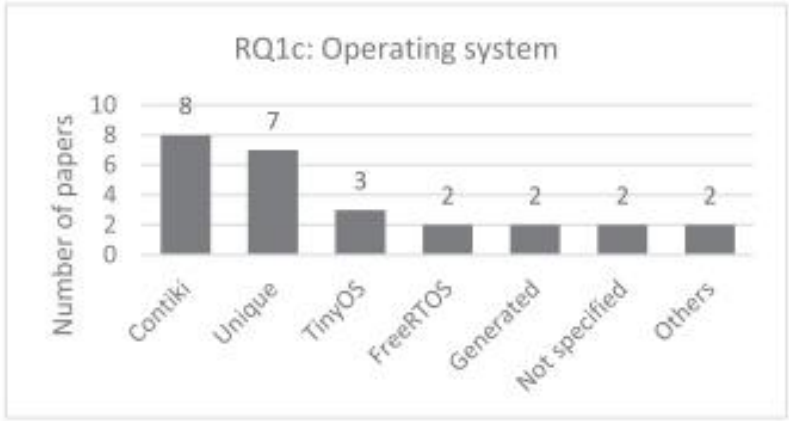


图 3 RQ1c: 所选类 C 语言研究使用的操作系统

在 26 个以 C、C++ 或 NesC 编写的案例中，13 个（50%）使用或扩展了广为人知的嵌入式操作系统，如 Contiki、TinyOS 与 FreeRTOS；另有 9 个（35%）使用独特操作系统，或者为每个应用自动生成操作系统。由于论文中提供的信息有限，作者无法进一步判断这 9 项研究中究竟有多少可以被视为真正的“裸机”方案。

B.4.5 存储器

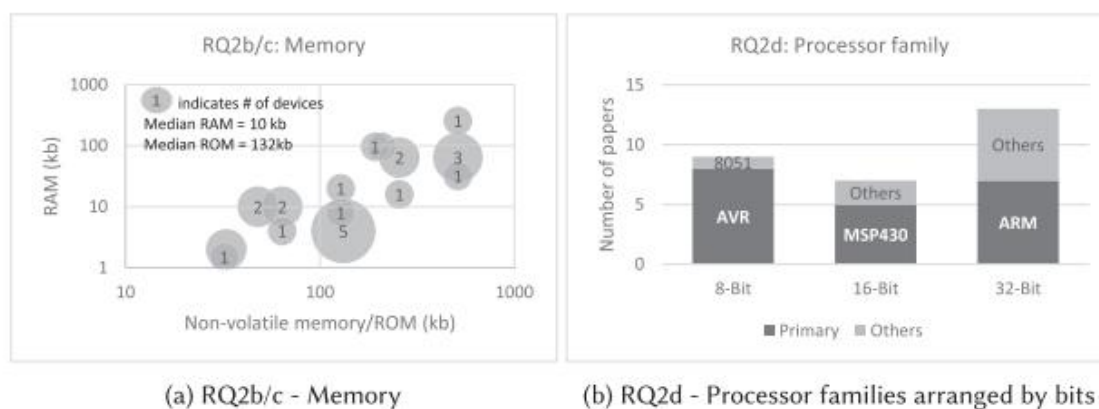


图 4 硬件结果

图 4(a) 用对数坐标展示了所选平台的 ROM 与 RAM 规模。就 RAM 而言，作者观察到许多系统的容量都非常小，其中位数仅为 10 KB。ROM 与 RAM 的大小之间呈正相关，相关系数为 0.64。

B.4.6 处理器

在识别出的 29 个 CPU 中，只有 13 个（45%）是 32 位处理器；另外还有 9 个 8 位处理器和 7 个 16 位处理器。也就是说，55% 的研究使用了 8 位或 16 位设备，这直接说明协程并不仅适用于性能较高的平台，也同样适用于极端受限的目标设备。

进一步看处理器家族，在 8 位处理器中，除了一个例外之外，其余都属于 megaAVR 家族；在 16 位处理器中，有 5 个使用 TI MSP430 架构；而在 32 位处理器中，略高于一半采用 ARM 架构。这些类型的微控制器恰恰是 IoT 与嵌入式系统中最常见的处理平台之一，图 4(b) 对这些结果进行了总结。

B.4.7 使用场景

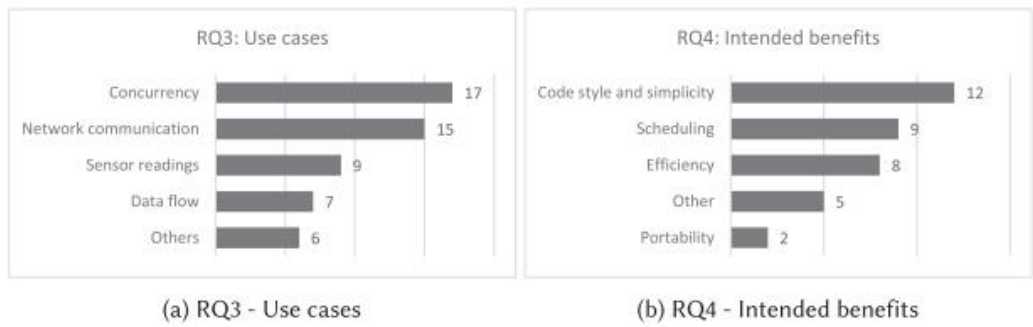


图 5 使用场景结果

最常见的四类使用场景分别是：并发（占全部论文的 49%）、网络通信（43%）、传感器读数（26%）以及数据流（20%），如图 5(a) 所示。值得注意的是，这四类场景通常都被视为程序员较难处理、容易导致实现复杂化的问题。

这些使用场景并不是资源受限设备独有的，但与桌面软件相比也呈现出明显差异。桌面环境中常常推动协程需求增长的用户界面场景在本研究中几乎缺席，而传感器读数这类在桌面系统中较少出现的需求，在资源受限平台上却非常突出。

B.4.8 预期收益

在预期收益方面，出现频率最高的三个类别分别是：（1）代码风格与简洁性（34%）；（2）调度（26%）；（3）效率（23%），图 5(b) 给出了汇总结果。

鉴于作者先前已经指出分阶段编程会导致易错且难维护的代码，因此“代码风格与简洁性”位居首位并不令人意外。相比之下，“调度”作为协程实现的重要收益，在主流桌面编程中并不常见，因此在 C++ 标准制定过程中未必会被放在很高优先级上；但对于许多嵌入式应用而言，协程正是构建自定义调度器的有力工具。效率这一收益出现频率也很高，则反映了嵌入式系统本身对延迟的严格要求。

B.4.9 应用程序编程接口

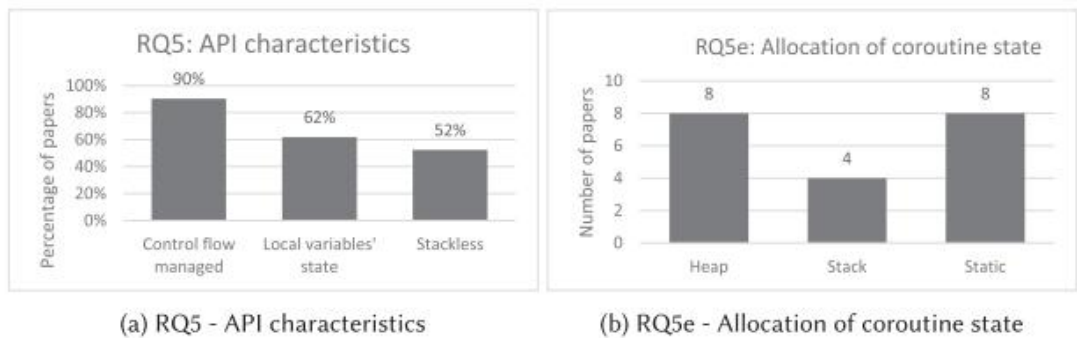


图 6 API 结果

在 35 篇被分析的论文中，有 21 篇讨论了某种协程实现；围绕 RQ5 的问卷结果列在补充材料中。

对于 API 相关问题（RQ5b 至 RQ5e），作者并非总能仅靠论文正文直接得到答案。在这种情况下，除非可以从补充材料、链接源代码或研究者熟知的背景中确定，否则答案会被标记为“未知”。有些研究引用的源代码如今已经无法获取。

图 6(a) 汇总了那些确实讨论协程实现的研究在 API 层面的基本特征。绝大多数实现（89%）都会代表程序员管理控制流；超过三分之二的实现会自动管理局部变量状态。至于无栈与有栈实现，分布则相对均衡：11 个无栈实现，对应 8 个有栈实现。

从内存分配角度看，图 6(b) 显示，在 21 个实现中，超过三分之一（8 个）支持在堆上分配协程状态；另有 4 个使用栈分配。但对于嵌入式系统而言，如果状态规模较大或编译期无法确定，堆分配与栈分配都可能并不理想。

B.5 分析与讨论

B.5.1 API 设计分析

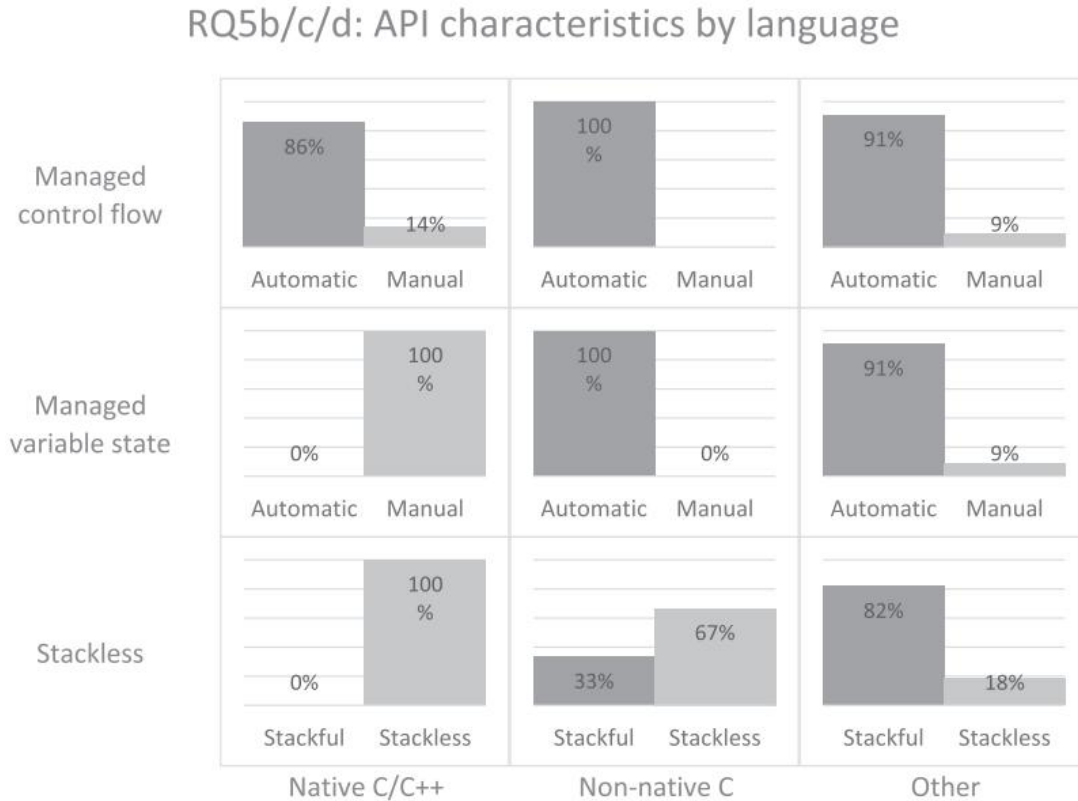


图 7 按语言分组的 API 特征（RQ5b/c/d）

图 7 按三类实现对 API 特征进行了分组分析：（1）原生 C/C++；（2）非原生 C；（3）非 C 语言。这里所谓“非原生 C”是指依赖语言扩展、转译器或其他会扩展 C 编译工具链的方案。作者之所以特别关注“非原生 C”与“非 C”两类实现，是因为它们更可能反映语言设计者和实现者认为哪些特征值得被纳入正式设计。

本文认为，“由实现代表开发者管理控制流”（RQ5b）是资源受限平台上理想编程语言的重要特征，而图 7 的结果基本支持这一观点：所有非原生 C 实现和几乎所有非 C 实现都提供了这一能力；原生 C 中也有 86% 的案例能够做到这一点，主要依赖宏技术。

作者同样认为，“自动管理协程局部变量状态”（RQ5c）是一个理想特征。结果再次表明：所有非原生 C 实现以及几乎所有非 C 实现都支持这一能力；而原生 C 实现由于语言本身的限制，基本无法提供这种支持。

就有栈与无栈而言，原生 C 实现中没有一个是具有栈的，非原生 C 实现中也只有一个属于有栈；相反，非 C 实现中有 82% 都是有栈的。作者认为，这并不意

味着 C 开发者不需要有栈协程，而是说明在 C/C++ 社区内部，出于内存和速度成本的顾虑，开发者依然对无栈协程保持强烈支持。

协程状态如何分配，是决定其能否适用于资源受限平台的关键设计问题。对于作者能够识别分配方式的 16 个实现而言，接近三分之一使用对象或结构体保存状态；44%（7 个实例）要求将状态放在静态（全局）内存中；1 个仅依赖栈；另有 3 个实现允许更灵活地选择状态存放位置。

有 5 项研究要求将状态存放在堆中。其中 3 项使用 Java、Scheme 或 Lua 这类本身就要求如此的语言，真正基于 C 或 C++ 的只有 2 项。以 Cohen 等人的工作为例，每个协程栈大小固定为 256 字节，并在堆上分配；但所需协程栈总数是预先已知的，因此仍可采用安全的分配策略。

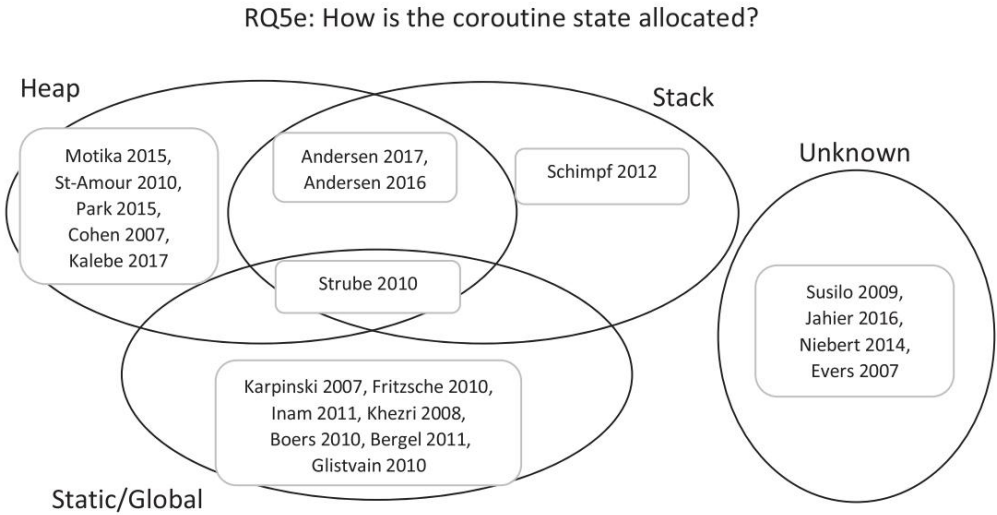


图 8 RQ5e: 协程状态如何分配？

综合来看，主流 C++ 面向的环境往往拥有相对充足的堆内存，因此标准协程实现必须支持用动态内存保存协程状态；但对嵌入式等资源受限平台而言，开发者还必须有能力选择使用栈内存或全局静态内存，并能够精确控制每一次实例化时使用哪一种策略。因此，理想的协程实现应同时支持这三种分配方式，并允许开发者进行显式控制。

B.5.2 研究空白

Article	Language was C/C++	Predictable memory usage	Integrates with other language features	Maintainability & readability	Labour cost of implementing the infrastructure	Memory and processing cost of infrastructure	Continued use of legacy code
Cohen et al. (2007)		✓✓	✓✓	✓✓	✓✓	✓✓	
Evers et al. (2007)		✓				✓✓	
Karpinski and Cahill (2007)		✓✓	✓✓	✓✓	✓✓	✓✓	✓✓
Kumar et al. (2007)	✓✓	✓				✓✓	✓✓
Khezri et al. (2008)			✓✓			✓	✓✓
Susilo et al. (2009)	✓✓					✓✓	
Boers et al. (2010)	✓✓		✓			✓✓	
Fritzsche and Siemers (2010)	✓✓		✓✓				
Glistvain and Aboelaze (2010)	✓✓	✓✓			✓✓	✓✓	
St-Amour and Feeley (2010)			✓✓	✓✓	✓✓	✓✓	
Strube et al. (2010)	✓✓	✓✓	✓				
Bergel et al. (2011)		✓✓		✓✓		✓✓	
Inam et al. (2011)	✓✓				✓	✓✓	✓
Schimpf (2012)	✓✓		✓	✓	✓✓	✓✓	✓
Niebert and Caralp (2014)	✓✓	✓✓				✓✓	
Motika and von Hanxleden (2015)			✓✓	✓		✓✓	
Park et al. (2015)		✓✓	✓✓	✓✓		✓✓	✓
Andersen et al. (2016)		✓✓	✓✓	✓✓	✓✓	✓✓	✓✓
Jahier (2016)		✓✓	✓✓		✓✓		✓✓
Andersen et al. (2017)		✓✓	✓✓	✓✓	✓✓	✓✓	✓✓
Kalebe et al. (2017)	✓✓		✓✓	✓✓	✓✓		
<i>Ideal outcome</i>	✓✓	✓✓	✓✓	✓✓	✓✓	✓✓	✓✓

Key: ✓✓ - The issue is considered and resolved. ✓ - The issue is addressed but not resolved.
Blank - The issue is not present.

表 3 研究空白总结

如果只关注那些真正描述了协程实现的研究，可以发现作者进一步从“这些工作解决了什么问题”这一角度做了研究空白分析。结果表明，大多数研究都会讨论协程基础设施带来的内存开销与计算开销，而较少有作者认真处理与遗留代码继续协作的问题。

对于嵌入式系统而言，可预测的内存使用是特别重要的问题。虽然 21 篇论文中有 10 篇给出了某种解决办法，但这些方案都无法直接适用于一种既是原生

C++ 方案、又能够自动管理局部变量状态的实现。作者据此得出结论：围绕“标准 C++ 是否适合作为资源受限设备上的异步程序开发语言”，目前仍存在明显研究空白。

B.5.3 检索结果的可重复性

作者在重复检索 IEEE Xplore 数据库时发现，新结果并不像预期那样是旧结果的超集。2017 年 10 月检索得到的 144 篇文章，在 11 个月后再次检索时只剩下 87 篇（60%）仍会出现在结果中；另一方面，新出现的 32 篇结果中，又只有一半发表于 2015 年之后，其余竟然更早。作者因此推断，IEEE 数据库的检索机制在两次检索之间发生了变化，这对系统性综述使用该数据库提出了方法学上的警示。其他在线数据库中则没有观察到这一问题。

B.5.4 讨论

大多数入选论文使用的是 C 语言。虽然当前协程提案针对的是 C++，但这并不意味着相关项目若转向 C++ 就必须经历大规模代码重写。

超过四分之一的论文使用了 Contiki 操作系统，而 Contiki 又通过基于 Duff's device 的 Protothreads 提供协程支持。考虑到第 2.3 节已经指出了 Duff's device 的多个问题，这一现象恰恰说明：即便存在缺陷，嵌入式开发者依然迫切需要一种能够提供类似协程设施的解决方案。

被选中的研究中，有超过一半使用了 8 位或 16 位处理器。当前主流 C++ 编译器对这类平台的支持仍然有限；如果能让 C++ 协程真正进入最小型平台，这一现实问题就必须被正视。

与嵌入式市场调查的结论一致，代码风格与简洁性是协程实现最常被期待的收益。紧随其后的收益是“为调度器提供基础”，这在主流平台上通常并不被视为协程的首要优势，因此值得进一步研究。

在资源受限设备上，通信、数据流与传感器读数这三类常见使用场景都往往需要借助分阶段编程来完成，因此编程难度较高。协程所支持的 `async/await` 与生成器模式恰好能够以直接式编程风格来表达这些逻辑，从而有望降低开发工作量并减少错误发生。本文调研中这类使用场景的高频出现，进一步表明它们是非常重要的、也非常值得继续研究的应用目标。

本次调研还表明，在资源受限设备上，确实存在大量对“基于协程的并发编程设施”的需求，而不仅仅是在高性能平台上才如此。作者注意到，只要语言支持对内存分配方式进行精细控制，研究者通常就会避免使用堆内存来保存协程状态

或协程栈。这意味着，对于构成本次调研主体的小型设备而言，“避免堆分配”几乎可以视为一项现实需求。

虽然 82% 的非 C 实现采用了有栈模型，但 C 实现中只有一个是有栈的。作者据此观察到：当语言从一开始就以协程为目标进行设计时，有栈实现相当常见；但在 C 语言中，如果还要同时保持向后兼容和可接受的内存开销，那么有栈实现会变得困难得多。因此，面向 C 程序员的设计中，无栈协程仍然具有重要地位，而这又与研究对象平台本身内存稀缺的现实密切相关。

更关键的是，作者没有找到任何一项工作能在 C 或 C++ 中提供这样一种协程实现：它既能自动管理变量状态，又是专门为资源受限平台上的事件驱动环境而设计的。作者因此认为，这构成了一个重要研究空白，也说明围绕此类实现继续开展工作是有必要的。

虽然本次调研中使用 C 的论文远多于使用 C++ 的论文，但市场调查表明，在资源受限设备上从 C 向 C++ 迁移的趋势已经出现。为了获得对 `async/await`、生成器模式与轻量级调度的整洁支持，开发者也可能因为 C++20 协程提案而主动转向 C++。不过，这种转向只有在语言与库层面提供真正适合资源受限设备的支持时才有意义。特别是，实现者应认真考虑如何避免 Goldthwaite 所指出的两个高风险特征：动态内存分配与异常。若能进一步证明协程实现可以提供编译期即可确定的、确定性的内存占用，就有希望完全规避动态分配。

综观既有工作，嵌入式系统为了解决分阶段代码带来的开发难题，已经尝试了多种专门化方案，包括 `Protothreads`、预编译器、语言扩展、编译后优化阶段以及不可移植的代码库。这说明，一方面协程对这类设备的软件开发确有许多价值；另一方面，要在真正资源受限的设备上把协程做好，又并非易事。相比之下，在主流企业系统中为 C++ 实现协程往往相对直接，因为这些环境拥有更充足的内存、操作系统设施与标准库支持。正因为如此，把协程适配到资源受限设备虽然更困难，却也更有价值，因为低成本、低功耗调度、通信与传感器管理等任务，本来就与物联网应用高度契合。

B.6 结论与未来工作

B.6.1 结论

本文系统梳理了学术界关于“在嵌入式系统中使用异步编程技术，尤其是基于协程的方式”的现有工作。作者据此得出结论：资源受限设备对这些设施确实存在显著需求，因此在 C++ 协程标准化的讨论中，嵌入式系统不应被忽视。C++ 协程提案确实为改善嵌入式系统的软件工程质量提供了机会，但前提是这些语言设施必须能够在极端资源受限的环境中真正可用。

B.6.2 未来工作

未来工作可以从以下几个方向展开。

(1) 考察 N4680 提案能否提供确定性的内存使用，并允许开发者充分控制分配细节；若不能，则需要明确标准层面应当作出哪些修改。同时，也应测试当前实现（如 Microsoft C++ 14.1 与 LLVM 7.0.1）是否能够提供这种确定性与可控性。

(2) 考察 N4680 提案及其现有实现能否在资源受限平台的事件驱动环境中有效工作，并进一步区分“带实时操作系统”和“不带实时操作系统”两种情况。

(3) 系统测量当前 N4680 实现在资源受限平台上的内存开销与性能开销，尤其是在操作系统支持极少甚至没有操作系统支持时的表现。

B.A 补充材料

本文的补充材料可在论文在线版本中获取。调研所使用的数据也公开托管在 GitHub 仓库中，原文给出的地址为：<https://github.com/bbelson2/supplementary-materials>。

书面翻译对应的原文索引

- [2] Belson B, Holdsworth J, Xiang W, Philippa B. A Survey of Asynchronous Programming Using Coroutines in the Internet of Things and Embedded Systems[J]. ACM Transactions on Embedded Computing Systems, 2019, 18(3): Article 21.

致 谢

衷心感谢向勇老师在完成毕业设计过程中对我的指导、鼓励和严格要求，让我能够最终完成相应的毕设任务，交出一份答卷。

感谢我的家人，尤其感谢我的父母、舅舅在背后对我的支持以及各种帮助，感谢我的外婆外公、表弟一直以来的陪伴。家人的温暖与期盼，是我不断前行的后盾。

感谢本科几年遇到的优秀老师、助教，感谢你们的答疑解惑、包容理解。感谢遇到的优秀同学们，一起学习一起进步。这些回忆是我面对未来人生的宝贵财富。

感谢清华大学对我的教育，感谢清华校园的环境和氛围。未来我将以自己的行动，践行“自强不息，厚德载物”的校训。

声 明

本人郑重声明：所呈交的综合论文训练论文，是本人在导师指导下，独立进行研究工作所取得的成果。尽我所知，除文中已经注明引用的内容外，本论文的研究成果不包含任何他人享有著作权的内容。对本论文所涉及的研究工作做出贡献的其他个人和集体，均已在文中以明确方式标明。

签 名： 黄昊颖 日 期： 2026-05-28

综合论文训练记录表

学生姓名	黄昊颖	学号	2020010924	班级	计 03
论文题目	Rust 异步驱动模块设计与实现				
主要内容以及进度安排	主要内容: 在 rCore-N 操作系统中实现基于 Rust Future 的定时器驱动 进度安排: 2025 年 12 月-2026 年 1 月上旬: 调研确定选题, 完成 rCore-2025S 课程 5 个实验 2026 年 1 月上旬: 开题答辩 2026 年 1 月下旬: 学习 Embassy 异步框架中关于 Executor 的实现 2026 年 2 月上旬: 学习 Embassy 异步框架中关于 Timer 的实现 2026 年 2 月下旬: 配置环境, 在 QEMU 中运行 rCore-N 内核并跑通用户态测例 2026 年 3 月上旬: 在 rCore-N 中设计实现 Executor、TaskRef、Waker 数据结构 2026 年 3 月下旬-4 月上旬: 参考 Embassy 设计实现 rCore-N 异步 timer 驱动 2026 年 4 月下旬: 编写用户态 sleep 测试用例验证 rCore-N 异步定时器正确性 2026 年 5 月-6 月: 代码工作收尾。写论文, 准备答辩 2026 年 6 月: 进行毕设答辩 指导教师签字: <u>何原</u> 考核组组长签字: <u>樊海子</u> 2026 年 1 月 16 日				
中期考核意见	论文工作按照原计划进行, 中期检查答辩时陈述清晰, 正确回答问题。通过中期考核。 考核组组长签字: <u>樊海子</u> 2026 年 4 月 4 日				

指导教师评语	黄昊颖同学在毕设工作中，首先对 Rust 异步编程机制和异步运行时 embassy 中的协程调度模块进行了分析，形成较详细的分析文档；然后在 rCore-N 内核上实现了定时器的异步驱动原型；最后通过功能验证性实验，验证了定时器异步驱动的基本功能是可用的。论文工作表示，黄昊颖同学基本掌握计算机专业的知识和技能，论文组织合理，初步达到本科毕设要求。 指导教师签字：何勇 2026 年 06 月 02 日
评阅教师评语	论文以 rCore-N 为实验平台，设计并实现了 Rust 异步驱动模块，工作具有一定的现实应用价值。论文首先分析了已有异步机制及相关 timer 实现，然后在 rCore-N 中设计并实现了 timer 原型模块，最后通过实验验证了设计与实现的可行性。论文工作为后续在 rCore-N 中继续探索异步驱动、异步 I/O 和多核异步调度机制提供了基础。论文结构合理，论述清楚，达到了综合论文训练的要求。 评阅教师签字：朱传良 2026 年 06 月 02 日
答辩小组评语	论文完成了原定工作计划，答辩时陈述清晰，达到了综合论文训练要求，通过答辩。 答辩小组组长签字：樊海平 2026 年 6 月 5 日

总成绩：B+

教学负责人签字：李丹

2026 年 6 月 11 日